LIC. EN **INTELIGENCIA ARTIFICIAL — LIC-EN-IA**



MATERIA

Programación para Inteligencia Artificial

PIA · PRIMER CUATRIMESTRE

UBP UNIVERSIDAD Blas Pascal

CONTACTO
0810 1223 3827
informes@ubp.edu.ar

WEB Y REDES
www.**ubp.edu.ar**

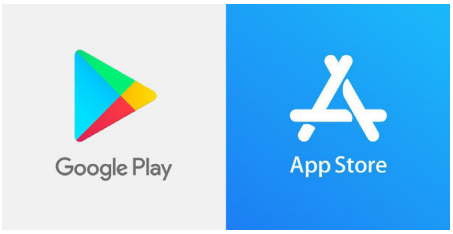
Códigos QR

Estimado Alumno:

Esta materia puede contener **Códigos QR** para agilizar el acceso a los contenidos audiovisuales que el docente ha desarrollado especialmente para Usted.

Para poder visualizar los videos utilice el lector de código QR que trae por defecto su teléfono o dispositivo móvil. En caso de no disponer de uno, puede descargarlo desde *Google Play* si su sistema operativo corresponde a Android o desde la *App Store* si utiliza IOS.





Una vez instalado el software en su dispositivo móvil:

1) Abra la aplicación
2) Apunte con la cámara de su dispositivo hacia los códigos QR que encuentre en el interior de su materia.



De esta manera estará visualizando los videos que el docente ha desarrollado y/o seleccionado especialmente para usted.

Índice

Bienvenida 4

Propósito 4

Objetivos 6

Competencias 6

Contenidos mínimos 8

Contenidos 8

Bibliografía 9

Planificación 9

Metodología 10

Evaluación 11

Mapa conceptual 12

Módulos

- *Módulo 1* 13
- *Módulo 2* 43
- *Módulo 3* 78
- *Módulo 4* 121

Importante

Lecturas básicas y complementarias disponibles desde plataforma.

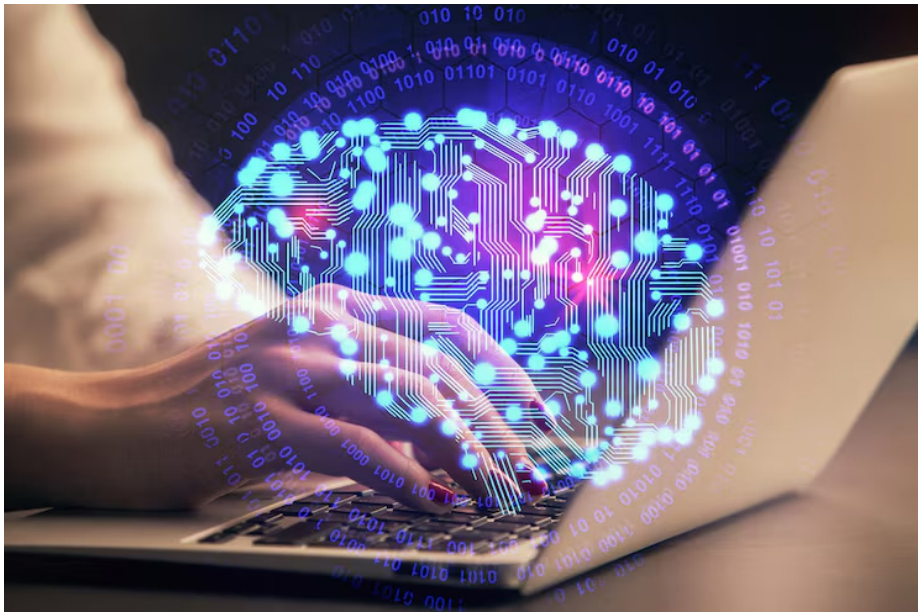
Impresión total del documento161 páginas

Bienvenida

Bienvenidos a la materia:

"Programación para Inteligencia Artificial"

Propósito



Fuente: Freepik

PROPÓSITO

¡Bienvenidos y bienvenidas a Programación para Inteligencia Artificial!

Esta asignatura tiene como propósito llevarlos/as desde los fundamentos de la programación hasta la construcción de soluciones reales basadas en inteligencia artificial. A lo largo de este recorrido, aprenderán Python como lenguaje principal, no solo en su sintaxis básica, sino como una herramienta poderosa para resolver problemas complejos en el campo de la IA.

Comenzaremos con los conceptos esenciales de programación: variables, tipos de datos, operadores y estructuras de control. Estos son los pilares sobre los cuales se construye todo lo demás. A partir de allí, avanzaremos hacia las estructuras de datos, tanto lineales como listas, pilas y colas, como no lineales tales como árboles y grafos. Estas últimas son especialmente importantes en el contexto de la inteligencia artificial, ya que muchos algoritmos de IA se basan en ellas para representar y procesar información.

Continuaremos con la programación orientada a objetos, un paradigma fundamental para organizar código de manera modular y escalable. Trabajaremos con clases, objetos, herencia y polimorfismo, herramientas que les permitirán modelar sistemas complejos de forma clara y eficiente.

Luego nos adentraremos en el manejo de datos con bibliotecas especializadas como NumPy y Pandas. Aprenderán a leer, escribir, transformar y manipular datos de manera profesional, habilidades indispensables para cualquier proyecto de IA. El preprocesamiento de datos es una etapa crucial en el desarrollo de modelos de aprendizaje automático, y estas bibliotecas son la base de ese proceso.

Finalmente, llegaremos al corazón de la materia: las bibliotecas de inteligencia artificial. Comenzaremos con Scikit-learn para machine learning tradicional, donde aprenderán a construir modelos de clasificación y regresión. Luego avanzaremos hacia frameworks de deep learning como TensorFlow y PyTorch, con los cuales podrán diseñar y entrenar redes neuronales para resolver problemas más complejos.

El enfoque de la materia será eminentemente práctico. La programación se aprende programando, y por eso cada tema estará acompañado de numerosos ejemplos de código, ejercicios y actividades que les permitirán aplicar lo aprendido. Dado que esta es una carrera a distancia, hemos desarrollado el material de manera exhaustiva y completa, con explicaciones detalladas, consejos para abordar los temas más desafiantes y recursos para que puedan practicar en sus computadoras.

Cada uno de estos contenidos constituye un escalón necesario en su formación como profesionales de la inteligencia artificial. Al finalizar esta materia, contarán con las herramientas fundamentales para comenzar a desarrollar sus propias soluciones de IA, transformando problemas del mundo real en modelos de aprendizaje automático que generen valor.

¡Comencemos este viaje hacia la programación para la IA!

Objetivos

- Comprender los fundamentos de la programación en Python, incluyendo sintaxis básica, tipos de datos, operadores y estructuras de control, para saber desarrollar programas que resuelvan problemas aplicados a la inteligencia artificial.

- Implementar estructuras de datos lineales y no lineales, tales como listas, pilas, colas, árboles y grafos, con el propósito de organizar y manipular información de manera eficiente en contextos de algoritmos de IA.

- Aplicar los principios de la programación orientada a objetos, trabajando con clases, objetos, encapsulación, herencia y polimorfismo, para comprender cómo modelar sistemas complejos y desarrollar soluciones modulares y escalables.

- Utilizar bibliotecas especializadas de Python como NumPy y Pandas para el manejo, lectura, escritura y transformación de datos, sentando las bases necesarias para el preprocesamiento de información en proyectos de inteligencia artificial.

- Explorar e integrar bibliotecas fundamentales de IA, iniciando con Scikit-learn para machine learning tradicional y avanzando hacia frameworks de deep learning como TensorFlow y PyTorch, para construir, entrenar y optimizar modelos de aprendizaje automático que resuelvan problemas reales de inteligencia artificial.

Competencias

Relación de la asignatura con las competencias de egreso

En la tabla siguiente se establece la relación de la asignatura con las competencias de egreso: Específicas, Genéricas Tecnológicas, Sociales, Políticas y Actitudinales de la carrera. Se incluyen las competencias de egreso a las que tributa su nivel de aporte (ninguno, bajo, medio y alto)

Competencias Genéricas Tecnológicas (CG)	Nivel
CG1. Identificación, formulación y resolución de problemas de informática.	Alta
CG2. Concepción, diseño y desarrollo de proyectos de informática.	Alta
CG3. Gestión, planificación, ejecución y control de proyectos de informática.	Media
CG4. Utilización de técnicas y herramientas de aplicación en la informática.	Alta
CG5. Generación de desarrollos tecnológicos y/o innovaciones tecnológicas.	Alta

Competencias Sociales, Políticas y Actitudinales (CSPA)	Nivel
CSPA6. Desempeño en equipos de trabajo.	Alta
CSPA7. Comunicación efectiva.	Alta
CSPA8. Acción ética y responsable.	Alta
CSPA9. Evaluación y actuación en relación con el impacto social de su actividad en el contexto global y local.	Alta
CSPA10. Aprendizaje continuo.	Alta
CSPA11. Acción emprendedora	Alta

Competencias Específicas (CE)[1]	Nivel
CE 1.1 SI Especificación, proyección y desarrollo de sistemas de información.	Media
CE 1.2 SCD Especificación, proyección y desarrollo de sistemas de comunicación de datos.	Media
CE 1.3 S Especificación, proyección y desarrollo de software.	Alta
CE 2.1 SI Proyección y dirección de lo referido a seguridad informática.	Media
CE 3.1 M Establecimiento de métricas y normas de calidad de software.	Media
CE 4.1 C Certificación del funcionamiento, condición de uso o estado de sistemas de información, sistemas de comunicación de datos, software, seguridad informática y calidad de software.	Media
CE 5.1 D Dirección y control de la implementación, operación y mantenimiento de sistemas de información, sistemas de comunicación de datos, software, seguridad informática y calidad de software.	Media

Resultados de Aprendizaje (RA)

RA N°	Resultado de Aprendizaje
RA1	Comprender los conceptos básicos de programación para desarrollar aplicaciones ejecutables en diferentes plataformas de software y hardware.
RA2	Comprender los conceptos básicos de multimedia y su aplicación en entornos interactivos para diseñar y desarrollar interfaces de usuario interactivas e inteligentes considerando la experiencia de este dentro de ese entorno.
RA3	Comprender los conceptos básicos de bases de datos y su aplicación en entornos interactivos para implementar técnicas de seguridad de datos dentro de los entornos interactivo e inteligentes.
RA4	Utilizar herramientas de programación para crear y mejorar el funcionamiento de entornos interactivos e inteligentes
RA5	Analizar el comportamiento de los usuarios de los entornos interactivos e inteligentes para utilizar estándares de programación para asegurar la calidad de los productos.

Contenidos mínimos

- Fundamentos de programación: Sintaxis y estructuras básicas (Python).
- Estructuras de datos: Listas, pilas, colas, árboles, grafos.
- Programación orientada a objetos: Clases, herencia, encapsulación, polimorfismo.
- Manejo de datos: Lectura, escritura y manipulación de datos (pandas, NumPy).
- Introducción a bibliotecas de IA: Scikit-learn, TensorFlow, PyTorch.

Contenidos

Módulo 1: Fundamentos de programación en Python

- ¿Qué significa programar? La programación como herramienta para resolver problemas.
- Concepto de algoritmo: características y ejemplos.
- Instalación y configuración de Python. Uso de un IDE o entorno interactivo.
- Primer programa: "Hola Mundo".
- Variables y tipos de datos básicos.
- Operadores aritméticos, relacionales y lógicos.
- Estructuras condicionales y repetitivas.
- Funciones: definición, parámetros y retorno de valores.

Módulo 2: Estructuras de datos

- Listas avanzadas y métodos de manipulación
- Tuplas, diccionarios y conjuntos en Python
- Pilas (Stack) y su uso en algoritmos
- Colas (Queue) para procesamiento secuencial

Módulo 3: Programación Orientada a Objetos

- Clases y objetos: definición, atributos y métodos.
- Características de la POO: encapsulamiento, herencia y polimorfismo.
- Árboles binarios y recorridos fundamentales.
- Grafos y su aplicación en problemas de IA.

Módulo 4: Manejo de datos y bibliotecas de IA

- NumPy: arrays y operaciones básicas.
- Pandas: Series, DataFrames, lectura, escritura y manipulación de datos.
- Transformación y limpieza de datos.
- Scikit-learn: preprocesamiento, modelos de clasificación y regresión.
- TensorFlow y Keras: construcción de redes neuronales básicas.
- PyTorch: tensores, construcción y entrenamiento de modelos.

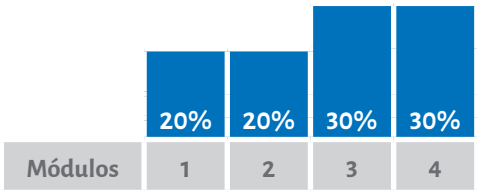
Bibliografía

Obligatoria

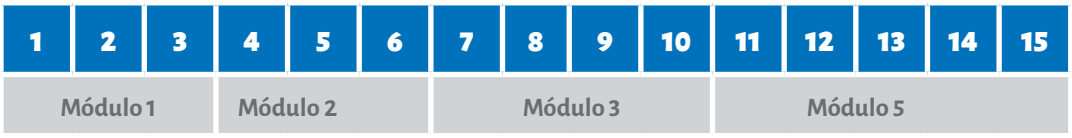
Material didáctico de producción institucional diseñado y desarrollado especialmente para cada módulo.

Planificación

Porcentaje estimativo por módulo según la cantidad y complejidad de contenidos y actividades:



Representación de porcentajes en semanas:



Primera parte de la Evaluación Integradora
Segunda parte de la Evaluación Integradora

Metodología

A) Estrategias de enseñanza: Este espacio curricular fue diseñado según lo previsto en el Sistema Institucional de Educación a Distancia (SIED) de la UBP (Resolución MECCYT 197/2019), las características de los destinatarios, las necesidades de esta disciplina, competencias que se espera promover en esta asignatura y el perfil del egresado.

En relación con lo anterior, para llevar a cabo la enseñanza se cuenta con:

- Contenidos y actividades de aprendizaje desarrollados en la plataforma virtual miUBP.
- Material bibliográfico especialmente seleccionado.
- Clases en video, en las que el docente expone contenidos que considera relevantes y que pueden ser visualizadas en cualquier momento.
- Tutorías virtuales que implican diferentes espacios de interacción.

B) Materiales curriculares: Durante el cursado, se fomenta el estudio de los contenidos (presentados en diferentes soportes y lenguajes) que componen el programa, y la resolución de las actividades de aprendizaje. De esta forma, se introduce al alumno en los conceptos que se deben dominar en todo lo que respecta a Programación para la Inteligencia Artificial. También, se invita a leer bibliografía que enriquece y profundiza dicho desarrollo. Por otra parte, se propone la resolución de actividades de aprendizaje tanto de carácter optativo como obligatorio que se encuentran debidamente identificadas en la plataforma. Algunas de ellas se centran en la apropiación de conceptos teóricos básicos de la asignatura y otras en la transferencia mediante un abordaje práctico a través del análisis de casos, ejercitaciones y situaciones problemáticas.

C) Modalidad de agrupamiento: Los alumnos pueden optar por agruparse libremente para estudiar la materia y resolver las actividades de aprendizaje planteadas, de forma individual o grupal. En este sentido, en esta asignatura se propone:

a) Trabajo individual, de modo que el alumno pueda desenvolverse en forma autónoma, reconociendo sus fortalezas y debilidades, a la vez que puede desarrollar sus propias estrategias de aprendizaje.

b) Trabajo colaborativo, de modo que se favorezca el desarrollo de las competencias del trabajo en equipo para resolver eficientemente las futuras situaciones a las cuales puede enfrentarse como profesional.

D) Interacción: La comunicación con el docente tutor se realiza a través de los diferentes medios disponibles en la plataforma miUBP.

E) Formación práctica: En lo que se refiere a la formación práctica se hace foco en la resolución de problemas que implique el uso de los contenidos estudiados en la asignatura contextualizados en situaciones que tengan que ver con la carrera.

Evaluación

La metodología de evaluación propuesta se presenta conforme al Reglamento Académico General de la UBP.

Cada instrumento de evaluación contiene sus criterios de evaluación explicitados y el puntaje otorgado a cada consigna.

En cuanto a los criterios de acreditación, la escala cuantitativa utilizada para evaluar es del 1 al 100 y el puntaje mínimo que se requiere para la aprobación es de 50.

Regularidad:

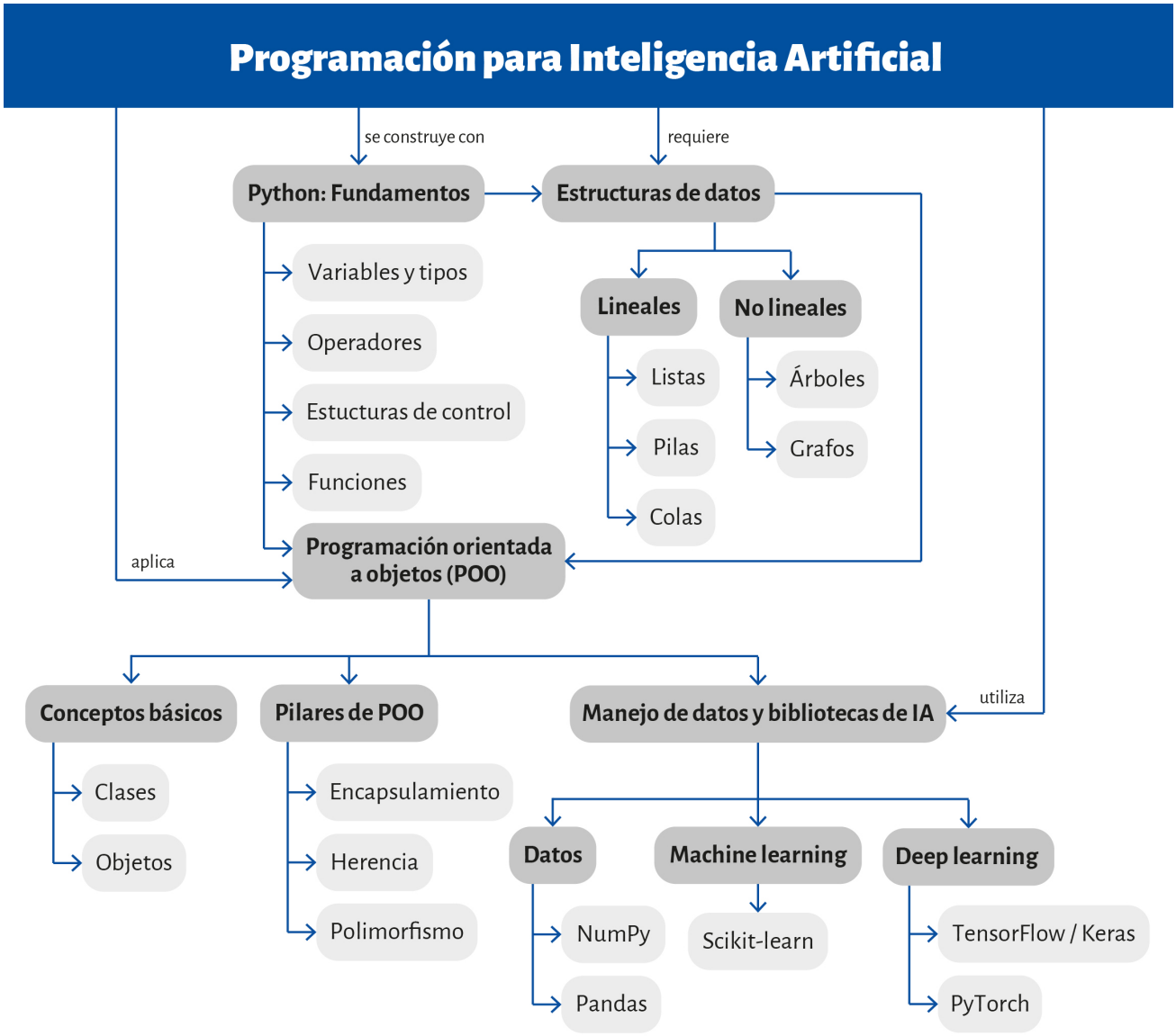
Para alcanzar la condición de alumno regular usted deberá aprobar la Evaluación Integradora dentro de los plazos previstos para el cursado.

Si reprueba esta instancia podrá reelaborar la evaluación según los requerimientos del docente.

Examen Final:

Deberá rendir, en condición de regular, una evaluación final que se realizará a través de un examen escrito integrador, presencial, en el periodo que disponga la Universidad.

Mapa conceptual



Mapa conceptual animado

Módulo 1

Microobjetivos

- Comprender la configuración básica de un entorno de trabajo con Python y VS Code, así como la ejecución de un primer programa "Hola Mundo", para iniciar la práctica con confianza y disponer de un espacio estable de experimentación con código.

- Comprender los fundamentos de la programación estructurada en Python para adquirir soltura en la escritura de programas funcionales y resolver problemas de forma algorítmica.

- Integrar el uso de funciones con parámetros y valores de retorno, para sentar las bases del desarrollo organizado de forma modular, reutilizable y compatible con Inteligencia Artificial.

Contenidos

MÓDULO 1: FUNDAMENTOS DE PROGRAMACIÓN EN PYTHON



Fuente: Freepik

Damos inicio al primer módulo de Programación para Inteligencia Artificial, donde aprenderemos los fundamentos de la programación en Python. Este módulo es la base sobre la cual construiremos todo lo demás, por lo que es fundamental comprender cada concepto y, sobre todo, practicar escribiendo código.

Nuestro enfoque será directo y práctico. Python es el lenguaje más utilizado en el campo de la IA, no solo por su sintaxis clara, sino por el ecosistema de bibliotecas especializadas que lo rodean. Comenzaremos desde cero, sin necesidad de conocimientos previos de programación.

Primero entenderemos qué significa programar y qué es un algoritmo, conceptos fundamentales que nos permitirán pensar de manera estructurada a la hora de resolver pro-

blemas. Luego configuraremos nuestro entorno de trabajo: instalaremos Python y Visual Studio Code, un editor profesional potente y versátil que nos acompañará no solo en esta materia, sino en toda la carrera.

A partir de nuestro primer programa "Hola Mundo", nos sumergiremos en los fundamentos de Python: variables, tipos de datos, operadores, estructuras condicionales, bucles y funciones. Cada concepto irá acompañado de ejemplos que podrán copiar, ejecutar y modificar. Como sostendremos durante todo el cursado: la programación se aprende programando, así que no tengamos miedo a los errores, son parte natural del proceso.

Dado que están cursando a distancia, este material está diseñado para ser completo y autosuficiente, con explicaciones detalladas y ejemplos abundantes. Al finalizar este módulo, serán capaces de escribir programas básicos en Python y de estructurar soluciones a problemas concretos.

¡Comencemos!

¿Qué significa programar?

Cuando hablamos de programar, solemos imaginar pantallas llenas de código, símbolos incomprensibles y personas trabajando en oficinas iluminadas por monitores. Sin embargo, programar es algo mucho más simple y, a la vez, mucho más poderoso de lo que parece a primera vista.

Énfasis

Programar significa dar instrucciones precisas y ordenadas a una computadora para que realice una tarea específica. Es, en esencia, comunicarse con una máquina.

Pero a diferencia de la comunicación entre humanos, donde podemos usar metáforas, contextos implícitos y ambigüedades que se resuelven con sentido común, una computadora necesita instrucciones absolutamente claras y sin ambigüedades. La máquina hace exactamente lo que le decimos, ni más ni menos, y si nuestras instrucciones son imprecisas o incorrectas, el resultado no será el esperado.

Pensemos en una situación cotidiana. Si le pedimos a alguien que prepare un café, esa persona usará su experiencia y conocimiento para interpretar la solicitud: calentará agua, buscará el café, elegirá una taza adecuada y realizará los pasos necesarios, incluso si no le dimos instrucciones detalladas. Una computadora no puede hacer eso. Si queremos que una máquina prepare café, tendríamos que especificar cada paso de manera explícita: cuánta agua usar, a qué temperatura calentarla, cuánto café agregar, cuánto tiempo esperar. Cada detalle cuenta.

Programar es, entonces, una forma de pensar. Implica analizar un problema, dividirlo en pasos más pequeños y manejables, y traducir esos pasos a un lenguaje que la computadora pueda entender y ejecutar. Esta habilidad de descomposición y pensamiento lógico es lo que realmente estamos aprendiendo cuando aprendemos a programar.

En el contexto de la inteligencia artificial, esta capacidad se vuelve aún más importante. Los sistemas de IA resuelven problemas complejos como reconocer imágenes, entender lenguaje natural o tomar decisiones estratégicas. Para construir esos sistemas, necesitamos poder expresar con claridad qué queremos que el programa haga, cómo debe procesar la información y qué resultados debe producir. Sin una base sólida en programación, sería imposible desarrollar o trabajar con modelos de machine learning o redes neuronales.

Concepto de algoritmo

Antes de escribir nuestro primer programa, necesitamos entender qué es un algoritmo. Este concepto es fundamental y aparecerá constantemente a lo largo de la materia y de toda la carrera.

Énfasis

Un algoritmo es una secuencia finita y ordenada de pasos que resuelve un problema o realiza una tarea específica.

Los algoritmos existen en nuestra vida cotidiana, aunque no siempre los identifiquemos como tales. Una receta de cocina es un algoritmo: indica qué ingredientes necesitamos, en qué orden debemos mezclarlos, cuánto tiempo cocinar y qué resultado esperamos obtener. Las instrucciones para armar un mueble son un algoritmo. Incluso el proceso

mental que seguimos para decidir qué ropa ponernos según el clima es, en cierto sentido, un algoritmo.

En programación, un algoritmo es el plan que diseñamos antes de escribir código. Define la lógica que resolverá nuestro problema. Un buen algoritmo debe cumplir con ciertas características esenciales. Primero, debe ser finito. Esto significa que, después de ejecutar un número determinado de pasos, el algoritmo termina y produce un resultado. No puede quedarse ejecutándose indefinidamente. Si diseñamos un algoritmo para sumar dos números, en algún momento debe devolvernos el resultado de esa suma y finalizar. Segundo, debe tener entradas y salidas bien definidas. Necesitamos saber qué información le vamos a proporcionar al algoritmo y qué resultado esperamos recibir. Si nuestro algoritmo calcula el promedio de una lista de números, la entrada es esa lista y la salida es el promedio calculado. En tercer lugar, cada paso debe ser claro y preciso. No puede haber ambigüedades. Instrucciones vagas como "procesar los datos adecuadamente" no sirven. Necesitamos especificar exactamente qué operación realizar en cada momento. Cuarto, debe ser efectivo. Cada paso debe ser realizable mediante operaciones básicas que la computadora pueda ejecutar. No podemos incluir instrucciones imposibles o que dependan de decisiones subjetivas. Finalmente, el orden importa. Cambiar la secuencia de los pasos puede alterar completamente el resultado o hacer que el algoritmo no funcione.

Veamos un ejemplo simple. Imaginemos que queremos diseñar un algoritmo para determinar si un número es par o impar. Los pasos serían:

1. Recibir el número a evaluar. Esa es nuestra entrada.
2. Dividir ese número entre dos y obtener el resto de esa división. En matemáticas, esto se llama operación módulo.
3. Verificar el resultado. Si el resto es cero, el número es par. Si el resto es uno, el número es impar.
4. Devolver el resultado indicando si el número es par o impar. Esa es nuestra salida.

Este algoritmo cumple con todas las características: es finito, tiene entrada y salida claras, cada paso es preciso, todo es efectivo y realizable, y el orden de los pasos es lógico y necesario.

Instalación de Python



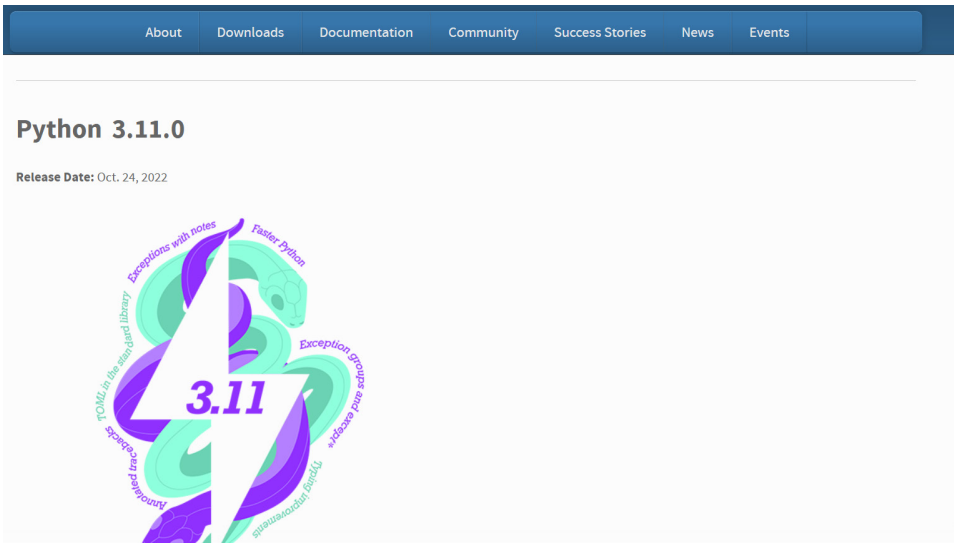
Fuente: Python.org

Ahora que entendemos qué es programar y qué son los algoritmos, es momento de preparar nuestro entorno de trabajo. Para programar en Python, necesitamos dos cosas: el lenguaje Python instalado en nuestra computadora y un editor donde escribir nuestro código.

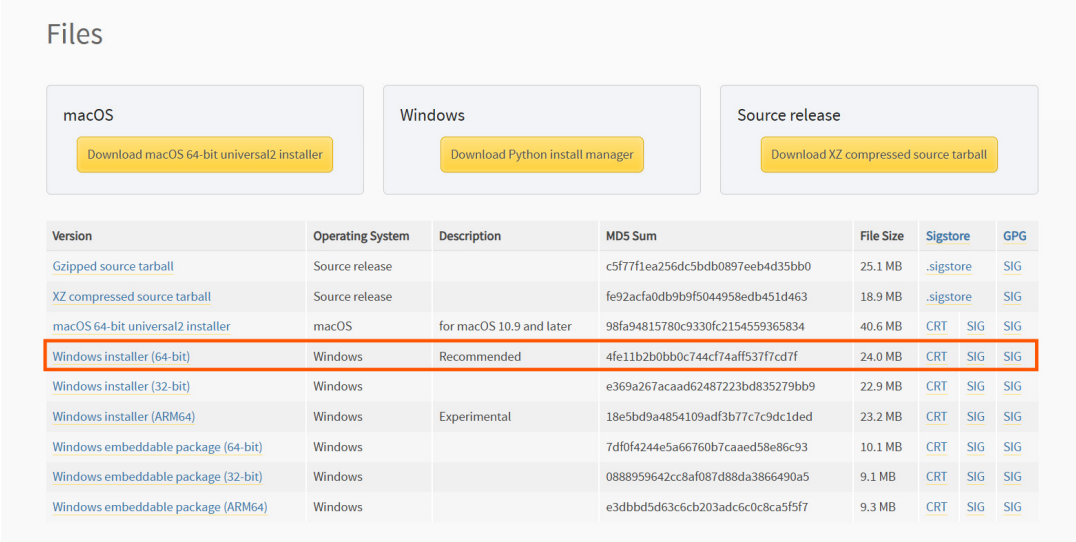
Python es un lenguaje de programación, pero también es el nombre del software que interpreta y ejecuta el código escrito en ese lenguaje.

Recursos

Para instalarlo, debemos dirigirnos al sitio oficial en https://www.python.org/. En la página de inicio aparece un botón que dice "*Downloads*" del cual se despliega un menú. Seleccionamos "*All releases*" y buscamos la release de Python 3.11.9. Ese número indica la versión más reciente de la versión 3.11. Esta es la URL de la versión que necesitamos para la materia https://www.python.org/downloads/release/python-3119/



Fuente: Captura de pantalla del sitio oficial de Python de elaboración propia.



Version	Operating System	Description	MD5 Sum	File Size	Sigstore	GPG
Gzipped source tarball	Source release		c5f77f1ea256dc5bdb0897eeb4d35bb0	25.1 MB	.sigstore	SIG
XZ compressed source tarball	Source release		fe92acfa0db9b9f5044958edb451d463	18.9 MB	.sigstore	SIG
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later	98fa94815780c9330fc2154559365834	40.6 MB	CRT SIG	SIG
Windows installer (64-bit)	Windows	Recommended	4fe11b2b0bb0c744cf74aff537f7cd7f	24.0 MB	CRT SIG	SIG
Windows installer (32-bit)	Windows		e369a267acaad62487223bd835279bb9	22.9 MB	CRT SIG	SIG
Windows installer (ARM64)	Windows	Experimental	18e5bd9a4854109adf3b77c7c9dc1ded	23.2 MB	CRT SIG	SIG
Windows embeddable package (64-bit)	Windows		7df0f4244e5a66760b7caaed58e86c93	10.1 MB	CRT SIG	SIG
Windows embeddable package (32-bit)	Windows		0888959642cc8af087d88da3866490a5	9.1 MB	CRT SIG	SIG
Windows embeddable package (ARM64)	Windows		e3dbbd5d63c6cb203adc6c0c8ca5f5f7	9.3 MB	CRT SIG	SIG

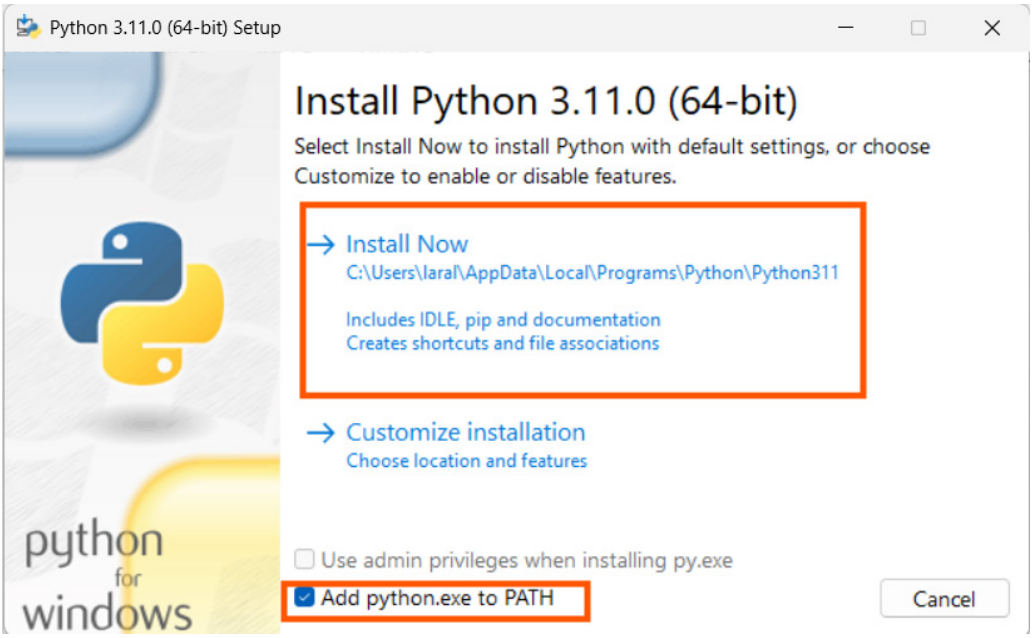
Fuente: Captura de pantalla del sitio oficial de Python de elaboración propia.

Al hacer clic en ese botón, se descarga un archivo instalador adecuado para nuestro sistema operativo. En Windows, el archivo tendrá extensión .exe. En macOS será un archivo .pkg. Una vez descargado el instalador, lo ejecutamos y aparecerá una ventana de bienvenida con opciones de configuración.

Énfasis

Aquí es fundamental marcar la casilla que dice "Add Python to PATH" antes de continuar. Esta opción permite que el sistema reconozca los comandos de Python desde cualquier

ubicación, lo cual simplifica mucho el trabajo posterior. Además, asegurémonos de estar instalando Python 3.11 verificando que el título de la ventana del instalador indique "Python 3.11" y no otra versión.



Fuente: Captura de pantalla de Python de elaboración propia.

Luego seleccionamos la opción "*Install Now*" y esperamos a que el proceso finalice. Al terminar, el instalador mostrará un mensaje de confirmación. Con esto, Python ya está listo para usarse en nuestra computadora.

Para verificar que la instalación fue exitosa, podemos abrir una terminal o consola. En Windows, buscamos "*cmd*" o "*PowerShell*" en el menú de inicio. Una vez abierta la consola, escribimos el siguiente comando y presionamos *Enter*:

```
python --version
```

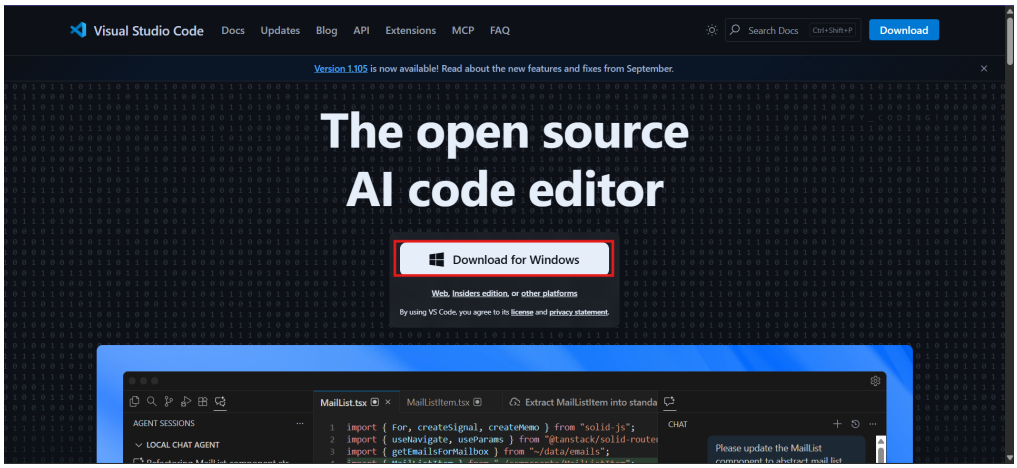
Si todo salió bien, aparecerá un mensaje indicando la versión de Python instalada, algo como "Python 3.11.4" o similar. Eso confirma que Python está correctamente instalado y accesible.

Instalación y configuración de Visual Studio Code

Ahora necesitamos un editor donde escribir nuestro código. Existen muchas opciones, pero recomendamos Visual Studio Code, o VS Code, por varias razones. Es gratuito, está disponible para todos los sistemas operativos, tiene soporte excelente para Python y es ampliamente utilizado en la industria, especialmente en proyectos de inteligencia artificial y machine learning.

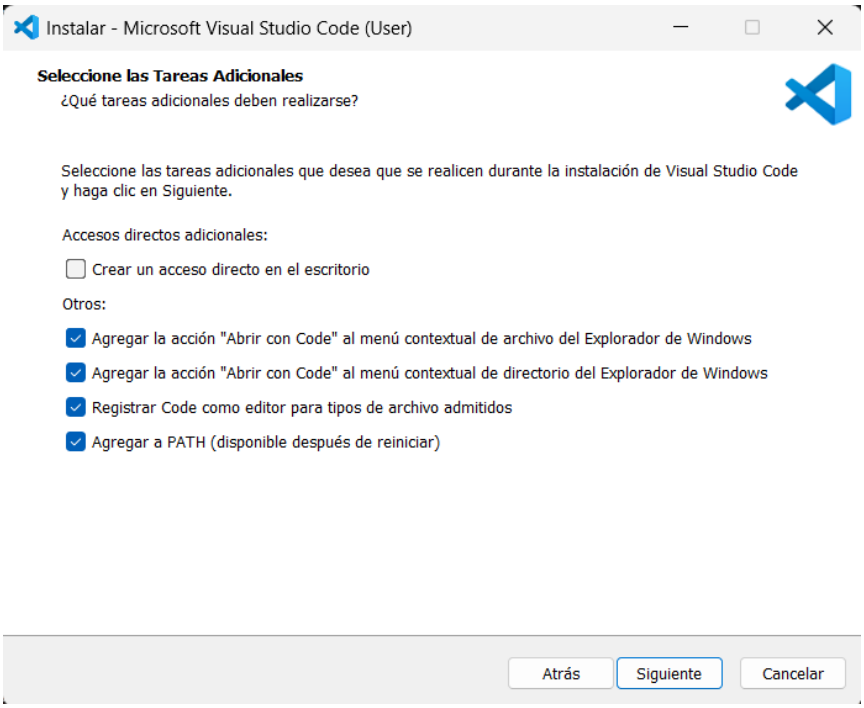
Recursos

Para instalar VS Code, ingresamos a https://code.visualstudio.com/. En la página principal veremos un botón de descarga. Hacemos clic y se descargará el instalador correspondiente a nuestro sistema operativo.



Fuente: Captura de pantalla de VS Code de elaboración propia.

Ejecutamos el instalador y seguimos los pasos. La instalación es sencilla y similar a la de cualquier programa. Aceptamos los términos, elegimos la carpeta de instalación y, cuando nos pregunte si queremos agregar opciones al menú contextual, recomendamos marcar esas opciones porque facilitan abrir archivos con VS Code directamente desde el explorador de archivos.



Fuente: Captura de pantalla de VS Code de elaboración propia.

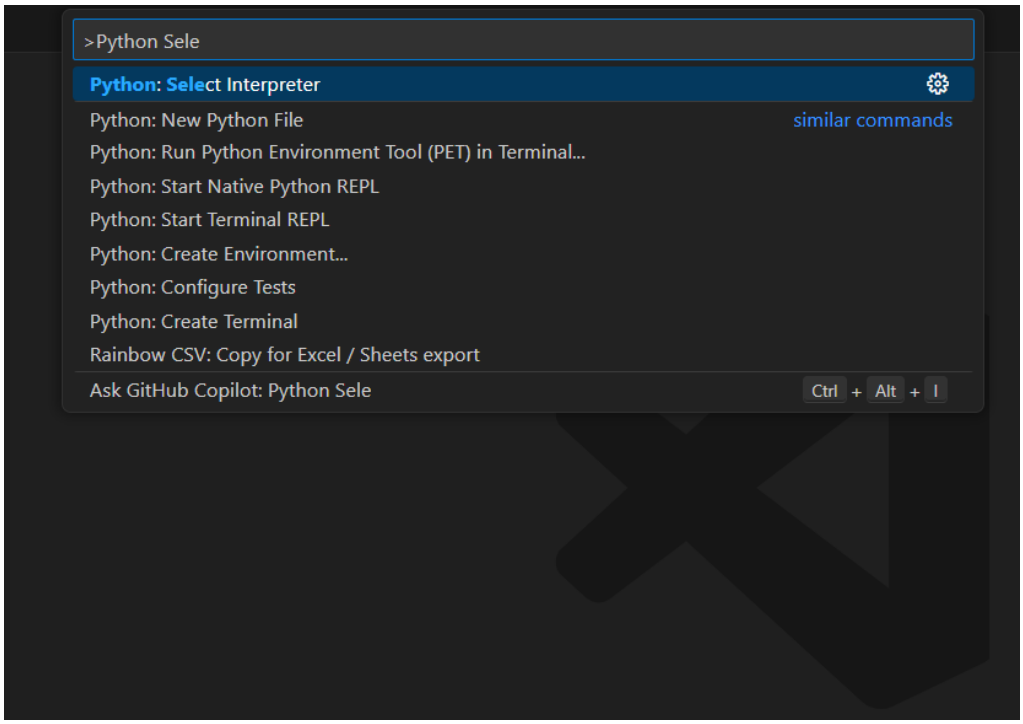
Al abrir VS Code por primera vez, veremos una interfaz limpia con varias secciones. En el lado izquierdo hay una barra de actividades con íconos. En el centro está el área principal donde editaremos nuestros archivos y, también, puede aparecer una página de bienvenida con recursos y tutoriales. Para trabajar con Python en VS Code, necesitamos instalar una extensión. Las extensiones son complementos que agregan funcionalidades específicas al editor. Hacemos clic en el ícono de extensiones en la barra lateral izquierda. Parece un cuadrado dividido en cuatro partes o piezas de rompecabezas.



Fuente: Captura de pantalla de VS Code de elaboración propia

Se abrirá un panel de búsqueda. Escribimos "Python" y aparecerán varios resultados. Buscamos la extensión llamada "Python" publicada por Microsoft. Es la más popular y tiene millones de descargas. Hacemos clic en "Install" y esperamos a que se instale. Esta extensión proporciona resaltado de sintaxis, autocompletado de código, depuración y muchas otras características que hacen que programar en Python sea mucho más cómodo.

Una vez instalada la extensión, VS Code puede pedirnos que seleccionemos un intérprete de Python. Esto significa elegir qué versión de Python queremos usar. Si instalamos Python correctamente, VS Code debería detectarlo automáticamente. Si no aparece, podemos presionar Ctrl+Shift+P para abrir la paleta de comandos, escribir "Python: Select Interpreter" y elegir la versión de Python que instalamos.



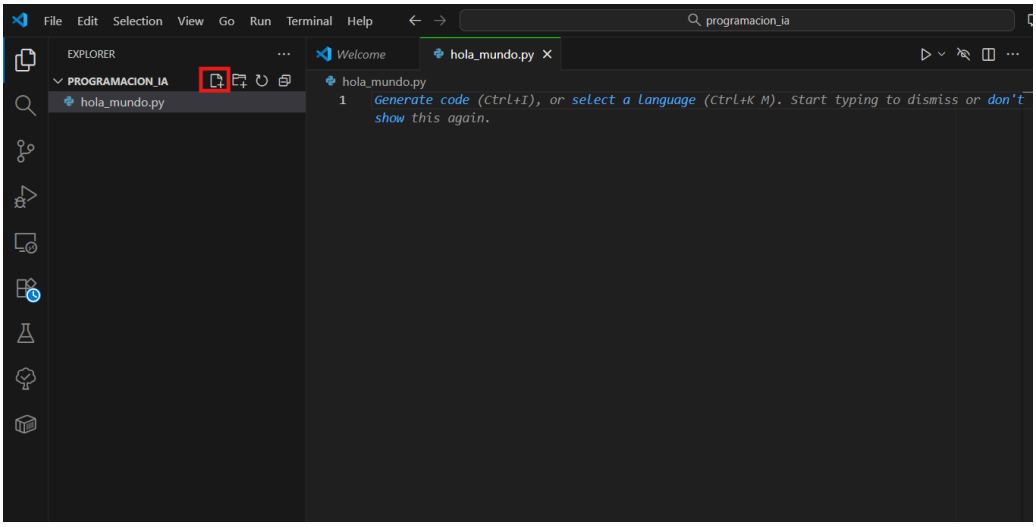
Fuente: Captura de pantalla de VS Code de elaboración propia

Creando nuestro primer programa

Con Python y VS Code instalados, estamos listos para escribir código. Creemos una carpeta en nuestro equipo donde guardaremos todos los programas de esta materia. Podemos llamarla "Programacion_IA" o cualquier nombre que nos resulte claro.

Abrimos VS Code y seleccionamos *"File" → "Open Folder" (Archivo → Abrir Carpeta)*. Navegamos hasta la carpeta que creamos y la seleccionamos. Ahora VS Code mostrará esa carpeta en el explorador lateral izquierdo.

Para crear un nuevo archivo, hacemos clic en el ícono de "Nuevo archivo" en el explorador, o presionamos Ctrl+N. VS Code creará un archivo sin título. Lo guardamos con el nombre "hola_mundo.py". La extensión .py le indica a VS Code y al sistema que este es un archivo de Python.

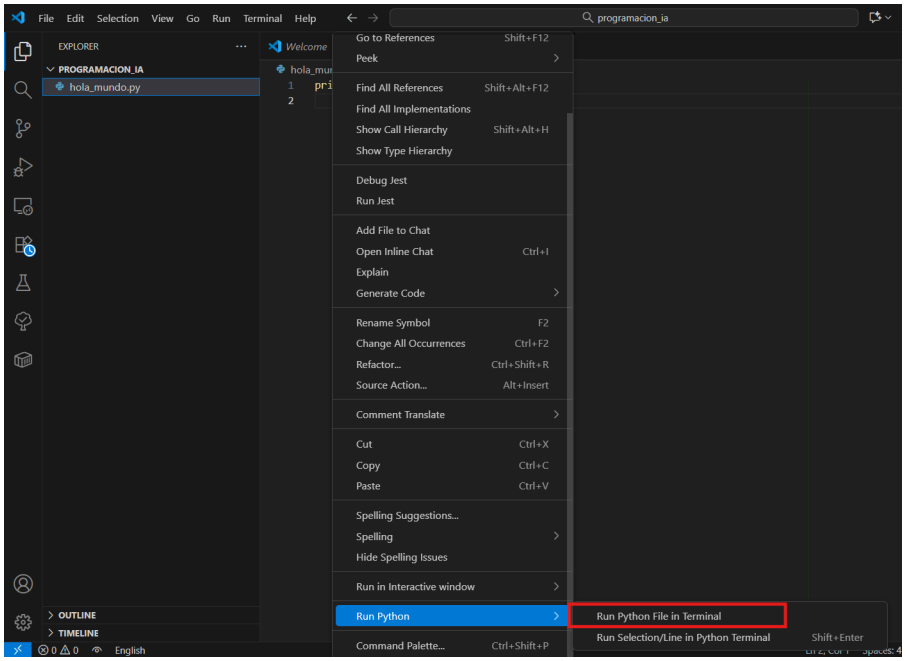


Fuente: Captura de pantalla de VS Code de elaboración propia.

En el archivo vacío, escribimos nuestro primer programa:

```
print("Hola Mundo")
```

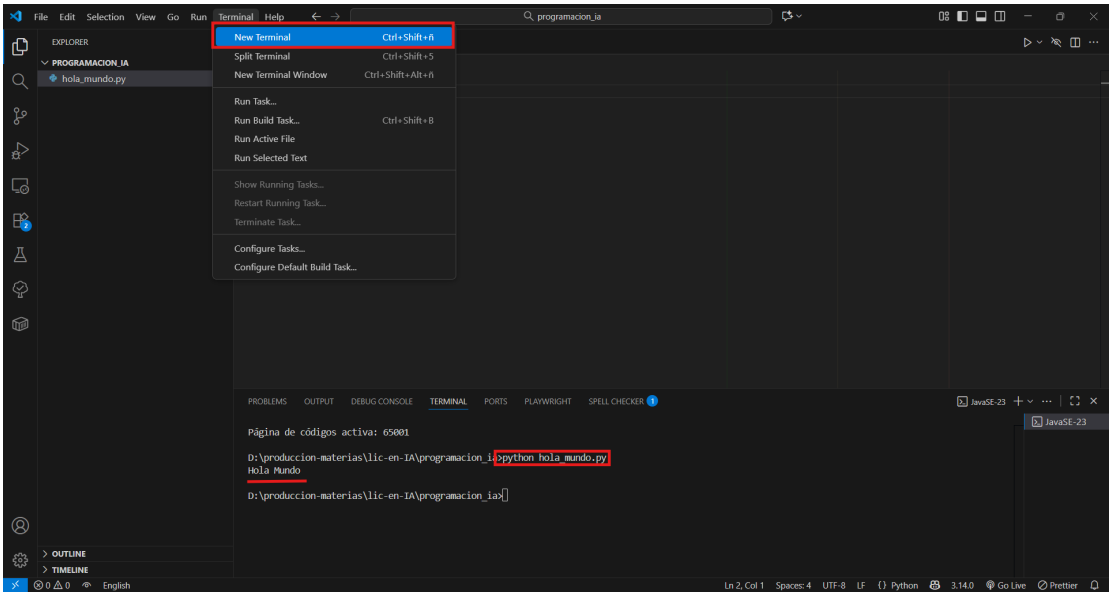
Este es el programa más simple que podemos escribir en Python. La función print() muestra en pantalla el texto que colocamos entre comillas. Para ejecutar el programa, tenemos varias opciones. Podemos hacer clic derecho en el editor y seleccionar "Run Python File in Terminal".



Fuente: Captura de pantalla de VS Code de elaboración propia.

O podemos abrir una terminal integrada en VS Code , navegar hasta nuestra carpeta si es necesario, y escribir:

```
python hola_mundo.py
```



Fuente: Captura de pantalla de VS Code de elaboración propia.

Si todo salió bien, veremos en la terminal el mensaje “Hola Mundo”. Ese texto en pantalla confirma que nuestro entorno está configurado correctamente y que estamos listos para continuar.

Puede parecer trivial, pero ese primer programa funcional es un hito importante. Acabamos de escribir una instrucción, Python la interpretó y ejecutó, y obtuvimos un resultado visible. A partir de aquí, construiremos programas cada vez más complejos.

Énfasis

Existen otros entornos similares a VS Code, como Jupyter Notebook o Google Colab. Son entornos interactivos que permite escribir y ejecutar código en celdas independientes, ver resultados inmediatos y combinar código con texto explicativo, gráficos y ecuaciones. ¡Lo/a invito a investigar de que se tratan!

Variables y tipos de datos básicos

Con el entorno configurado y el primer programa ejecutado, llega el momento de empezar a construir programas que hagan algo más que mostrar un mensaje fijo. Para eso necesitamos aprender a manejar información que pueda cambiar, que pueda provenir del usuario o de un cálculo, y que podamos guardar temporalmente mientras el programa se ejecuta. Esa información se almacena en variables.

Énfasis

Una variable es un espacio de memoria con un nombre que usamos para guardar un valor. Ese valor puede cambiar durante la ejecución del programa, y por eso se llama variable.

Podemos pensar en una variable como una caja etiquetada donde guardamos algo. La etiqueta es el nombre de la variable, y lo que hay dentro es el valor. En cualquier momento podemos abrir la caja, mirar qué contiene, cambiar su contenido o usar ese contenido en una operación.

En Python, crear una variable es tan simple como asignarle un valor con el signo igual. Por ejemplo:

```
edad = 25
```

Con esa línea, estamos creando una variable llamada edad y guardando en ella el número 25. A partir de ese momento, cada vez que escribamos edad en el programa, Python entenderá que nos referimos al valor 25. Si más adelante hacemos:

```
edad = 30
```

el valor de la variable cambia, y ahora edad contiene 30 en lugar de 25. La variable sigue siendo la misma, pero su contenido se actualizó.

Los nombres de las variables deben seguir ciertas reglas. Pueden contener letras, números y guiones bajos, pero no pueden empezar con un número. Tampoco pueden contener espacios ni caracteres especiales como acentos, eñes o símbolos matemáticos. Además, Python distingue entre mayúsculas y minúsculas, por lo que edad, Edad y EDAD son tres variables diferentes.

Énfasis

Es una buena práctica elegir nombres descriptivos que reflejen qué información contiene la variable. Un nombre como edad es mucho más claro que x o a. Escribir código legible facilita entenderlo más adelante y compartirlo con otros.

Tipos de datos básicos

Cada valor que almacenamos en una variable pertenece a un tipo de dato. El tipo define qué clase de información es y qué operaciones podemos hacer con ella. Python reconoce automáticamente el tipo de dato cuando creamos una variable, pero es importante que nosotros entendamos qué tipos existen y cómo se comportan.

Los tipos de datos básicos más importantes son los números enteros, los números decimales, las cadenas de texto y los valores lógicos.

Veamos en qué consisten cada uno de ellos.

› *Números enteros–int*

Los números enteros, llamados int en Python, son valores sin parte decimal. Pueden ser positivos, negativos o cero. Por ejemplo:

```
cantidad = 10
temperatura = -5
cero = 0
```

Con los números enteros podemos hacer operaciones aritméticas como sumas, restas, multiplicaciones y divisiones. Python maneja automáticamente incluso números enteros muy grandes, sin límites prácticos de tamaño.

› *Números decimales–float*

Los números decimales, llamados float en Python, tienen una parte fraccionaria. Se escriben usando un punto como separador decimal. Por ejemplo:

```
precio = 19.99
pi = 3.14159
porcentaje = 0.75
```

Los números decimales son esenciales cuando necesitamos precisión en cálculos que involucran fracciones, como porcentajes, promedios o medidas físicas. Es importante recordar que Python usa el punto y no la coma para separar la parte entera de la decimal, siguiendo la notación anglosajona.

› *Cadenas de texto–str*

Las cadenas de texto, llamadas str en Python, representan secuencias de caracteres. Pueden ser letras, números, símbolos o espacios, y siempre se escriben entre comillas, ya sean simples o dobles. Por ejemplo:

```
nombre = "Ana"
apellido = 'García'
mensaje = "Hola, ¿cómo estás?"
numero_como_texto = "123"
```

Es importante distinguir entre un número y un número escrito como texto. El valor 123 es un número entero con el que podemos hacer operaciones matemáticas. El valor "123" es una cadena de texto, y aunque se vea igual, no podemos sumarlo directamente con otros números sin antes convertirlo.

Las cadenas de texto son fundamentales cuando trabajamos con nombres, direcciones, mensajes o cualquier información que no sea estrictamente numérica.

› *Valores lógicos–bool*

Los valores lógicos, o booleanos, representan verdadero o falso. En Python se escriben como True y False, con la primera letra en mayúscula. Son útiles cuando necesitamos tomar decisiones o verificar condiciones. Por ejemplo:

```
es_mayor_de_edad = True
tiene_descuento = False
```

Aunque parezcan simples, los valores booleanos son la base de toda la lógica de control en programación. Permiten que un programa reaccione de manera distinta según el estado de las variables.

Conversión entre tipos de datos

A veces necesitamos transformar un valor de un tipo a otro. Por ejemplo, si pedimos al usuario que ingrese su edad, esa entrada llegará como texto, y deberemos convertirla a número entero para poder hacer cálculos con ella. Python ofrece funciones para estas conversiones:

```
edad_texto = "25"
edad_numero = int(edad_texto)
```

Ahora edad_numero contiene el entero 25, y podemos usarlo en operaciones matemáticas. Si queremos convertir un número a texto, usamos str():

```
edad = 25
mensaje = "Tienes " + str(edad) + " años"
```

Del mismo modo, podemos convertir enteros a decimales con float(), o textos a decimales si contienen un número válido:

```
precio_texto = "19.99"
precio_decimal = float(precio_texto)
```

Estas conversiones son frecuentes y esenciales para integrar distintas partes de un programa. Hay que tener cuidado: si intentamos convertir un texto que no representa un número, Python mostrará un error.

Actividad

En este momento, está en condiciones de realizar la **actividad 1** de módulo.

Operadores

Los operadores son símbolos que nos permiten realizar operaciones con variables y valores. Existen operadores aritméticos para cálculos matemáticos, operadores de comparación para evaluar relaciones entre valores, y operadores lógicos para combinar condiciones.

› *Operadores aritméticos*

Los operadores aritméticos funcionan con números y producen resultados numéricos. Los más comunes son la suma, la resta, la multiplicación y la división. Veamos ejemplos en Python:

```
a = 10
b = 3
suma = a + b # 13
resta = a–b # 7
multiplicacion = a * b # 30
division = a / b # 3.333...
```



En Python, los comentarios en el código se escriben con #, es decir, todo aquello que esté después de un '#' no será ejecutado. Para comentarios más largos se utilizan comillas de la siguiente manera:

```
'''
Esto es un comentario largo
Puedo escribir en varias líneas
y no será ejecutado.
'''
```

La división con / siempre devuelve un número decimal, incluso si los operandos son enteros. Si queremos una división entera, que descarte la parte decimal y devuelva solo el cociente, usamos //:

```
division_entera = a // b # 3
```

También existe el operador módulo %, que devuelve el resto de la división:

```
resto = a % b # 1, porque 10 dividido 3 da 3 con resto 1
```

El módulo es muy útil para saber si un número es par o impar. Si n % 2 da 0, el número es par; si da 1, es impar.

Finalmente, tenemos el operador de potencia **:

```
cuadrado = a ** 2 # 100
cubo = b ** 3 # 27
```

› *Operadores de comparación*

Los operadores de comparación evalúan la relación entre dos valores y devuelven un resultado booleano: True o False. Son fundamentales para tomar decisiones en el programa. En Python:

```
a = 10
b = 3
mayor = a > b # True
menor = a < b # False
mayor_o_igual = a >= b # True
menor_o_igual = a <= b # False
igual = a == b # False
distinto = a != b # True
```

Observemos que para comparar igualdad usamos == con dos signos igual, no con uno solo. Un solo = es asignación, mientras que == es comparación. Es un error común confundirlos, y puede causar comportamientos inesperados.

› *Operadores lógicos*

Los operadores lógicos combinan expresiones booleanas y permiten construir condiciones más complejas. Los principales son and (y), or (o) y not (no). En Python:

```
edad = 20
tiene_carnet = True
puede_conducir = edad >= 18 and tiene_carnet # True
```

Énfasis

El operador and devuelve True solo si ambas condiciones son verdaderas. Si alguna es falsa, el resultado es falso.

```
es_fin_de_semana = True
es_feriado = False
descansa = es_fin_de_semana or es_feriado # True
```

Énfasis

El operador or devuelve True si al menos una de las condiciones es verdadera. Solo devuelve False si ambas son falsas.

```
esta_lloviendo = False
puede_salir = not esta_lloviendo # True
```

Énfasis

El operador not invierte el valor booleano. Si la expresión es verdadera, not la convierte en falsa, y viceversa.

Los operadores lógicos son esenciales para expresar condiciones complejas de manera clara y precisa.

Estructuras de control

Hasta ahora hemos aprendido a almacenar datos en variables y a realizar operaciones con ellos. Pero un programa que siempre ejecuta las mismas instrucciones en el mismo

orden tiene un valor limitado. Lo que realmente hace poderosa a la programación es la capacidad de tomar decisiones y alterar el flujo de ejecución según las condiciones que se presenten.

Condicionales

Las estructuras de control condicionales permiten que el programa elija entre distintos caminos en función de si una condición es verdadera o falsa. La más básica es la instrucción if, que en español significa "si".

› *if simple*

La estructura if evalúa una condición y, si es verdadera, ejecuta un bloque de código. Si la condición es falsa, ese bloque se omite y el programa continúa con la siguiente instrucción. En Python:

```
edad = 20
if edad >= 18:
    print("Eres mayor de edad")
```

Observemos algunos detalles importantes. Primero, la línea que contiene el if termina con dos puntos (:). Esto le indica a Python que lo que sigue es un bloque de código asociado a esa condición. Segundo, la línea siguiente está indentada, es decir, desplazada hacia la derecha con espacios o tabulaciones.



La indentación es el desplazamiento hacia la derecha, con espacios o tabulaciones. Python usa la indentación para saber qué instrucciones pertenecen al bloque del if. Todo lo que esté indentado al mismo nivel se ejecutará solo si la condición es verdadera.

› *if-else*

A menudo queremos que el programa haga una cosa si la condición es verdadera y otra distinta si es falsa. Para eso usamos else, que significa "si no". En Python:

```
edad = 15
if edad >= 18:
   print("Eres mayor de edad")
else:
   print("Eres menor de edad")
```

Si edad >= 18 es verdadero, se ejecuta el primer bloque. Si es falso, se ejecuta el segundo. No hay posibilidad de que ambos bloques se ejecuten, ni de que ninguno lo haga. Siempre se elige uno de los dos caminos. Esta estructura es fundamental cuando necesitamos reaccionar de manera distinta ante situaciones opuestas.

› *if-elif-else*

Cuando hay más de dos caminos posibles, usamos elif, que es una contracción de "else if" y significa "si no, si". Permite encadenar múltiples condiciones. En Python:

```
nota = 85
if nota >= 90:
   print("Excelente")
elif nota >= 70:
   print("Aprobado")
elif nota >= 50:
   print("Regular")
else:
   print("Desaprobado")
```

Python evalúa cada condición en orden. Si nota >= 90 es verdadero, ejecuta ese bloque y omite el resto. Si es falso, pasa a evaluar nota >= 70. Si esa es verdadera, ejecuta su bloque y omite las siguientes. Y así sucesivamente. El bloque else al final es opcional y se ejecuta solo si ninguna de las condiciones anteriores fue verdadera. Esta estructura permite modelar decisiones complejas de manera clara y organizada.

Condiciones compuestas

Además, podemos combinar varias condiciones en un mismo if usando operadores lógicos. Por ejemplo, si queremos verificar que un número esté en un rango determinado:

```
temperatura = 22
if temperatura >= 18 and temperatura <= 25:
   print("Clima agradable")
```

Aquí la condición es verdadera solo si ambas partes lo son. También podemos usar or para aceptar cualquiera de las condiciones:

```
dia = "sábado"
if dia == "sábado" or dia == "domingo":
   print("Es fin de semana")
```

Las condiciones compuestas permiten expresar lógica compleja en una sola instrucción, manteniendo el código limpio y legible.

Actividad

En este momento, está en condiciones de realizar la **actividad 2** del módulo.

Repetitivas

Además de tomar decisiones, los programas necesitan repetir tareas. Ejecutar la misma instrucción cientos o miles de veces sería inviable si tuviéramos que escribirla manual-

mente cada vez. Para eso existen las estructuras de control repetitivas, también llamadas bucles o ciclos.

Los bucles permiten que un bloque de código se ejecute múltiples veces, mientras se cumpla una condición o mientras haya elementos que procesar. Los dos tipos principales son el bucle while y el bucle for.

› *Bucle while*

El bucle while repite un bloque de código mientras una condición sea verdadera. Antes de cada iteración, se evalúa la condición. Si es verdadera, se ejecuta el bloque. Si es falsa, el bucle termina y el programa continúa con la siguiente instrucción. Un ejemplo sencillo en Python:

```
contador = 1
while contador <= 5:
    print(contador)
contador = contador + 1
```

Este programa muestra los números del 1 al 5. Al inicio, contador vale 1. La condición contador <= 5 es verdadera, así que se ejecuta el bloque: imprime 1 y luego incrementa contador a 2. Vuelve a evaluar la condición, que sigue siendo verdadera, imprime 2, incrementa a 3, y así sucesivamente. Cuando contador llega a 6, la condición es falsa y el bucle termina.

Es fundamental asegurarse de que la condición del while eventualmente se vuelva falsa. Si no lo hace, el bucle se ejecutará indefinidamente, y el programa quedará atrapado en un ciclo infinito.

Énfasis

Un error común al usar while es olvidar modificar la variable que aparece en la condición. Si contador nunca cambia, la condición nunca será falsa y el programa no terminará.

› *Bucle for*

El bucle for se usa cuando sabemos de antemano cuántas veces queremos repetir algo, o cuando queremos recorrer una colección de elementos. En Python, for itera sobre una secuencia. Por ejemplo, para mostrar los números del 1 al 5:

```
for i in range(1, 6):
    print(i)
```

La función range(1, 6) genera una secuencia de números desde 1 hasta 5. El número final, 6, no se incluye. En cada iteración, i toma el valor del siguiente número en la secuencia, y se ejecuta el bloque del for.

Si queremos empezar desde 0, podemos escribir solo el límite superior:

```
for i in range(5):
    print(i)
```

Esto mostrará 0, 1, 2, 3, 4.

El bucle for también sirve para recorrer cadenas de texto, donde en cada iteración tomamos un carácter:

```
palabra = "Hola"
for letra in palabra:
    print(letra)
```

Esto mostrará cada letra de "Hola" en una línea separada: H, o, l, a.

› *Bucles anidados*

Podemos colocar un bucle dentro de otro, formando bucles anidados. Esto es útil cuando necesitamos realizar tareas repetitivas dentro de tareas repetitivas. Por ejemplo, si queremos mostrar una tabla de multiplicar:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

El bucle exterior controla las filas, y el interior controla las columnas. Por cada valor de i, el bucle interno recorre todos los valores de j, imprimiendo el producto. Luego se pasa a la siguiente fila. Los bucles anidados son poderosos, pero deben usarse con cuidado, porque aumentan rápidamente la cantidad de operaciones que realiza el programa.

Interrumpir y saltar en bucles

A veces necesitamos salir de un bucle antes de que termine naturalmente, o saltar a la siguiente iteración sin completar la actual. Para eso existen las instrucciones *break* y *continue*.

La instrucción *break* detiene el bucle inmediatamente y continúa con el resto del programa:

```
contador = 1
while True:
    print(contador)
    contador += 1
    if contador > 5:
        break
```

Aquí while True crearía un bucle infinito, pero *break* lo interrumpe cuando contador supera 5.

Por otra parte, la instrucción *continue* salta a la siguiente iteración sin ejecutar el resto del bloque:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Esto imprimirá 1, 2, 4, 5, omitiendo el 3, porque cuando i == 3, continue salta directamente a la siguiente iteración sin ejecutar print(i).

Funciones básicas

A medida que los programas aumentan en tamaño y complejidad, repetir el mismo código comienza a generar dificultades. Por ejemplo, si en distintas partes del programa necesitamos calcular el área de varios rectángulos, sin utilizar funciones deberíamos escribir el mismo cálculo una y otra vez. Si más adelante quisiera modificar la forma de realizar ese cálculo, tendría que hacerlo en todos esos lugares. El uso de funciones permite evitar estas repeticiones y facilita el mantenimiento del código.

Énfasis

Una función es un bloque de código con un nombre que realiza una tarea específica. Podemos invocarla desde distintas partes del programa, y cada vez que lo hacemos, el código dentro de ella se ejecuta.

Pensemos en una analogía simple. Cuando decimos "batir las claras a punto nieve" en una receta, esa frase encapsula toda una secuencia de pasos que no necesitamos repetir cada vez. Las funciones funcionan igual: encapsulan una secuencia de instrucciones bajo un nombre significativo.

Definir y usar funciones

Para definir una función usamos la palabra clave def, seguida del nombre de la función, paréntesis y dos puntos. El cuerpo de la función va indentado:

```
def saludar():
    print("Hola, bienvenido al programa")
# Invocar la función
saludar()
```

Cuando escribimos saludar(), Python salta a la definición de la función, ejecuta su código, y luego continúa con el resto del programa.

Parámetros y return

Una función puede recibir parámetros, que son valores que necesita para trabajar:

```
def saludar_persona(nombre):
    print(f"Hola, {nombre}")
saludar_persona("Ana")
saludar_persona("Carlos")
```

Los parámetros nombre, edad funcionan como variables dentro de la función. Cuando invocamos la función con valores específicos, esos valores se asignan a los parámetros en el mismo orden.

Las funciones pueden devolver valores usando return. Esto es fundamental porque nos permite obtener un resultado de vuelta:

```
def sumar(a, b):
  resultado = a + b
  return resultado
total = sumar(5, 3)
print(f"La suma es: {total}")
```

La palabra return hace dos cosas: devuelve el valor y termina la ejecución de la función inmediatamente. Cualquier código después de return no se ejecuta. Veamos un ejemplo más completo:

```
def calcular_area_rectangulo(base, altura):
  if base <= 0 or altura <= 0:
      return 0
  area = base * altura
  return area
area1 = calcular_area_rectangulo(10, 5)
area2 = calcular_area_rectangulo(-8, 3)
print(f"Área 1: {area1}")
print(f"Área 2: {area2}")
```

Observemos que agregamos una validación. Si los valores son inválidos, devolvemos 0. Esto muestra cómo las funciones pueden contener lógica compleja y reutilizarla fácilmente.

Los parámetros pueden tener valores por defecto. Si no proporcionamos un valor al llamar la función, se usa el predeterminado:

```
def saludar_con_titulo(nombre, titulo="Sr./Sra."):
    print(f"Buenos días, {titulo} {nombre}")
saludar_con_titulo("Gómez")
saludar_con_titulo("López", "Dr.")
```

Una función también puede devolver múltiples valores separándolos con comas:

```
def calcular_estadisticas(numeros):
    minimo = min(numeros)
    maximo = max(numeros)
    promedio = sum(numeros) / len(numeros)
    return minimo, maximo, promedio
datos = [15, 8, 23, 4, 16]
min_val, max_val, prom_val = calcular_estadisticas(datos)
print(f"Mínimo: {min_val}, Máximo: {max_val}, Promedio: {prom_val}")
```

Alcance de las variables

Las variables definidas dentro de una función son locales, es decir, solo existen mientras la función se ejecuta y no son accesibles desde fuera:

```
def calcular_doble(numero):
    resultado = numero * 2
    return resultado
valor = calcular_doble(5)
print(valor)
print(resultado) # Esto daría error, resultado no existe fuera de la función
```

Esto evita que las variables de distintas funciones interfieran entre sí, lo cual es muy útil en programas grandes.

Criterios para el diseño de funciones

Al momento de diseñar funciones, conviene tener en cuenta algunos criterios básicos. En primer lugar, cada función debería encargarse de una única tarea y resolverla de manera clara. En segundo lugar, es importante elegir nombres descriptivos, preferentemente verbos, que permitan anticipar qué acción realiza la función. Por último, las funciones deberían recibir toda la información necesaria a través de sus parámetros y devolver un resultado mediante return, evitando depender de variables externas.



Las funciones son esenciales en programación profesional. Nos permiten escribir código organizado, reutilizable y fácil de mantener. En el desarrollo de sistemas de IA trabajaremos con muchas funciones que procesan datos, entrenan modelos y evalúan resultados.

En los próximos módulos profundizaremos en funciones más avanzadas y cómo trabajan con estructuras de datos complejas.



En este momento, está en condiciones de realizar la **actividad** 3 del módulo.

Módulo 1 | Glosario

Extensión: complemento que se instala en un IDE para agregar funcionalidades específicas. La extensión de Python para VS Code proporciona resaltado de sintaxis, autocompletado y depuración.

Google Colab: versión gratuita en la nube de Jupyter Notebook que no requiere instalación local y permite usar GPUs para entrenar modelos de IA de forma gratuita.

IDE (Integrated Development Environment): entorno de desarrollo integrado que combina editor de código, terminal, depurador y otras herramientas en una sola aplicación. Visual Studio Code (VS Code) es un IDE muy popular para programar en Python.

Intérprete: programa que lee y ejecuta código línea por línea, traduciendo las instrucciones a lenguaje máquina en tiempo real. Python es un lenguaje interpretado.

Jupyter Notebook: entorno interactivo que permite escribir y ejecutar código en celdas independientes, combinando código con texto explicativo y visualizaciones, muy utilizado en experimentación con IA y análisis de datos.

Machine learning (aprendizaje automático): campo de la inteligencia artificial que permite a las computadoras aprender patrones a partir de datos sin ser programadas explícitamente para cada tarea específica.

None: valor especial en Python que representa la ausencia de valor o un valor nulo. Se devuelve cuando una función no tiene un return explícito o cuando queremos indicar que no hay resultado.

PATH: variable del sistema operativo que indica en qué ubicaciones buscar los programas ejecutables. Agregar Python al PATH permite ejecutarlo desde cualquier ubicación en la terminal.

Redes neuronales: modelos computacionales inspirados en el funcionamiento del cerebro humano, formados por capas de nodos interconectados que procesan información, fundamentales en el aprendizaje profundo.

Sintaxis: conjunto de reglas que definen cómo deben escribirse las instrucciones en un lenguaje de programación para que sean válidas. Python tiene una sintaxis clara que facilita la lectura y escritura de código.

Terminal (o consola): interfaz de texto donde se ejecutan comandos del sistema operativo, se muestran los resultados de los programas y el usuario puede ingresar datos. También llamada línea de comandos.

Módulo 1 | Actividades

ACTIVIDAD 1
Primeros pasos con Python

Ahora que ya cuenta con un entorno de desarrollo configurado, es momento de verificar que todo funciona correctamente y dar los primeros pasos escribiendo código Python. Esta actividad tiene como objetivo asegurarnos de que puede ejecutar programas sin inconvenientes y que se siente cómodo/a con las herramientas elegidas.

Por ello, le pedimos que:

1. Escriba un programa que muestre en pantalla un mensaje de bienvenida personalizado. El programa debe incluir su nombre y la materia que está cursando. Por ejemplo:

Hola, soy [su nombre] y estoy cursando Programación para Inteligencia Artificial

2. Escriba un programa que realice las siguientes operaciones y muestre los resultados en pantalla:

   - Sumar dos números de su elección.
   - Restar dos números de su elección.
   - Multiplicar dos números de su elección.
   - Dividir dos números de su elección.
   - Calcular el resto de una división entre dos números de su elección.

Cada operación debe mostrarse con un mensaje descriptivo. Por ejemplo:

La suma de 15 y 7 es: 22

3. Escriba un programa que le pida al usuario que ingrese su nombre y su edad, y luego calcule en qué año cumplirá 100 años. El programa debe mostrar un mensaje como:

Hola [Nombre], cumplirás 100 años en el año [Año]

Recomendaciones

Pruebe investigando el comando *input()* en Python.

4. ¿Cómo se le ocurre que podría mejorar el programa del ejercicio dos? Esta pregunta es una invitación a que investigue, pruebe y juegue con el código.

Para la entrega deberá generar un documento PDF o Word que contenga capturas de pantalla de los ejercicios 1, 2 y 3, mostrando tanto el código como los resultados de la ejecución. El punto 4 puede abordarse mediante una propuesta escrita o desarrollarse siguiendo el mismo formato que los ejercicios anteriores.

Recomendaciones

Esta actividad puede realizarse de manera individual o junto a un/a compañero/a. Si trabajan en pareja, ambos deben ejecutar los programas en sus respectivos entornos para asegurarse de que todo funcione correctamente.

ACTIVIDAD 2

Tomando decisiones con estructuras condicionales

Hemos aprendido a trabajar con variables, operadores y estructuras condicionales. Ahora es el momento de poner en práctica estos conceptos resolviendo problemas que requieren que el programa tome decisiones en función de distintas condiciones.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código en Python completo y funcional, incluyendo capturas de pantalla de la ejecución de los programas, mostrando diferentes casos de prueba para verificar que funcionan correctamente.

Recomendaciones

Al resolver estos ejercicios, no se concentre únicamente en que el programa funcione. Preste atención también a la claridad del código, a usar nombres de variables descriptivos y a estructurar las condiciones de manera lógica y ordenada. Un/a buen/a programador/a no solo escribe códigos que funcionan, sino códigos que otros puedan entender.

Asimismo, esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos la lógica de cada problema y ayudarse a detectar posibles errores en el código Python.

Ejercicio 1: Calculadora de descuentos

Una tienda ofrece descuentos según el monto de la compra. Escriba un programa que solicite al usuario el monto total de su compra y aplique el siguiente criterio:

- Si el monto es menor a $5000, no hay descuento.
- Si el monto está entre $5000 y $10000 (inclusive), hay un 10% de descuento.
- Si el monto es mayor a $10000, hay un 15% de descuento.

El programa debe mostrar el monto original, el porcentaje de descuento aplicado y el monto final a pagar.

Ejercicio 2: Clasificador de edades

Escriba un programa que solicite la edad de una persona y la clasifique en una de las siguientes categorías:

- Niño: 0 a 12 años
- Adolescente: 13 a 17 años
- Adulto joven: 18 a 29 años
- Adulto: 30 a 59 años
- Adulto mayor: 60 años o más

Además, el programa debe validar que la edad ingresada sea un número positivo. Si el usuario ingresa un número negativo o cero, debe mostrar un mensaje de error.

Ejercicio 3: Calculadora de índice de masa corporal (IMC)

El índice de masa corporal se calcula dividiendo el peso (en kilogramos) por el cuadrado de la altura (en metros). Escriba un programa que solicite al usuario su peso y altura, calcule su IMC y lo clasifique según los siguientes rangos:

- Bajo peso: IMC menor a 18.5
- Peso normal: IMC entre 18.5 y 24.9
- Sobrepeso: IMC entre 25 y 29.9
- Obesidad: IMC mayor o igual a 30

El programa debe mostrar el valor del IMC calculado y la clasificación correspondiente.

Ejercicio 4: Validador de contraseñas

Escriba un programa que solicite al usuario que cree una contraseña y valide que cumpla con los siguientes criterios de seguridad:

- Debe tener al menos 8 caracteres de longitud.
- No debe contener espacios.

El programa debe indicar si la contraseña ingresada es válida o no. En caso de no serlo, debe informar de manera explícita cuál de los criterios establecidos no se cumple. Para verificar la longitud de la cadena de texto, puede utilizar la función len(contraseña), y para comprobar si la contraseña contiene espacios, puede emplear la expresión " " in contraseña, que devuelve True cuando existen espacios.

Ejercicio 5: Sistema de calificación de examen

Escriba un programa que simule un sistema de calificación. El usuario debe ingresar una nota numérica entre 0 y 100, y el programa debe validar que la nota esté en el rango válido (0-100). Si no lo está, mostrar un mensaje de error. Si la nota es válida, clasificarla según el siguiente criterio:

- Insuficiente: 0 a 59
- Aprobado: 60 a 69
- Notable: 70 a 84
- Sobresaliente: 85 a 100

Además, si la nota es mayor o igual a 95, el programa debe mostrar un mensaje adicional de felicitación.



ACTIVIDAD 3
Integrando funciones y estructuras de control

Llegamos a la actividad final del primer módulo, donde integraremos todos los conceptos aprendidos: variables, operadores, condicionales, bucles y funciones. Esta nos permitirá

construir programas más complejos y estructurados, organizando la lógica en funciones reutilizables que resuelven tareas específicas.

En el desarrollo de aplicaciones de inteligencia artificial, es fundamental saber descomponer problemas complejos en funciones más pequeñas y manejables. Esta habilidad nos acompañará durante toda la carrera, especialmente cuando trabajemos con procesamiento de datos, entrenamiento de modelos y análisis de resultados.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código en Python completo y funcional, incluyendo capturas de pantalla de la ejecución de los programas, mostrando diferentes casos de prueba para verificar que funcionan correctamente.



Recomendaciones

Al resolver estos ejercicios, no se concentre únicamente en que el programa funcione. Preste atención también a organizar el código en funciones claras y bien definidas, usar nombres descriptivos y comentar las partes más complejas. Un/a buen/a programador/a escribe código que otros puedan entender y reutilizar.

Además, esta actividad puede realizarse con un/a compañero/a para discutir las distintas formas de resolver cada problema. Cada ejercicio puede abordarse de múltiples maneras, y comparar diferentes soluciones enriquece enormemente el aprendizaje.

Ejercicio 1: Calculadora modular

Desarrollen un programa que implemente una calculadora utilizando funciones. Para ello, deberán definir las siguientes funciones:

- sumar(a, b): devuelve la suma de dos números.
- restar(a, b): devuelve la resta de dos números.
- multiplicar(a, b): devuelve la multiplicación de dos números.
- dividir(a, b): devuelve la división de dos números. Debe validar que el divisor no sea cero y en ese caso devolver None.
- potencia(base, exponente): devuelve base elevado a exponente.

El programa principal debe mostrar un menú que se repita hasta que el usuario elija salir. Para cada operación, debe solicitar los números necesarios, invocar la función correspondiente y mostrar el resultado. Si la división no es posible, debe mostrar un mensaje apropiado.

Ejercicio 2: Análisis de lista de números

Escriban un programa que le pida al usuario una serie de números y finalice cuando se introduzca el valor 0. Luego, mediante el uso de funciones, el programa debe calcular y mostrar distintas estadísticas sobre esos valores. Para ello, deberán definir las siguientes funciones:

- calcular_promedio(lista): recibe una lista de números y devuelve el promedio.
- encontrar_maximo(lista): recibe una lista de números y devuelve el mayor.

- encontrar_minimo(lista): recibe una lista de números y devuelve el menor.
- contar_pares(lista): recibe una lista de números y devuelve cuántos son pares.
- contar_impares(lista): recibe una lista de números y devuelve cuántos son impares.

El programa principal debe recopilar los números en una lista, invocar cada función y mostrar los resultados de forma clara y ordenada.



Este tipo de análisis estadístico básico es fundamental en procesamiento de información para IA. Saber calcular promedios, encontrar valores extremos y clasificar datos son operaciones que realizaremos constantemente.

Ejercicio 3: Validador de datos

Escriban un programa que solicite al usuario su edad, altura (en metros) y correo electrónico. El programa debe validar cada dato según los siguientes criterios:

- La edad debe ser un número positivo.
- La altura debe estar entre 0.5 y 2.5 metros.
- El email debe contener "@" y al menos un punto después del "@".

Si algún dato es inválido, el programa debe solicitar que se ingrese nuevamente hasta que sea correcto. Al final, debe mostrar todos los datos ingresados confirmando que son válidos.

Ejercicio 4: Conversor de temperaturas

Escriban un programa que convierta temperaturas entre diferentes escalas. El programa debe mostrar un menú con las siguientes opciones:

- Convertir de Celsius a Fahrenheit (fórmula: $F = C \times 9/5 + 32$)
- Convertir de Fahrenheit a Celsius (fórmula: $C = (F–32) \times 5/9$)
- Convertir de Celsius a Kelvin (fórmula: $K = C + 273.15$)
- Salir

El menú debe repetirse hasta que el usuario elija salir. Para cada conversión, debe solicitar la temperatura, realizar la conversión y mostrar el resultado de forma clara.

Ejercicio 5: Procesamiento de texto

Escriban un programa que solicite al usuario una frase y realice los siguientes análisis sobre ella:

- Contar cuántas vocales (a, e, i, o, u) contiene, sin distinguir entre mayúsculas y minúsculas.
- Contar cuántas palabras tiene.
- Mostrar el texto invertido.
- Determinar si la frase es un palíndromo (se lee igual de izquierda a derecha que de derecha a izquierda, ignorando espacios).

El programa debe mostrar todos los resultados de forma clara y organizada.

Ejercicio 6: Generador de tablas de multiplicar

Desarrolle un programa que pida un número y un valor límite para mostrar su tabla de multiplicar. Ambos valores deben ser positivos; en caso contrario, el programa deberá solicitarlos nuevamente hasta que sean válidos. Una vez verificados los datos, muestre la tabla de multiplicar correspondiente desde 1 hasta el límite indicado. Por ejemplo, si se elige el número 5 y el límite 10, el programa debe presentar:

5 X 1 = 5

5 X 2 = 10

...

5 X 10 = 50

Ejercicio 7: Buscador de números primos

Escriban un programa que solicite al usuario un número entero positivo y muestre todos los números primos menores o iguales a ese número. El programa debe validar que el número ingresado sea positivo. Si no lo es, debe solicitarlo nuevamente hasta obtener un valor válido.


Énfasis

Un número primo es aquel que solo es divisible por 1 y por sí mismo. Por ejemplo, si el usuario ingresa 20, el programa debe mostrar: 2, 3, 5, 7, 11, 13, 17, 19.

Ejercicio 8: Sistema de gestión de calificaciones

Escriban un programa que gestione las calificaciones de un estudiante. El programa debe:

- Solicitar el nombre del estudiante.
- Preguntar cuántas calificaciones se van a ingresar.
- Solicitar cada calificación una por una, validando que cada una esté entre 0 y 100. Si una calificación es inválida, debe solicitarla nuevamente.
- Calcular el promedio de todas las calificaciones.
- Determinar si el estudiante aprobó (promedio mayor o igual a 60) o reprobó.

Al finalizar, debe mostrar un reporte completo con el nombre del estudiante, todas las calificaciones ingresadas, el promedio y el estado (Aprobado/Reprobado).

Módulo 2

Microobjetivos

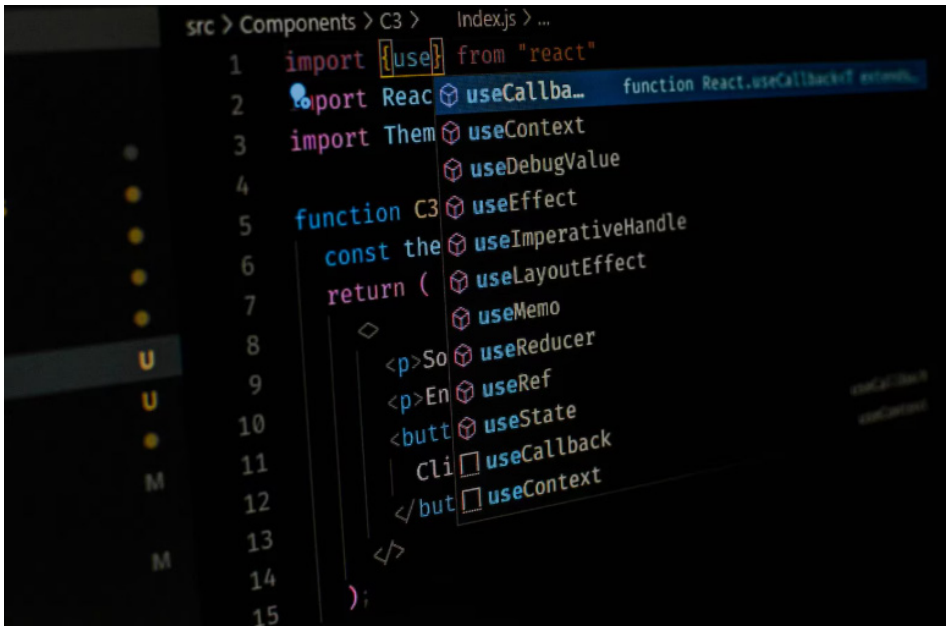
- Comprender las estructuras de datos fundamentales y sus métodos de manipulación, para saber procesar y organizar eficientemente conjuntos de información que alimentan sistemas de inteligencia artificial.

- Analizar las estructuras especializadas y sus principios de funcionamiento LIFO y FIFO, para aplicarlas en algoritmos de procesamiento secuencial y resolución de problemas que requieren recordar pasos anteriores.

- Aplicar técnicas avanzadas de manipulación de datos como comprensiones de listas, slicing y operaciones con conjuntos, para preprocesar y transformar información de manera eficiente en el contexto del desarrollo de aplicaciones de inteligencia artificial.

Contenidos

MÓDULO 2: ESTRUCTURA DE DATOS



Fuente: Freepik

Llegamos al segundo módulo de Programación para Inteligencia Artificial, donde daremos un paso fundamental en nuestro recorrido por la materia. En el módulo anterior construimos las bases: aprendimos a programar en Python, trabajamos con variables, estructuras condicionales, bucles y funciones. Escribimos nuestros primeros programas y comenzamos a pensar algorítmicamente. Ahora es momento de avanzar.

Este módulo está dedicado a las estructuras de datos, herramientas esenciales para cualquier desarrollador/a de sistemas inteligentes. Hasta ahora trabajamos con datos individuales: un número, una cadena, un booleano. Pero los problemas reales requieren

organizar y manipular colecciones de información: listas de valores, conjuntos de características, secuencias de decisiones, asociaciones entre elementos.

La diferencia entre usar la estructura correcta y la incorrecta puede ser enorme. No solo afecta si nuestro programa funciona, sino qué tan rápido lo hace y cuánta memoria consume. Un algoritmo que tarda segundos con una estructura puede tardar horas con otra. Esta es la razón por la que dominar estructuras de datos es tan importante en el desarrollo de aplicaciones de inteligencia artificial.

De esta manera, exploraremos estructuras fundamentales para el desarrollo de IA. Comenzaremos profundizando en las listas con métodos avanzados. Trabajaremos con tuplas, diccionarios y conjuntos. Avanzaremos hacia estructuras especializadas: pilas (stacks) para algoritmos de retroceso, colas (queues) para procesamiento secuencial, árboles binarios para clasificación jerárquica y grafos para representar relaciones complejas en sistemas de recomendación y análisis de redes.

Cada concepto irá acompañado de ejemplos prácticos que podrán ejecutar y modificar. Veremos cómo implementar estas estructuras en Python, cuándo usar cada una y cómo aplicarlas a problemas reales. La invitación es a experimentar y entender que elegir la estructura correcta es tan importante como escribir el algoritmo correcto.

Al finalizar, habrán ampliado su caja de herramientas como programadores/ras y desarrollado la intuición para reconocer qué estructura usar en cada situación. Esta habilidad es fundamental cuando trabajemos con bibliotecas de machine learning, implementemos algoritmos de búsqueda y construyamos sistemas inteligentes capaces de resolver problemas complejos.

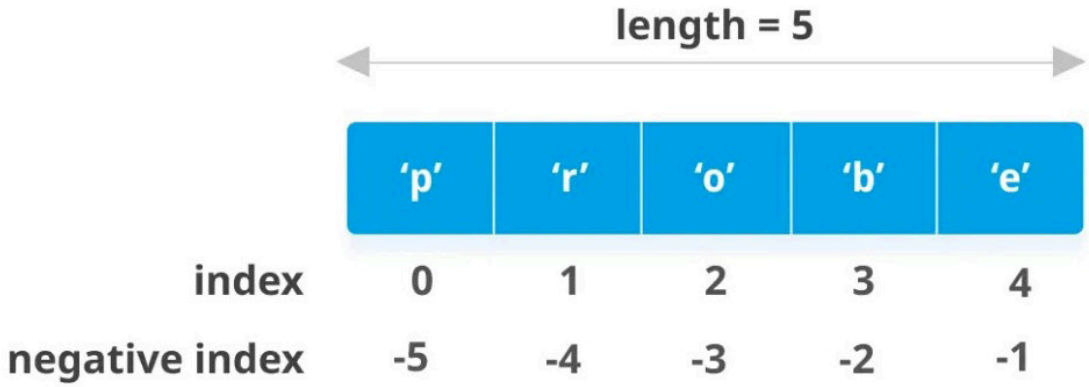
¡Comencemos!

Listas avanzadas y métodos de manipulación

En el módulo anterior tuvimos un primer acercamiento a las listas cuando trabajamos con estructuras repetitivas. Vimos que una lista es una colección ordenada de elementos que podemos modificar. Ahora profundizaremos en las listas, explorando métodos avanzados y técnicas de manipulación que nos permitirán trabajar con datos de forma más eficiente y elegante.

Recordemos que las listas se crean usando corchetes y pueden contener cualquier tipo de dato:

```
numeros = [10, 20, 30, 40, 50]
nombres = ["Ana", "Carlos", "María"]
mezclada = [1, "Python", 3.14, True]
vacia = []
```



Fuente: https://somoshackersdelaprogramacion.es/

Una característica fundamental de las listas es que son mutables, es decir, podemos modificar sus elementos después de crearlas. Esto las diferencia de otros tipos de datos que veremos más adelante.

Métodos de manipulación

Las listas en Python tienen métodos integrados que nos permiten realizar operaciones comunes de manera sencilla. Veamos los más importantes y útiles.

Para agregar elementos al final de una lista usamos append():

```
estudiantes = ["Ana", "Carlos"]
estudiantes.append("María")
print(estudiantes) # ['Ana', 'Carlos', 'María']
```

Si queremos agregar un elemento en una posición específica, usamos insert():

```
estudiantes.insert(1, "Diego") # Inserta en posición 1
print(estudiantes) # ['Ana', 'Diego', 'Carlos', 'María']
```

Para eliminar elementos tenemos varias opciones. El método remove() elimina la primera ocurrencia de un valor:

```
estudiantes.remove("Carlos")
print(estudiantes) # ['Ana', 'Diego', 'María']
```

El método pop() permite quitar un elemento de una lista y, al mismo tiempo, obtener su valor. Si se utiliza sin argumentos, elimina el último elemento de la lista. Si se le pasa un índice, elimina y devuelve el elemento ubicado en esa posición:

```
ultimo = estudiantes.pop()
print(ultimo) # María
print(estudiantes) # ['Ana', 'Diego']

primero = estudiantes.pop(0)
print(primero) # Ana
print(estudiantes) # ['Diego']
```

Para extender una lista con los elementos de otra lista usamos extend():

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
lista1.extend(lista2)
print(lista1) # [1, 2, 3, 4, 5, 6]
```

Podemos invertir el orden de los elementos con reverse():

```
numeros = [1, 2, 3, 4, 5]
numeros.reverse()
print(numeros) # [5, 4, 3, 2, 1]
```

Para ordenar una lista usamos sort(). Por defecto ordena de menor a mayor:

```
valores = [15, 8, 23, 4, 16]
valores.sort()
print(valores) # [4, 8, 15, 16, 23]
```

Para ordenar en orden descendente:

```
valores.sort(reverse=True)
print(valores) # [23, 16, 15, 8, 4]
```

Finalmente, si queremos obtener una copia ordenada sin modificar la lista original, usamos la función sorted():

```
original = [15, 8, 23, 4, 16]
ordenada = sorted(original)
print(original) # [15, 8, 23, 4, 16]
print(ordenada) # [4, 8, 15, 16, 23]
```

Slicing avanzado

Además del acceso a elementos individuales mediante índices, es posible trabajar con porciones de una lista utilizando slicing. Esta técnica permite extraer subconjuntos de elementos de manera flexible y concisa.

La sintaxis general es lista[inicio:fin:paso], donde inicio indica la posición desde la cual se comienza a tomar elementos (incluida), fin señala la posición donde se detiene la selección (excluida) y paso determina el intervalo o salto entre los elementos seleccionados.

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numeros[2:7:1]) # [2, 3, 4, 5, 6]
print(numeros[1:7]) # [1,2, 3, 4, 5, 6] solo marcamos inicio y fin
print(numeros[:5]) # [0, 1, 2, 3, 4] solo marcamos el fin
print(numeros[5:]) # [5, 6, 7, 8, 9] solo marcamos el inicio
print(numeros[::2]) # [0, 2, 4, 6, 8] (de 2 en 2)
print(numeros[1::2]) # [1, 3, 5, 7, 9] (impares)
print(numeros[::-1]) # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (invertida)
```

Como puede observar, no siempre es necesario usar todos los componentes de la sintaxis. Si se indica solo el valor de inicio, la porción obtenida va desde esa posición hasta el final de la lista, ya que no se establece un límite. De manera análoga, si se especifica únicamente el fin, la selección comienza desde la posición 0 y se detiene en el valor indicado. Cuando no se explicita el paso, este toma por defecto el valor 1, es decir, se recorren los elementos de uno en uno. Finalmente, la notación [::-1] resulta especialmente útil, ya que permite invertir cualquier secuencia de forma rápida y sencilla.

Comprensiones de listas

Las comprensiones de listas son una forma concisa y elegante de crear nuevas listas a partir de otras secuencias. Esta característica de Python es extremadamente potente y se usa constantemente en el procesamiento de datos.

La sintaxis básica es [expresión for elemento in secuencia]:

```
# Crear una lista con los cuadrados de los números del 1 al 10
cuadrados = [n ** 2 for n in range(1, 11)]
print(cuadrados) # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Podemos agregar una condición para filtrar elementos:

```
# Solo los números pares del 1 al 20
pares = [n for n in range(1, 21) if n % 2 == 0]
print(pares) # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Las comprensiones de listas también pueden transformar elementos:

```
nombres = [“ana”, “carlos”, “maría”]
nombres_mayus = [nombre.upper() for nombre in nombres]
print(nombres_mayus) # ['ANA', 'CARLOS', 'MARÍA']
```

Podemos combinar transformación y filtrado:

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
cuadrados_pares = [n ** 2 for n in numeros if n % 2 == 0]
print(cuadrados_pares) # [4, 16, 36, 64, 100]
```

Funciones útiles con listas

Python proporciona varias funciones integradas que trabajan con listas y simplifican muchas operaciones comunes:

```
numeros = [15, 8, 23, 4, 16, 42, 11]

print(len(numeros)) # 7 (cantidad de elementos)
print(sum(numeros)) # 119 (suma de todos los elementos)
print(min(numeros)) # 4 (elemento más pequeño)
print(max(numeros)) # 42 (elemento más grande)
```

Para verificar si un elemento está en una lista usamos in:

```
if 23 in numeros:
    print(“El 23 está en la lista”)

if 100 not in numeros:
    print(“El 100 no está en la lista”)
```

La función enumerate() es muy útil cuando necesitamos tanto el índice como el valor:

```
frutas = [“manzana”, “banana”, “naranja”]
for indice, fruta in enumerate(frutas):
    print(f”Posición {indice}: {fruta}”)
```

Listas anidadas

Las listas en Python pueden almacenar cualquier tipo de dato, incluso otras listas. Esta característica permite construir estructuras de datos multidimensionales, que resultan especialmente útiles para representar tablas, matrices o conjuntos de registros de manera clara y organizada.

```
# Lista de listas representando una tabla
tabla = [
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]
]

# Acceder a elementos
print(tabla[0]) # [1, 2, 3] (primera fila)
print(tabla[0][0]) # 1 (elemento en fila 0, columna 0)
print(tabla[1][2]) # 6 (elemento en fila 1, columna 2)
```

Podemos recorrer listas anidadas con bucles anidados:

```
for fila in tabla:
    for elemento in fila:
        print(elemento, end=" ")
    print() # salto de línea después de cada fila
```

Este tipo de estructura es fundamental cuando trabajamos con datasets, donde cada sublista representa una instancia con múltiples características.

Algunas aplicaciones: procesamiento de datos

A continuación, se integran los conceptos trabajados mediante un ejemplo inspirado en una situación habitual del machine learning. Se propone copiar el código en su editor, ejecutarlo, observar su funcionamiento y luego modificar los datos o incorporar nuevos cálculos según lo considere pertinente. Supongamos que se dispone de un conjunto de datos con características numéricas y que se desea realizar un primer preprocesamiento antes de utilizarlos en un modelo.

```
# Dataset de ejemplo: [edad, altura_cm, peso_kg]
datos = [
 [25, 175, 70],
 [30, 180, 85],
 [22, 165, 60],
 [35, 178, 80],
 [28, 172, 75]
]

# Extraer solo las edades
edades = [persona[0] for persona in datos]
print("Edades:", edades)

# Calcular estadísticas
edad_promedio = sum(edades) / len(edades)
edad_minima = min(edades)
edad_maxima = max(edades)

print(f"Edad promedio: {edad_promedio}")
print(f"Edad mínima: {edad_minima}")
print(f"Edad máxima: {edad_maxima}")

# Filtrar personas mayores de 25 años
mayores_25 = [persona for persona in datos if persona[0] > 25]
print(f"Personas mayores de 25: {len(mayores_25)}")
```

Este tipo de operaciones son fundamentales cuando trabajamos con datasets para entrenar modelos de inteligencia artificial.

Tuplas: secuencias inmutables

Las tuplas se parecen a las listas porque permiten almacenar varios elementos en un orden determinado. Sin embargo, existe una diferencia clave que las distingue: las tuplas son inmutables. Esto significa que, una vez creada una tupla, su contenido no puede modificarse; no es posible cambiar valores, agregar nuevos elementos ni eliminar los existentes.



Este rasgo no es un detalle menor. Justamente por ser inmutables, las tuplas resultan especialmente adecuadas para representar datos que deben mantenerse constantes a lo largo del programa, evitando modificaciones accidentales y aportando mayor seguridad y claridad al código.

Creamos tuplas usando paréntesis:

```
coordenadas = (10, 20)
datos_persona = ("Ana", 25, "Ingeniera")
numeros = (1, 2, 3, 4, 5)
```

También podemos crear tuplas sin paréntesis, simplemente separando valores con comas:

```
punto = 5, 10
print(punto) # (5, 10)
print(type(punto)) # <class 'tuple'>
```

Para crear una tupla de un solo elemento, debemos incluir una coma:

```
tupla_unitaria = (5,)
no_es_tupla = (5) # Esto es solo el número 5 entre paréntesis

print(type(tupla_unitaria)) # <class 'tuple'>
print(type(no_es_tupla)) # <class 'int'>
```

Accedemos a los elementos de una tupla igual que con las listas:

```
coordenadas = (10, 20, 30)
print(coordenadas[0]) # 10
print(coordenadas[-1]) # 30
print(coordenadas[1:]) # (20, 30)
```

Aunque no podemos modificar una tupla, sí podemos recorrerla y realizar operaciones que no alteren su contenido:

```
valores = (5, 10, 15, 20)

for valor in valores:
    print(valor)

print(len(valores)) # 4
print(sum(valores)) # 50
print(min(valores)) # 5
print(max(valores)) # 20
```

¿Cuándo usar tuplas?

Las tuplas son especialmente útiles cuando necesitamos representar datos que están asociados entre sí y que, por su naturaleza, no deberían modificarse a lo largo del programa. En estos casos, la inmutabilidad no es una limitación, sino una ventaja. Por ejemplo, para coordenadas geográficas o puntos en un plano:

```
ubicacion = (40.7128, -74.0060) # latitud, longitud
```

Para datos de configuración que no deben modificarse:

```
config = ("localhost", 8080, "admin")
```

Asimismo, cuando una función devuelve varios valores separados por comas, Python los agrupa automáticamente en una tupla. Luego, es posible recibir esos valores asignándolos a distintas variables mediante el desempaquetado, lo que permite trabajar con cada resultado de forma clara y directa:

```
def obtener_estadisticas(numeros):
 return min(numeros), max(numeros), sum(numeros) / len(numeros)

# Desempaquetar los valores en tres variables
minimo, maximo, promedio = obtener_estadisticas([10, 20, 30, 40])
print(f"Min: {minimo}, Max: {maximo}, Prom: {promedio}")

# También podemos capturar la tupla completa
estadisticas = obtener_estadisticas([10, 20, 30, 40])
print(estadisticas) # (10, 40, 25.0)
print(type(estadisticas)) # <class 'tuple'>
```

Énfasis

En inteligencia artificial, las tuplas se usan frecuentemente para representar datos que van naturalmente juntos y no deben modificarse. Por ejemplo, las dimensiones de una imagen (ancho, alto, canales de color) se representan como `(1920, 1080, 3)`, o las coordenadas de un punto en un espacio bidimensional como `(x, y)`. También se usan para indicar el tamaño y forma de estructuras de datos multidimensionales que usaremos más adelante en machine learning.

Diccionarios: asociando claves y valores

Las listas y tuplas organizan datos por posición, usando índices numéricos. Pero muchas veces necesitamos organizar información por nombres o etiquetas. Para eso usamos diccionarios.



Un diccionario almacena pares de clave-valor. En lugar de acceder a los datos por índice numérico, los accedemos por su clave, que puede ser cualquier valor inmutable (generalmente strings).

Creamos diccionarios usando llaves {}:

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Lic. en Inteligencia Artificial",
    "promedio": 8.5
}

print(estudiante["nombre"]) # Ana
print(estudiante["promedio"]) # 8.5
```

A los diccionarios se les pueden agregar nuevas claves o modificar valores existentes de manera directa, indicando la clave correspondiente:

```
estudiante["email"] = "ana@ejemplo.com"
estudiante["edad"] = 21
print(estudiante)
```

Si se desea eliminar una clave junto con su valor, se utiliza la instrucción *del*:

```
del estudiante["email"]
```

Para comprobar si una clave existe dentro del diccionario, se puede usar el operador in

```
if "promedio" in estudiante:
    print(f"El promedio es {estudiante['promedio']}")

if "telefono" not in estudiante:
    print("No hay teléfono registrado")
```

Otra alternativa segura para acceder a los valores es el método get(). Este método devuelve *None* si la clave no existe, o bien un valor predeterminado si se lo indicamos:

```
print(estudiante.get("nombre")) # Ana
print(estudiante.get("telefono")) # None
print(estudiante.get("telefono", "N/A")) # N/A
```

Finalmente, los diccionarios permiten obtener sus claves, sus valores o todos los pares clave–valor mediante los métodos keys(), values() e items(), respectivamente:

```
print(estudiante.keys()) # dict_keys(['nombre', 'edad', 'carrera', 'promedio'])
print(estudiante.values()) # dict_values(['Ana', 21, 'Inteligencia Artificial', 8.5])
print(estudiante.items()) # dict_items([('nombre', 'Ana'), ...])
```

Para recorrer un diccionario:

```
# Solo las claves
for clave in estudiante:
    print(clave)

# Claves y valores
for clave, valor in estudiante.items():
    print(f"{clave}: {valor}")
```

Un uso común de los diccionarios es **contar la frecuencia de elementos.** Imaginemos que queremos analizar qué palabras aparecen más en un texto:

```
def contar_palabras(texto):
    palabras = texto.lower().split()
    frecuencias = {}

for palabra in palabras:
    if palabra in frecuencias:
        frecuencias[palabra] += 1
    else:
        frecuencias[palabra] = 1

    return frecuencias

texto = "python es genial python es poderoso python es versátil"
frecuencias = contar_palabras(texto)
print(frecuencias)
# {'python': 3, 'es': 3, 'genial': 1, 'poderoso': 1, 'versátil': 1}
```

En este caso, el diccionario permite asociar cada palabra con la cantidad de veces que aparece, una operación muy habitual en análisis de texto y procesamiento de datos.

También es común combinar listas y diccionarios para representar información más compleja. Por ejemplo, puede utilizarse una lista de diccionarios para almacenar datos de varios estudiantes, donde cada diccionario representa un registro con múltiples atributos:

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "promedio": 8.5},
    {"nombre": "Carlos", "edad": 22, "promedio": 7.0},
    {"nombre": "Beatriz", "edad": 21, "promedio": 9.0}
]

# Encontrar estudiantes con promedio mayor a 8
destacados = [est for est in estudiantes if est["promedio"] >= 8.0]

for estudiante in destacados:
    print(f"{estudiante['nombre']}: {estudiante['promedio']}")
```

Este tipo de estructuras anidadas aparece con frecuencia en contextos de análisis de datos y machine learning, donde cada registro tiene múltiples características.

Conjuntos: colecciones sin duplicados

Por otra parte, los conjuntos (sets) son colecciones desordenadas de elementos únicos. No admiten duplicados y resultan especialmente eficientes para verificar pertenencia y realizar operaciones matemáticas entre conjuntos.

Énfasis

Es importante destacar que un conjunto no mantiene un orden y no permite repetir elementos, lo que lo vuelve adecuado para representar colecciones donde solo importa la presencia o ausencia de valores, es decir, es ideal para operaciones matemáticas como la unión, intersección y diferencia.

Creamos conjuntos usando llaves o la función set():

```
numeros = {1, 2, 3, 4, 5}
vocales = {'a', 'e', 'i', 'o', 'u'}
conjunto_vacio = set() # {} crea un diccionario vacío, no un conjunto
```

También es posible construir un conjunto a partir de una lista, eliminando automáticamente los elementos duplicados:

```
lista_con_duplicados = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5]
conjunto_sin_duplicados = set(lista_con_duplicados)
print(conjunto_sin_duplicados) # {1, 2, 3, 4, 5}
```

Para agregar elementos usamos add():

```
frutas = {"manzana", "banana"}
frutas.add("naranja")
print(frutas) # {'manzana', 'banana', 'naranja'}
```

Mientras que para eliminarlos pueden emplearse remove() o discard(), con la diferencia

de que discard() no genera error si el elemento no existe:

```
frutas.remove("banana") # Error si no existe
frutas.discard("pera") # No genera error si no existe
```

Operaciones matemáticas con conjuntos

Como se planteó anteriormente, una de las principales ventajas de los conjuntos es la facilidad con la que permiten realizar operaciones matemáticas como unión, intersección y diferencia:

```
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

# Unión: elementos en A o en B
union = set_a | set_b
print(union) # {1, 2, 3, 4, 5, 6, 7, 8}

# Intersección: elementos en A y en B
interseccion = set_a & set_b
print(interseccion) # {4, 5}

# Diferencia: elementos en A pero no en B
diferencia = set_a–set_b
print(diferencia) # {1, 2, 3}

# Diferencia simétrica: elementos en A o B pero no en ambos
dif_simetrica = set_a ^ set_b
print(dif_simetrica) # {1, 2, 3, 6, 7, 8}
```

Aplicaciones: análisis de similitud

Finalmente, los conjuntos resultan muy útiles para comparar colecciones de datos. Por ejemplo, pueden emplearse para identificar elementos comunes entre distintas muestras o para contabilizar la cantidad total de valores distintos observados:

```
# Características observadas en diferentes muestras
muestra_1 = {“fiebre”, “tos”, “dolor_cabeza”, “fatiga”}
muestra_2 = {“tos”, “fatiga”, “mareo”, “nausea”}
muestra_3 = {“fiebre”, “tos”, “dolor_garganta”}

# Síntomas comunes a todas
comunes = muestra_1 & muestra_2 & muestra_3
print(f”Síntomas comunes: {comunes}”) # {‘tos’}

# Todos los síntomas observados
todos = muestra_1 | muestra_2 | muestra_3
print(f”Total de síntomas distintos: {len(todos)}”)
```



En este momento, está en condiciones de realizar la **actividad 1** del módulo.

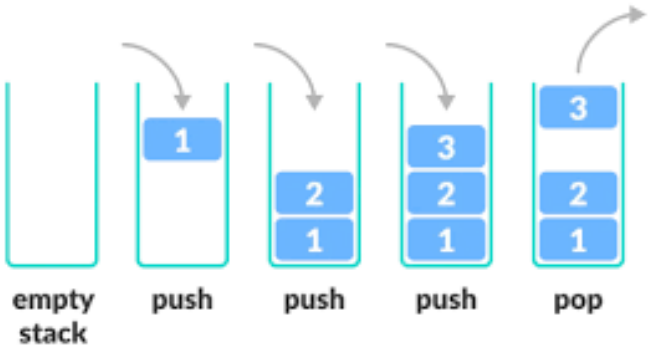
Pilas (Stack): último en entrar, primero en salir

Ahora avanzamos hacia estructuras de datos más especializadas.

Énfasis

Las pilas son estructuras que siguen el principio LIFO (Last In, First Out): el último elemento agregado es el primero en ser retirado, como una pila de platos. Solo podemos agregar o quitar elementos desde el tope de la pila.

Imaginemos una pila de platos en una cocina: al lavarlos, los colocamos uno sobre otro formando una pila. Cuando necesitamos un plato, tomamos el que está arriba, que es el último que se colocó. No es posible retirar un plato del medio sin antes quitar los que están encima. Esta forma de funcionamiento refleja exactamente la lógica de una pila en programación.



Fuente: https://somoshackersdelaprogramacion.es/

Las pilas tienen dos operaciones fundamentales:

- **push**: agregar un elemento al tope
- **pop**: retirar el elemento del tope

En Python, podemos implementar una pila usando simplemente una lista y sus métodos append() y pop():

```
# Crear una pila vacía (es solo una lista)
pila = []

# Agregar elementos al tope (push)
pila.append(10)
pila.append(20)
pila.append(30)

print(pila) # [10, 20, 30]

# Ver el elemento del tope sin sacarlo
if len(pila) > 0:
    print(f"Tope: {pila[-1]}") # 30

# Sacar elementos del tope (pop)
elemento = pila.pop()
print(f"Sacamos: {elemento}") # 30
print(f"Pila ahora: {pila}") # [10, 20]

elemento = pila.pop()
print(f"Sacamos: {elemento}") # 20
print(f"Pila ahora: {pila}") # [10]
```

Observemos cómo el último elemento agregado (30) es el primero en salir. Esto es LIFO en acción. Asímismo, para hacer el código más legible, podemos crear funciones que encapsulen estas operaciones:

```
def crear_pila():
    return []

def esta_vacia(pila):
    return len(pila) == 0

def push(pila, elemento):
    pila.append(elemento)

def pop(pila):
    if not esta_vacia(pila):
        return pila.pop()
    return None

def ver_tope(pila):
    if not esta_vacia(pila):
        return pila[-1]
    return None

def tamano(pila):
    return len(pila)
```

Ahora usemos estas funciones:

```
# Crear y usar una pila
mi_pila = crear_pila()

# Agregar elementos
push(mi_pila, "A")
push(mi_pila, "B")
push(mi_pila, "C")

print(f"Tamaño: {tamano(mi_pila)}") # 3
print(f"Tope: {ver_tope(mi_pila)}") # C

# Sacar elementos
print(f"Sacamos: {pop(mi_pila)}") # C
print(f"Sacamos: {pop(mi_pila)}") # B
print(f"Nuevo tope: {ver_tope(mi_pila)}") # A
```

Un uso clásico de las pilas es verificar si los paréntesis de una expresión matemática están correctamente balanceados. Cada vez que aparece un paréntesis de apertura (, se lo

agrega a la pila. Cuando aparece un paréntesis de cierre), se comprueba que exista un paréntesis de apertura previamente almacenado en la pila para poder emparejarlo:

```
def parentesis_balanceados(expresion):
    pila = []

for caracter in expresion:
    if caracter == '(':
    pila.append(caracter) # push
elif caracter == ')':
    if len(pila) == 0: # pila vacía, no hay apertura
return False
    pila.pop() # sacamos la apertura correspondiente

# Al final, la pila debe estar vacía
return len(pila) == 0

# Probar
print(parentesis_balanceados("((()))")) # True–todos balanceados
print(parentesis_balanceados("(()")) # False–falta un cierre
print(parentesis_balanceados("(a + b) * c")) # True–balanceados
print(parentesis_balanceados("())")) # False–cierre sin apertura
```

Las pilas resultan ideales para implementar funcionalidades de "deshacer" o para gestionar el historial de navegación. En estos casos, cada acción nueva se coloca en la parte superior de la pila, y al deshacer o retroceder se recupera la última acción realizada, siguiendo el principio de último en entrar, primero en salir.

```
def crear_navegador():
 return {
 “historial”: [],
 “pagina_actual”: None
 }

def visitar(navegador, url):
 # Guardar página actual en el historial antes de cambiar
 if navegador[“pagina_actual”] is not None:
 navegador[“historial”].append(navegador[“pagina_actual”])
 navegador[“pagina_actual”] = url
 print(f”Visitando: {url}”)

def volver(navegador):
 if len(navegador[“historial”]) == 0:
 print(“No hay páginas anteriores”)
 return

 # Sacar la página anterior del historial
 navegador[“pagina_actual”] = navegador[“historial”].pop()
 print(f”Volviendo a: {navegador[‘pagina_actual’]}”)

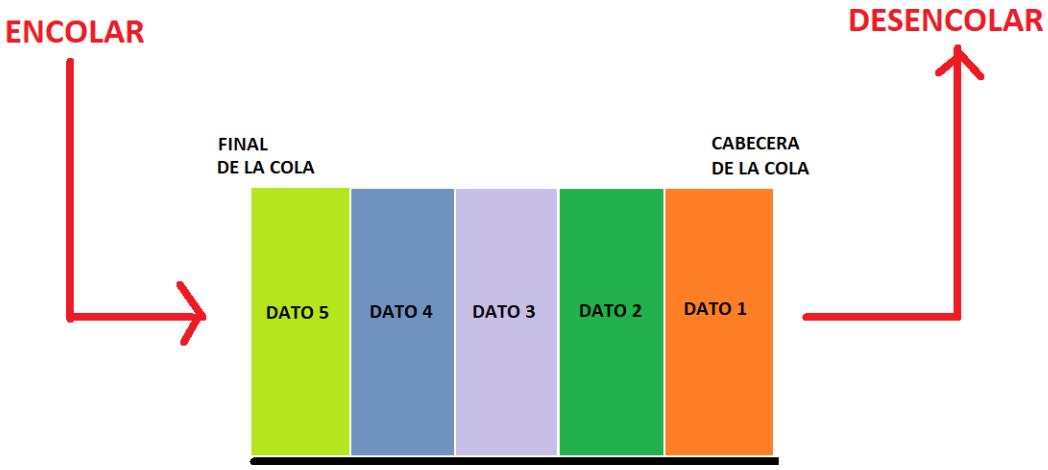
# Usar el navegador
mi_navegador = crear_navegador()
visitar(mi_navegador, “google.com”)
visitar(mi_navegador, “wikipedia.org”)
visitar(mi_navegador, “github.com”)
volver(mi_navegador) # Vuelve a wikipedia.org
volver(mi_navegador) # Vuelve a google.com
volver(mi_navegador) # No hay páginas anteriores
```

Finalmente, las pilas cumplen un papel central en inteligencia artificial, ya que permiten recordar pasos anteriores y retroceder cuando una decisión no conduce a una solución válida. Se utilizan en algoritmos que exploran alternativas de manera secuencial, en la evaluación de expresiones matemáticas complejas y en situaciones en las que el programa debe ensayar una solución y, si no resulta adecuada, volver atrás para intentar otra. Más adelante en la carrera se abordarán estos algoritmos con mayor profundidad.

Colas (Queue): primero en entrar, primero en salir

Énfasis

Una cola es una estructura que sigue el principio FIFO (First In, First Out), es decir, el primer elemento agregado es el primero en ser retirado. Los elementos se agregan al final y se retiran desde el frente. Para entenderlo mejor, pensemos en una fila de personas esperando en un banco. La primera persona que llega es la primera en ser atendida. No podemos atender primero a quien llegó último, eso sería injusto. Las colas en programación funcionan exactamente así: respetan el orden de llegada.



Fuente: https://programacionpython80889555.wordpress.com/

Un ejemplo gráfico habitual de este concepto es una fila de supermercado: quien llega primero avanza primero, mientras que el resto espera detrás siguiendo ese mismo orden



Fuente: Freepik

Las operaciones fundamentales son:

- **enqueue**: agregar un elemento al final
- **dequeue**: retirar el elemento del frente

En Python, una cola puede implementarse de manera sencilla utilizando una lista. Para agregar elementos se usa append(), que los coloca al final, y para retirar el primero se utiliza pop(0):

```
# Crear una cola vacía (es solo una lista)
cola = []

# Agregar elementos al final (enqueue)
cola.append("Primero")
cola.append("Segundo")
cola.append("Tercero")

print(cola) # ['Primero', 'Segundo', 'Tercero']

# Ver el elemento del frente sin sacarlo
if len(cola) > 0:
    print(f"Frente: {cola[0]}") # Primero

# Sacar elementos del frente (dequeue)
elemento = cola.pop(0)
print(f"Atendimos: {elemento}") # Primero
print(f"Cola ahora: {cola}") # ['Segundo', 'Tercero']

elemento = cola.pop(0)
print(f"Atendimos: {elemento}") # Segundo
print(f"Cola ahora: {cola}") # ['Tercero']
```

Puede observarse que el primer elemento agregado es también el primero en salir, lo que ilustra claramente el comportamiento FIFO.

Al igual que con las pilas, es posible definir funciones para organizar mejor el código y hacerlo más claro:

```
def crear_cola():
    return []

def esta_vacia(cola):
    return len(cola) == 0

def enqueue(cola, elemento):
    cola.append(elemento)

def dequeue(cola):
    if not esta_vacia(cola):
        return cola.pop(0)
    return None

def ver_frente(cola):
    if not esta_vacia(cola):
        return cola[0]
    return None

def tamano(cola):
    return len(cola)
```

Y luego utilizarlas de la siguiente manera:

```
# Crear y usar una cola
mi_cola = crear_cola()

# Agregar elementos
enqueue(mi_cola, "Tarea A")
enqueue(mi_cola, "Tarea B")
enqueue(mi_cola, "Tarea C")

print(f"Tamaño: {tamano(mi_cola)}") # 3
print(f"Frente: {ver_frente(mi_cola)}") # Tarea A

# Procesar elementos
print(f"Procesamos: {dequeue(mi_cola)}") # Tarea A
print(f"Procesamos: {dequeue(mi_cola)}") # Tarea B
print(f"Nuevo frente: {ver_frente(mi_cola)}") # Tarea C
```

Las colas son especialmente útiles en situaciones donde las solicitudes deben procesarse en el orden en que llegan. Un ejemplo típico es un sistema de impresión:

```
def crear_sistema_impresion():
    return {
        “cola_trabajos”: [],
        “trabajos_completados”: 0
}

def agregar_trabajo(sistema, documento):
    sistema[“cola_trabajos”].append(documento)
    print(f”Trabajo agregado a la cola: {documento}”)

def procesar_siguiente(sistema):
    if len(sistema[“cola_trabajos”]) == 0:
        print(“No hay trabajos pendientes”)
        return

    trabajo = sistema[“cola_trabajos”].pop(0)
    sistema[“trabajos_completados”] += 1
    print(f”Imprimiendo: {trabajo}”)

def trabajos_pendientes(sistema):
    return len(sistema[“cola_trabajos”])

# Usar el sistema
impresora = crear_sistema_impresion()
agregar_trabajo(impresora, “informe.pdf”)
agregar_trabajo(impresora, “presentacion.pptx”)
agregar_trabajo(impresora, “tesis.docx”)

print(f”Trabajos en cola: {trabajos_pendientes(impresora)}”) # 3
procesar_siguiente(impresora) # Imprime informe.pdf
procesar_siguiente(impresora) # Imprime presentacion.pptx
print(f”Trabajos en cola: {trabajos_pendientes(impresora)}”) # 1
print(f”Trabajos completados: {impresora[‘trabajos_completados’]}”) # 2
```

Otro ejemplo clásico puede ser la atención de personas en una fila:

```
def simular_atencion_clientes():
    cola_clientes = []

    # Llegan clientes
    clientes = ["Ana", "Carlos", "María", "Diego", "Beatriz"]
    for cliente in clientes:
        cola_clientes.append(cliente)
        print(f"{cliente} entró a la cola. Posición: {len(cola_clientes)}")

print(f"\nTotal en cola: {len(cola_clientes)}")
print("\nComienza la atención:")

# Atender clientes
numero_atencion = 1
while len(cola_clientes) > 0:
    cliente = cola_clientes.pop(0)
    print(f"Ventanilla {numero_atencion}: Atendiendo a {cliente}")
    print(f" Quedan {len(cola_clientes)} clientes esperando")
    numero_atencion += 1

# Ejecutar simulación
simular_atencion_clientes()
```



En este momento, está en condiciones de realizar la **actividad 2** del módulo.

Comparación entre estructuras FIFO y LIFO (pilas y colas)

Es importante comprender una diferencia fundamental entre estas dos estructuras de datos: el orden en el que los elementos son retirados. A continuación, se muestra un ejemplo con los mismos elementos almacenados primero en una pila y luego en una cola, para observar cómo cambia el orden de salida.

```
# Mismos elementos, diferente orden de salida
elementos = ["A", "B", "C", "D"]

# Con pila (LIFO)
pila = []
for elem in elementos:
    pila.append(elem)

print("Sacando de la pila:")
while len(pila) > 0:
    print(pila.pop(), end=" ") # D C B A
print()

# Con cola (FIFO)
cola = []
for elem in elementos:
    cola.append(elem)

print("Sacando de la cola:")
while len(cola) > 0:
    print(cola.pop(0), end=" ") # A B C D
```

Como puede observarse, en la pila el último elemento agregado es el primero en salir (LIFO), mientras que en la cola el primer elemento agregado es el primero en ser retirado (FIFO), aun cuando ambas contienen exactamente los mismos datos.

Las colas son fundamentales en inteligencia artificial para gestionar tareas que deben procesarse respetando el orden de llegada. Se utilizan en sistemas que exploran alternativas de manera organizada, en el procesamiento secuencial de múltiples tareas y en simulaciones donde los eventos deben ocurrir en un orden específico. También resultan clave en sistemas que procesan datos en tiempo real, como chatbots que responden consultas en el orden en que son recibidas.

Con esto, se concluye el abordaje de las bases del manejo de estructuras de datos en Python. Se trabajó con listas, tuplas, diccionarios y conjuntos para almacenar y organizar información, y se introdujeron las pilas y colas como estructuras especializadas que continuarán apareciendo a lo largo de la carrera.

En el próximo módulo nos sumergiremos en la programación orientada a objetos, un paradigma fundamental para organizar código complejo. Aprenderemos a crear clases, trabajar con objetos, y aplicar conceptos como herencia y polimorfismo. También veremos

cómo implementar estructuras más complejas como árboles binarios y grafos utilizando este nuevo paradigma.

Más adelante, cuando trabajemos con bibliotecas de machine learning como NumPy, Pandas, Scikit-learn y TensorFlow, veremos que todo lo aprendido aquí se aplica constantemente: estas bibliotecas utilizan internamente las mismas estructuras de datos que acabamos de estudiar, optimizadas para procesar grandes volúmenes de información.



Evaluación

Usted ya está en condiciones de realizar la **primera parte** de la **evaluación integradora.**

Módulo 2 | Glosario

Comprensión de listas: sintaxis concisa de Python para crear listas aplicando una expresión y opcionalmente un filtro sobre una secuencia existente. Muy utilizada en procesamiento de datos para transformar y filtrar información eficientemente.

Dataset: conjunto de datos organizado que se utiliza para entrenar, validar o probar modelos de machine learning. Típicamente se representa como una colección de instancias donde cada instancia tiene múltiples características.

Dequeue: operación de colas que retira el elemento del frente de la estructura, procesando los elementos en el orden en que fueron agregados.

Enqueue: operación de colas que agrega un elemento al final de la estructura, respetando el principio FIFO de primero en entrar, primero en salir.

FIFO (First In, First Out): principio de las colas donde el primer elemento agregado es el primero en ser retirado, como una fila de personas donde quien llega primero es atendido primero.

Inmutable: característica de un tipo de dato que no permite modificar su contenido una vez creado. Cualquier operación que parezca modificarlo en realidad crea un nuevo objeto. Las tuplas y cadenas de texto son inmutables.

LIFO (Last In, First Out): principio de las pilas donde el último elemento agregado es el primero en ser retirado, como una pila de platos donde solo podemos tomar el de arriba.

Mutable: característica de un tipo de dato que permite modificar su contenido después de haber sido creado. Las listas y diccionarios son mutables, mientras que las tuplas y strings son inmutables.

NLP (Natural Language Processing): procesamiento de lenguaje natural, campo de la IA que permite a las computadoras entender, interpretar y generar lenguaje humano. Usa estructuras de datos como diccionarios para análisis de frecuencia de palabras.

Normalización: proceso de escalar valores numéricos a un rango común (típicamente 0 a 1) para que diferentes características tengan peso comparable en algoritmos de machine learning.

Pop: operación fundamental en pilas y colas que retira un elemento de la estructura. En pilas retira del tope, en colas retira del frente.

Push: operación fundamental en pilas que agrega un elemento al tope de la estructura. En Python se implementa típicamente usando el método append() de las listas.

Slicing: técnica para extraer porciones o subsecuencias de una lista, tupla o cadena usando la sintaxis [inicio:fin:paso]. Permite acceder a rangos de elementos sin necesidad de bucles.

Módulo 2 | Actividades

ACTIVIDAD 1
Trabajando con estructuras de datos fundamentales

Hemos aprendido a trabajar con listas avanzadas, tuplas, diccionarios y conjuntos. Estas estructuras son fundamentales para el procesamiento de datos en inteligencia artificial. Ahora es el momento de aplicar estos conceptos resolviendo problemas que simulan situaciones reales en el desarrollo de sistemas inteligentes.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código en Python completo y funcional, incluyendo capturas de pantalla de la ejecución de los programas, mostrando diferentes casos de prueba para verificar que funcionan correctamente.

Recomendaciones

Al resolver estos ejercicios, no se concentre únicamente en que el programa funcione. Preste atención también a la claridad del código, a usar nombres de variables descriptivos y a elegir la estructura de datos más adecuada para cada problema. Un/a buen/a programador/a no solo escribe códigos que funcionan, sino códigos eficientes y elegantes.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos qué estructura de datos es más apropiada para cada problema y ayudarse a detectar posibles mejoras en el código Python.

Ejercicio 1: Preprocesamiento de un dataset

Tenemos un dataset con mediciones que contiene algunos valores duplicados y necesita limpieza. Escriba un programa que cree una lista con los siguientes valores: [23, 45, 12, 23, 56, 89, 12, 45, 67, 23, 89, 34] y luego:

- Elimine los valores duplicados usando conjuntos.
- Ordene los valores de menor a mayor.
- Calcule la cantidad total de valores únicos, el valor mínimo, el valor máximo y el promedio.
- Usando comprensión de listas, cree una nueva lista con solo los valores mayores al promedio.

Ejercicio 2: Análisis de frecuencia de palabras

En procesamiento de lenguaje natural (NLP) es común analizar la frecuencia de palabras en un texto. Escriba un programa que reciba el siguiente texto: "inteligencia artificial machine learning deep learning artificial neural networks machine learning" y luego:

- Convierta el texto a minúsculas y divida en palabras usando split().
- Use un diccionario para contar cuántas veces aparece cada palabra.
- Muestre las palabras ordenadas de mayor a menor frecuencia.
- Identifique cuál es la palabra más frecuente.

Ejercicio 3: Gestión de estudiantes

Escriba un programa que gestione información de estudiantes usando diccionarios. El programa debe crear una lista con al menos 4 estudiantes, donde cada estudiante es un diccionario con las claves: "nombre", "edad", "carrera" y "promedio", y luego:

- Mostrar todos los estudiantes.
- Filtrar y mostrar solo los estudiantes con promedio mayor o igual a 8.0.
- Calcular el promedio general de todos los estudiantes.
- Encontrar al estudiante con el promedio más alto y mostrar su información completa.

Ejercicio 4: Operaciones con conjuntos de características

Escriba un programa que defina tres conjuntos de características observadas en diferentes muestras: muestra_a = {"edad", "altura", "peso", "presion"} muestra_b = {"edad", "peso", "temperatura", "frecuencia_cardiaca"} muestra_c = {"altura", "peso", "edad", "glucosa"} y luego encuentre e imprima:

- Características comunes a todas las muestras (intersección de los tres).
- Características únicas de muestra_a (que no están en b ni en c).
- Todas las características diferentes observadas (unión de todas).
- Cantidad total de características únicas.
- Para cada resultado, incluya un mensaje descriptivo explicando qué representa.

Ejercicio 5: Normalización de datos

La normalización es un paso crucial en el preprocesamiento de datos para IA. Escriba un programa que cree una lista con estos valores: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100] y luego:

- Calcule el valor mínimo y máximo de la lista.
- Usando comprensión de listas, normalice todos los valores aplicando la fórmula: valor_normalizado = (valor–minimo) / (maximo–minimo)
- Muestre la lista original y la lista normalizada (los valores deberían estar entre 0 y 1).
- Verifique que el primer valor normalizado sea 0 y el último sea 1.

Ejercicio 6: Agrupación por categorías

Escriba un programa que organice datos por categorías usando diccionarios. El programa debe crear una lista de tuplas con información de productos: productos = [("Laptop", "Electrónica", 850000), ("Mouse", "Electrónica", 5000), ("Escritorio", "Muebles", 120000), ("Silla", "Muebles", 45000), ("Teclado", "Electrónica", 15000), ("Estante", "Muebles", 65000)], y luego:

- Crear un diccionario donde las claves sean las categorías y los valores sean listas de nombres de productos de esa categoría.

- Para cada categoría, calcular y mostrar el precio promedio de sus productos.
- Mostrar qué categoría tiene el precio promedio más alto.

Ejercicio 7: Análisis de datos de sensores

Escriba un programa que procese lecturas de sensores. El programa debe crear una lista con lecturas de temperatura: [22.5, 23.1, 22.8, 25.3, 26.7, 24.2, 23.5, 22.9, 24.1, 25.8] y luego:

- Filtrar usando comprensión de listas las temperaturas que están fuera del rango normal (menores a 23.0 o mayores a 25.0).
- Contar cuántas lecturas anormales hay.
- Calcular el promedio de las temperaturas normales.
- Crear un diccionario con las estadísticas: "total_lecturas", "lecturas_normales", "lecturas_anormales", "promedio_normales".

Ejercicio 8: Sistema de etiquetas

En machine learning usamos etiquetas para clasificar datos. Escriba un programa que tenga una lista de instancias donde cada instancia es un diccionario con "id", "valor" y "etiqueta":

datos = [{"id": 1, "valor": 10, "etiqueta": "bajo"}, {"id": 2, "valor": 50, "etiqueta": "medio"}, {"id": 3, "valor": 15, "etiqueta": "bajo"}, {"id": 4, "valor": 80, "etiqueta": "alto"}, {"id": 5, "valor": 55, "etiqueta": "medio"}, {"id": 6, "valor": 90, "etiqueta": "alto"}]

Y luego:

- Agrupe las instancias por etiqueta en un diccionario.
- Para cada etiqueta, muestre cuántas instancias hay y el valor promedio.
- Cree un conjunto con todas las etiquetas únicas encontradas.

ACTIVIDAD 2
Implementando pilas y colas

Hasta aquí hemos aprendido sobre estructuras de datos especializadas: pilas (LIFO) y colas (FIFO). Estas estructuras son fundamentales en algoritmos de búsqueda, procesamiento de tareas y muchos otros problemas de inteligencia artificial. En esta actividad se proponen distintos ejercicios para implementar estas estructuras y aplicarlas en situaciones concretas.

Para cada uno de ellos deberá entregar un documento Word o PDF con el código en Python completo y funcional, incluyendo capturas de pantalla de la ejecución de los programas, mostrando diferentes casos de prueba para verificar que funcionan correctamente.

Recomendaciones

Al resolver estos ejercicios, asegúrese de implementar correctamente las operaciones de cada estructura. Las pilas deben respetar el orden LIFO (último en entrar, primero en salir) y las colas el orden FIFO (primero en entrar, primero en salir). Preste atención a los

casos especiales como intentar sacar elementos de estructuras vacías.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos la lógica de cada implementación y verificar que las estructuras se comporten correctamente en diferentes escenarios.

Ejercicio 1: Validador de expresiones

Escriba un programa que use una pila para verificar si una expresión matemática tiene paréntesis, corchetes y llaves balanceados. El programa debe implementar las funciones para manejar pilas vistas en el contenido (crear_pila, push, pop, esta_vacia, ver_tope). Luego crear una función validar_expresion(expresion) que reciba una cadena de texto y verifique:

- Por cada '(', '[', '{' encontrado, agregarlo a la pila
- Por cada ')', ']', '}' encontrado, verificar que coincida con el tope de la pila
- Al final, la pila debe estar vacía

Probar con estas expresiones:

- "{[()]}" (debe ser válida)
- "{[(])}" (debe ser válida)
- "{[()]]" (debe ser inválida)
- "((a + b) * (c–d))" (debe ser válida)

Ejercicio 2: Simulador de historial de navegación mejorado

Escriba un programa que simule el historial de un navegador web usando dos pilas. El programa debe usar una pila para el historial hacia atrás (páginas visitadas) y otra pila para el historial hacia adelante (páginas cuando volvemos atrás).

Implementar las siguientes funciones:

- visitar(url): visita una nueva página
- volver(): regresa a la página anterior
- avanzar(): avanza a la página siguiente (si hemos retrocedido)
- mostrar_actual(): muestra la página actual

Simular la siguiente navegación:

- Visitar "google.com".
- Visitar "wikipedia.org".
- Visitar "github.com".
- Volver dos veces.
- Avanzar una vez.
- Visitar "stackoverflow.com".
- Intentar avanzar (debería indicar que no se puede).

Ejercicio 3: Lista de reproducción de canciones

Escriba un programa que simule una lista de reproducción de música usando una cola. El programa debe implementar las funciones para manejar colas vistas en el contenido (crear_cola, enqueue, dequeue, esta_vacia, ver_frente). Crear una función reproducir_canciones() que:

- Agregue 5 canciones a la cola de reproducción (pueden ser nombres inventados, por ejemplo, pueden ser: "Canción A", "Canción B", "Canción C", "Canción D", "Canción E").
- Reproduzca las canciones una por una (dequeue).
- Muestre qué canción se está reproduciendo.
- Muestre cuántas canciones quedan por reproducir después de cada una.

Ejercicio 4: Fila de emergencias en un hospital

Escriba un programa que simule una sala de emergencias donde hay dos filas de pacientes: urgente y no urgente. El programa debe tener una cola para pacientes urgentes y otra para pacientes no urgentes. Implementar una función agregar_paciente(nombre, urgencia) que agregue el paciente a la cola correspondiente (urgencia puede ser "urgente" o "no urgente").

Implementar una función atender_siguiente() que:

- Primero intente atender de la cola urgente.
- Si está vacía, atienda de la cola no urgente.
- Si ambas están vacías, indique que no hay pacientes esperando.

Agregar varios pacientes mezclados (urgentes y no urgentes) y atenderlos todos, mostrando el orden en que son atendidos.

Ejercicio 5: Inversión de elementos

Escriba un programa que use una pila para invertir elementos. El programa debe crear una función invertir_lista(lista) que use una pila para invertir el orden de los elementos:

- Agregar todos los elementos a la pila.
- Sacar los elementos y agregarlos a una nueva lista.

Pruebe con la lista: [1, 2, 3, 4, 5]

Luego, implemente una función invertir_cadena(texto) que aplique el mismo criterio para invertir una cadena de texto, utilizando una pila para almacenar y recuperar los caracteres. Pruebe la función con la siguiente cadena:"Inteligencia Artificial"

Ejercicio 6: Promedio de las últimas calificaciones

Escriba un programa que calcule el promedio de las últimas 5 calificaciones de un estu-

diante usando una cola con capacidad limitada. El programa debe implementar una cola que solo pueda tener como máximo 5 elementos.

Cuando la cola está llena y llega una nueva calificación:

- Saque la calificación más antigua (dequeue).
- Agregue la nueva calificación (enqueue).

Crear una función agregar_calificacion(nota) que:

- Agregue la nota a la cola (si está llena, saque la más antigua primero).
- Calcule el promedio de las notas actuales en la cola.
- Muestre cuántas notas tiene guardadas y el promedio actual.

Simule agregar estas calificaciones una por una: [6, 7, 8, 9, 10, 8, 9, 7, 10] y muestre cómo va cambiando el promedio a medida que se agregan notas.

Ejercicio 7: Evaluación de expresiones postfijas

La notación postfija, también conocida como **notación polaca inversa**, se utiliza para evaluar expresiones matemáticas colocando los operadores después de los operandos. Por ejemplo, la expresión "3 4 +" representa la operación "3 + 4".

Escriba un programa que utilice una **pila** para evaluar expresiones escritas en notación postfija.

El programa debe recibir la expresión como una lista de cadenas, por ejemplo: ["5", "3", "+", "2", "*"]

Considere el siguiente procedimiento para procesar cada elemento de la lista:

- Si el elemento es un número, agréguelo a la pila.
- Si el elemento es un operador (+, -, *, /), extraiga dos números de la pila, realice la operación correspondiente y agregue el resultado nuevamente a la pila.

Al finalizar el procesamiento de la expresión, el valor que quede en la pila corresponde al resultado final.

Pruebe el programa con la expresión: ["5", "3", "+", "2", "*"]. El resultado esperado es 16, ya que la expresión equivale a (5 + 3) * 2.

Luego, pruebe también con la siguiente expresión: ["15", "7", "+", "3", "/"]. El resultado esperado es aproximadamente 7.33.

Ejercicio 8: Simulación de atención al cliente

Desarrolle un programa que simule un sistema de atención con múltiples ventanillas. El sistema debe contar con una cola de clientes en espera, identificados por nombre o

número de ticket, y con tres ventanillas de atención, que pueden representarse mediante variables.

Además, el programa debe implementar el siguiente proceso:

- Si una ventanilla está libre y hay clientes en cola, se asigna a esa ventanilla el siguiente cliente en espera.
- La atención de cada cliente debe simular una duración determinada, por ejemplo, mediante un contador.
- Una vez finalizada la atención, la ventanilla queda nuevamente disponible para atender a otro cliente.

Agregue inicialmente 10 clientes a la cola y simule el funcionamiento del sistema, mostrando en cada paso:

- Qué cliente está siendo atendido en cada ventanilla.
- Cuántos clientes quedan en espera.
- Qué ventanilla queda libre.

Módulo 3

Microobjetivos

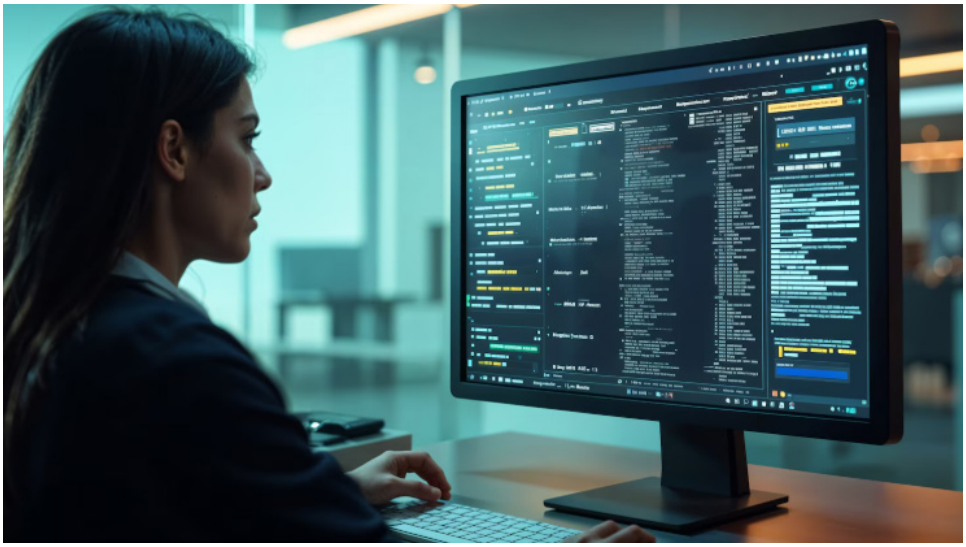
› Comprender los fundamentos de la programación orientada a objetos, para aprender a modelar problemas complejos de forma organizada y crear código que represente entidades del mundo real con sus características y comportamientos.

› Aplicar las características avanzadas de POO, para proteger datos sensibles, reutilizar código eficientemente mediante jerarquías de clases, y diseñar sistemas flexibles donde diferentes tipos de objetos puedan trabajarse de manera uniforme.

› Implementar estructuras de datos avanzadas usando programación orientada a objetos, para resolver problemas de clasificación, búsqueda de caminos, análisis de relaciones y toma de decisiones que son fundamentales en aplicaciones de inteligencia artificial.

Contenidos

MÓDULO 3: PROGRAMACIÓN ORIENTADA A OBJETOS



Fuente: Freepik

Llegamos al tercer módulo de Programación para Inteligencia Artificial, un momento crucial en nuestro aprendizaje. Hasta ahora hemos construido bases sólidas: aprendimos Python desde cero, trabajamos con variables, funciones, estructuras condicionales y bucles. En el segundo módulo profundizamos en estructuras de datos fundamentales como listas, tuplas, diccionarios, conjuntos, pilas y colas. Ahora nos adentramos en un paradigma de programación que cambiará completamente la forma en que organizamos y pensamos nuestro código: la programación orientada a objetos.

La programación orientada a objetos, conocida como POO, no es simplemente una técnica más. Es una manera diferente de modelar el mundo y los problemas que queremos

resolver. Hasta ahora escribimos código de forma procedural: una secuencia de instrucciones que se ejecutan una tras otra, funciones que procesan datos. Este enfoque funciona perfectamente para programas pequeños, pero cuando los sistemas crecen en complejidad, mantener y organizar el código se vuelve cada vez más difícil.

La POO nos permite pensar en términos de objetos que representan entidades del mundo real. Cada objeto tiene características llamadas atributos y comportamientos llamados métodos. Por ejemplo, un estudiante tiene nombre, edad y promedio como atributos, y puede inscribirse a materias o calcular su promedio como comportamientos; al igual que un vehículo tiene marca, modelo y velocidad como atributos, y puede acelerar o frenar como comportamientos. Esta forma de pensar es natural, intuitiva y poderosa.

En este módulo aprenderemos los conceptos fundamentales de la POO: qué son las clases y los objetos, cómo definir atributos y métodos y cómo crear instancias. Luego exploraremos las tres características fundamentales del paradigma: el encapsulamiento, que protege los datos internos de un objeto; la herencia, que permite crear nuevas clases a partir de otras existentes reutilizando código; y el polimorfismo, que permite que objetos de distintos tipos respondan a las mismas operaciones de manera diferente.

Una vez que dominemos estos conceptos, los aplicaremos para implementar estructuras de datos más complejas. Veremos árboles binarios, estructuras jerárquicas fundamentales para la clasificación y búsqueda eficiente en inteligencia artificial. También trabajaremos con grafos, estructuras que modelan relaciones complejas y son la base de sistemas de recomendación, análisis de redes sociales y algoritmos de navegación.

Este módulo es especialmente importante porque la programación orientada a objetos es el paradigma dominante en el desarrollo de sistemas de inteligencia artificial. Las bibliotecas más usadas están construidas usando POO. Cada concepto que aprendamos aquí será una herramienta que usaremos constantemente en el resto de la carrera.

¡Comencemos este viaje hacia la programación orientada a objetos!

¿Qué es la programación orientada a objetos?

Hasta ahora programamos de manera procedural. Escribimos funciones que reciben datos, los procesan y devuelven resultados. Los datos y las operaciones están separados. Esta separación funciona bien para programas pequeños, pero a medida que el código crece, mantener esa separación se vuelve problemático. Necesitamos recordar qué funciones trabajan con qué datos, y cualquier cambio en la estructura de datos puede requerir modificar múltiples funciones dispersas por todo el código.

La programación orientada a objetos propone algo diferente: agrupar los datos y las operaciones que trabajan sobre esos datos en una única entidad llamada objeto. Un objeto es, entonces, una unidad que contiene tanto información como comportamiento.



La programación orientada a objetos organiza el código en torno a objetos, que integran datos (atributos) y funciones (métodos) encargadas de operar sobre esos datos. Estos objetos se crean a partir de clases, que actúan como plantillas o modelos.

Pensemos en un ejemplo para comprenderlo mejor. Imaginemos que estamos desarrollando un sistema para gestionar estudiantes en una universidad. En programación procedural, tendríamos variables separadas para almacenar nombres, edades y promedios, y funciones separadas para calcular promedios, inscribir materias y generar reportes. Si necesitamos agregar un nuevo dato, como el número de materias aprobadas, debemos modificar múltiples funciones y recordar actualizar todas las partes del código que manejan estudiantes.

Con POO, creamos una **clase** Estudiante que encapsula toda la información y comportamiento relacionado con un estudiante. La clase define qué datos tiene un estudiante y qué puede hacer. Cada estudiante individual es un **objeto** creado a partir de esa clase, con sus propios valores específicos, pero compartiendo la misma estructura y comportamiento.

Esta forma de organizar el código tiene ventajas enormes. Primero, el código es más fácil de entender porque agrupa conceptos relacionados. Segundo, es más fácil de mantener porque los cambios se localizan en un solo lugar. Tercero, podemos reutilizar clases en diferentes partes del programa. Y cuarto, podemos crear jerarquías de clases que comparten código común, evitando duplicación.

Clases y objetos: definición, atributos y métodos

Comencemos con los conceptos básicos: clases y objetos. Estos dos términos están íntimamente relacionados, pero representan cosas diferentes.



Una clase es una plantilla o molde que define la estructura y el comportamiento de un tipo de objeto. Un objeto es una instancia concreta de una clase, con valores específicos para sus atributos.

La analogía más común es pensar en una clase como el plano de una casa. El plano define las habitaciones, sus tamaños, dónde van las puertas y ventanas. Pero el plano no es

una casa real, es solo la descripción. Una casa construida siguiendo ese plano sí es real, tangible, tiene habitantes y muebles específicos. La clase es el plano, el objeto es la casa construida. Podemos construir múltiples casas usando el mismo plano, y cada casa será diferente en sus detalles específicos, pero todas compartirán la misma estructura básica definida en el plano. Del mismo modo, podemos crear múltiples objetos a partir de una misma clase. Si tenemos una clase Estudiante, podemos crear objetos para Ana, Carlos y María. Cada uno es un objeto diferente con sus propios datos, pero todos comparten la estructura y comportamiento definidos en la clase Estudiante.

Definiendo nuestra primera clase

Veamos cómo se define una clase en Python. Usaremos un ejemplo simple para empezar: una clase que representa un perro.

```
class Perro:
        def __init__(self, nombre, raza, edad):
                self.nombre = nombre
                self.raza = raza
                self.edad = edad
def ladrar(self):
    return f"{self.nombre} dice: ¡Guau guau!"

def cumplir_años(self):
    self.edad += 1
    return f"{self.nombre} ahora tiene {self.edad} años"
def describir(self):
 return f"{self.nombre} es un {self.raza} de {self.edad} años"
```

Analicemos este código detalladamente. La palabra clave **class** indica que estamos definiendo una clase. Le damos un nombre, en este caso Perro, siguiendo la convención de usar mayúscula inicial.

El método __init__ es especial y se llama **constructor**. Se ejecuta automáticamente cuando creamos un nuevo objeto. El nombre __init__ debe escribirse exactamente así, con dos guiones bajos antes y después. Este método inicializa el objeto con los valores que le pasamos.

El primer parámetro self es una referencia al objeto que se está creando. Python lo pasa automáticamente, no necesitamos proporcionarlo cuando creamos el objeto. self representa al objeto mismo desde su propia perspectiva. Los siguientes parámetros son nombre, raza y edad, los valores que proporcionaremos al crear el objeto.

Dentro del método, las líneas como self.nombre = nombre crean **atributos** del objeto y les asignan los valores recibidos. self.nombre es un atributo del objeto, mientras que nombre es el parámetro recibido.

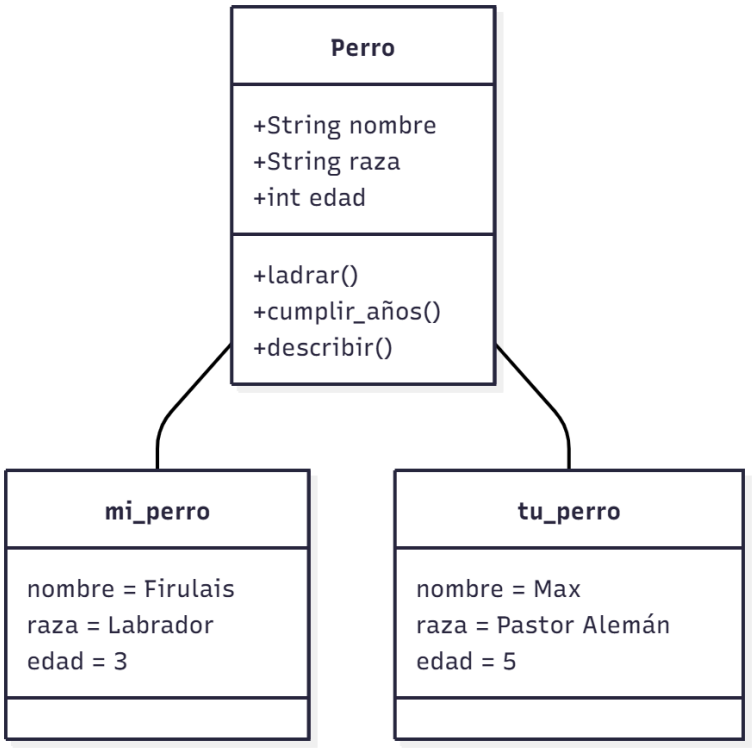
Los otros **métodos** definen comportamientos. El método ladrar() usa el atributo nombre para devolver un mensaje personalizado. El método cumplir_años() modifica el atributo edad, incrementándolo en uno. El método describir() usa todos los atributos para generar una descripción completa. Observemos que todos tienen self como primer parámetro porque necesitan acceder a los atributos del objeto.

Creemos y usemos objetos:

```
mi_perro = Perro("Firulais", "Labrador", 3)
tu_perro = Perro("Max", "Pastor Alemán", 5)
print(f"Mi perro se llama {mi_perro.nombre} y tiene {mi_perro.edad} años")
print(mi_perro.describir())
print(mi_perro.ladrar())
print(mi_perro.cumplir_años())
print(mi_perro.describir())
```

Cuando escribimos Perro("Firulais", "Labrador", 3), Python automáticamente llama al método __init__ pasando esos valores. Se crea un objeto Perro con esos datos específicos. Accedemos a los atributos usando la sintaxis objeto.atributo y llamamos a los métodos con objeto.metodo().

Cada objeto tiene sus propios valores independientes. Cambiar mi_perro.edad no afecta a tu_perro.edad. Son entidades separadas en memoria. Los objetos mantienen su estado entre llamadas a métodos, como vemos cuando cumplir_años() modifica permanentemente la edad.



Fuente: Elaboración propia.

Creemos una clase más completa para gestionar información de estudiantes. Esta clase demuestra cómo los objetos pueden mantener colecciones de datos y realizar operaciones más complejas:

```
class Estudiante:
    def __init__(self, nombre, edad, carrera):
        self.nombre = nombre
        self.edad = edad
        self.carrera = carrera
        self.materias = []
        self.notas = {}

def inscribir_materia(self, materia):
    if materia not in self.materias:
        self.materias.append(materia)
        return f"{self.nombre} se inscribió en {materia}"
    return f"{self.nombre} ya está inscripto en {materia}"

def registrar_nota(self, materia, nota):
    if materia in self.materias:
        self.notas[materia] = nota
        return f"Nota {nota} registrada en {materia} para {self.nombre}"
    return f"{self.nombre} no está inscripto en {materia}"

def calcular_promedio(self):
    if len(self.notas) == 0:
        return 0
    return sum(self.notas.values()) / len(self.notas)

def mostrar_informacion(self):
    info = f"\n=== Información de {self.nombre} ==="
    info += f"\nEdad: {self.edad}"
    info += f"\nCarrera: {self.carrera}"
    info += f"\nMaterias cursadas: {len(self.materias)}"
    info += f"\nPromedio: {self.calcular_promedio():.2f}"
    return info
```

Esta clase es más sofisticada que el ejemplo del perro. No solo inicializa atributos simples, sino que además crea una lista vacía para las materias y un diccionario vacío para las notas. Además, sus métodos incorporan validaciones previas antes de modificar los datos, lo que permite asegurar la consistencia de la información.

Usemos esta clase:

```
estudiante1 = Estudiante("Ana García", 20, "Lic. en Inteligencia Artificial")

print(estudiante1.inscribir_materia("Programación para IA"))
print(estudiante1.inscribir_materia("Matemática"))
print(estudiante1.inscribir_materia("Estadística"))
print(estudiante1.registrar_nota("Programación para IA", 9))
print(estudiante1.registrar_nota("Matemática", 8))
print(estudiante1.registrar_nota("Estadística", 10))
print(estudiante1.mostrar_informacion())

estudiante2 = Estudiante("Carlos López", 22, "Lic. en Ciencia de Datos")

print(estudiante2.inscribir_materia("Python Avanzado"))
print(estudiante2.registrar_nota("Python Avanzado", 9.5)
print(estudiante2.mostrar_informacion())
```

Observemos cómo cada objeto estudiante mantiene su propia lista de materias y diccionario de notas, son completamente independientes. Esta independencia es una de las grandes ventajas de la POO.



En este momento, está en condiciones de realizar la **actividad 1** del módulo.

Encapsulamiento: protegiendo los datos internos

Ahora que entendemos qué son las clases y los objetos, exploremos la primera característica fundamental de la programación orientada a objetos: el **encapsulamiento**.



El encapsulamiento es el principio de ocultar los detalles internos de implementación de un objeto y exponer solo lo necesario a través de una interfaz pública. Los datos internos se protegen del acceso directo desde fuera del objeto.

Por ejemplo, pensemos en un automóvil. Cuando conducimos, interactuamos con una interfaz simple: volante, pedales, palanca de cambios. No necesitamos entender cómo funciona el motor internamente, cómo se inyecta el combustible o cómo trabaja la transmisión. Esos detalles están encapsulados dentro del motor. Solo necesitamos saber que, al pisar el acelerador, el auto avanza. Si el fabricante cambia el motor por uno más eficiente, mientras la interfaz (pedales y volante) siga funcionando igual, no necesitamos aprender nada nuevo.

En este sentido el encapsulamiento tiene varios beneficios importantes. Primero, simplifica el uso del objeto porque no necesitamos conocer sus detalles internos. Segundo, protege la integridad de los datos porque el objeto puede validar cualquier cambio antes de aplicarlo. Tercero, permite cambiar la implementación interna sin afectar el código que usa el objeto.

En Python, el encapsulamiento se implementa usando convenciones de nombres. Los atributos y métodos que comienzan con un guión bajo se consideran internos y no deberían accederse directamente desde fuera de la clase:

```
class CuentaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular
        self._saldo = saldo_inicial
        self._movimientos = []

    def depositar(self, monto):
        if monto > 0:
            self._saldo += monto
            self._movimientos.append(f"Depósito: +{monto}")
            return f"Depósito exitoso. Nuevo saldo: {self._saldo}"
        return "El monto debe ser positivo"

    def retirar(self, monto):
        if monto > 0 and monto <= self._saldo:
            self._saldo -= monto
            self._movimientos.append(f"Retiro: -{monto}")
            return f"Retiro exitoso. Nuevo saldo: {self._saldo}"
        return "Fondos insuficientes o monto inválido"

    def obtener_saldo(self):
        return self._saldo

    def obtener_movimientos(self):
        return self._movimientos.copy()
```

En esta clase, _saldo y _movimientos son atributos internos marcados con guión bajo. Esto indica a otros programadores que no deberían modificarse directamente desde fuera de la clase. En su lugar, proporcionamos métodos públicos como depositar(), retirar() y obtener_saldo() que controlan cómo se accede y modifica el saldo.

```
cuenta = CuentaBancaria("Ana García", 1000)
print(cuenta.depositar(500))
print(cuenta.retirar(200))
print(f"Saldo actual: {cuenta.obtener_saldo()}")
```

Aunque Python técnicamente permite acceder a cuenta._saldo directamente, el guión bajo es una señal clara de que no deberíamos hacerlo. Los métodos públicos validan las operaciones: no podemos depositar montos negativos ni retirar más de lo que tenemos. Si accediéramos directamente al _saldo, podríamos romper estas reglas y dejar la cuenta en un estado inconsistente.

Niveles de acceso: público, protegido y privado

En Python existen tres niveles de acceso para los atributos y métodos de una clase, definidos por convenciones de nombres:

Los atributos y métodos **públicos** no tienen ningún guión bajo al inicio de su nombre. Son la interfaz pública de la clase y están diseñados para ser usados libremente desde fuera del objeto. Por ejemplo: titular, depositar() y obtener_saldo() son públicos en nuestra clase CuentaBancaria. Cualquier código puede acceder a ellos sin restricciones.

Los atributos y métodos **protegidos** comienzan con un guión bajo simple. La convención indica que estos elementos son internos de la clase y no deberían accederse directamente desde fuera, aunque Python técnicamente lo permite. Por ejemplo, _saldo y _movimientos son protegidos. Son para uso interno de la clase y sus subclases. Si una clase hija hereda de CuentaBancaria, entonces podrá acceder a estos atributos protegidos.

Por otra parte, los atributos y métodos **privados** comienzan con doble guión bajo. Python aplica un mecanismo llamado "name mangling" que hace más difícil (aunque no imposible) acceder a estos desde fuera de la clase. Por ejemplo:

```
class SistemaSeguro:
def __init__(self, usuario, clave):
self.usuario = usuario
self.__clave = clave
def verificar_acceso(self, clave_ingresada):
    return self.__clave == clave_ingresada

def cambiar_clave(self, clave_actual, clave_nueva):
    if self.__clave == clave_actual:
        self.__clave = clave_nueva
        return "Clave cambiada exitosamente"
    return "Clave actual incorrecta"

sistema = SistemaSeguro("ana", "secreto123") print(sistema.usuario)
print(sistema.verificar_acceso("secreto123"))
```

En este ejemplo, usuario es público y podemos accederlo libremente. Pero __clave es privado y solo debe modificarse a través de los métodos proporcionados como cambiar_clave(). Intentar acceder a sistema.__clave directamente generaría un error, protegiendo así la información sensible.

La elección entre público, protegido y privado depende de qué tan estricto queremos ser con el acceso. En general, usamos públicos para la interfaz que queremos que otros usen, protegidos para detalles internos que las subclases podrían necesitar y privados para información realmente sensible que nadie más debería tocar.

Herencia: reutilizando código

La segunda característica fundamental de la POO es la **herencia**. La herencia permite crear nuevas clases basadas en clases existentes, reutilizando y extendiendo su funcionalidad.



La herencia permite que una clase (clase hija o subclase) herede atributos y métodos de otra clase (clase padre o superclase). La clase hija puede agregar nuevos atributos y métodos, o modificar los heredados.

Para comprender mejor esta característica, pensemos en una taxonomía biológica. Todos los mamíferos comparten características comunes: tienen pelo, son de sangre caliente, amamantan a sus crías. Luego, dentro de los mamíferos, los perros tienen características adicionales específicas y los gatos tienen otras. No necesitamos redefinir las características comunes de mamíferos para cada especie, simplemente decimos que los perros son mamíferos y agregan características específicas. La herencia funciona igual en programación. Si tenemos una clase que representa características y comportamientos comunes, podemos crear clases más específicas que heredan todo eso y agregan sus propias características:

```
class Animal:
        def __init__(self, nombre, edad):
                self.nombre = nombre
                self.edad = edad
        def hacer_sonido(self):
            return “El animal hace un sonido”

        def describir(self):
            return f”{self.nombre} tiene {self.edad} años”

class Perro(Animal):
        def __init__(self, nombre, edad, raza):
        super().__init__(nombre, edad)
                self.raza = raza
        def hacer_sonido(self):
            return f”{self.nombre} dice: ¡Guau guau!”

        def describir(self):
 return f”{super().describir()} y es un {self.raza}”

class Gato(Animal):
        def __init__(self, nombre, edad, color):
            super().__init__(nombre, edad)
            self.color = color

def hacer_sonido(self):
     return f”{self.nombre} dice: ¡Miau!”

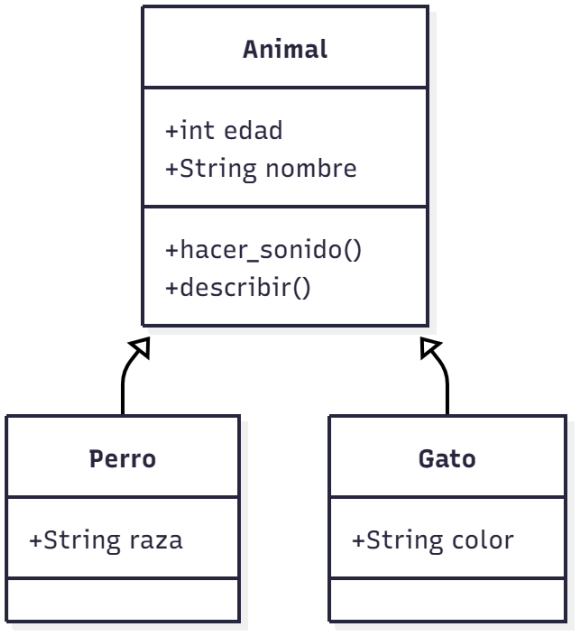
def describir(self):
    return f”{super().describir()} y es de color {self.color}”
```

La clase Animal es la clase padre que define atributos y métodos comunes a todos los animales. Las clases Perro y Gato son clases hijas que heredan de Animal, indicado por class Perro(Animal). En el __init__ de las clases hijas, usamos super().__init__(nombre, edad) para llamar al constructor de la clase padre. Esto inicializa los atributos heredados. Luego agregamos atributos específicos como raza para Perro y color para Gato.

Los métodos hacer_sonido() y describir() están definidos tanto en la clase padre como en las hijas. Cuando una clase hija define un método con el mismo nombre que en la clase padre, se dice que lo sobrescribe. Cuando creamos un objeto Perro y llamamos a hacer_sonido(), se ejecuta la versión de Perro, no la de Animal.

```
animal_generico = Animal("Criatura", 5)
print(animal_generico.describir())
print(animal_generico.hacer_sonido())
mi_perro = Perro("Firulais", 3, "Labrador")
print(mi_perro.describir())
print(mi_perro.hacer_sonido())
mi_gato = Gato("Michi", 2, "naranja")
print(mi_gato.describir())
print(mi_gato.hacer_sonido())
```

Observemos cómo las clases hijas tienen acceso a los atributos y métodos de la clase padre, pero también pueden agregar los suyos propios o modificar los heredados:



Fuente: Elaboración propia.

Veamos un ejemplo más completo que demuestra el poder de la herencia en un contexto del mundo real:

```
class Vehiculo:
        def __init__(self, marca, modelo, año):
                self.marca = marca
                self.modelo = modelo
                self.año = año
                self.velocidad_actual = 0

        def acelerar(self, incremento):
            self.velocidad_actual += incremento
            return f"Acelerando. Velocidad: {self.velocidad_actual} km/h"

        def frenar(self, decremento):
            self.velocidad_actual = max(0, self.velocidad_actual–decremento)
            return f"Frenando. Velocidad: {self.velocidad_actual} km/h"

        def obtener_info(self):
            return f"{self.marca} {self.modelo} ({self.año})"

class Auto(Vehiculo):
        def __init__(self, marca, modelo, año, puertas):
                super().__init__(marca, modelo, año)
                self.puertas = puertas
        def obtener_info(self):
            return f"{super().obtener_info()}–Auto de {self.puertas} puertas"

class Camion(Vehiculo):
        def __init__(self, marca, modelo, año, capacidad_carga):
                super().__init__(marca, modelo, año)
                self.capacidad_carga = capacidad_carga
                self.esta_cargado = False
        def cargar(self):
                    self.esta_cargado = True
                    return "Camión cargado"

          def descargar(self):
                    self.esta_cargado = False
                    return "Camión descargado"

          def obtener_info(self):
                    estado = "cargado" if self.esta_cargado else "vacío"
                    return f"{super().obtener_info()}–Camión {estado}, capacidad:
```

```
{self.capacidad_carga}kg"

mi_auto = Auto("Toyota", "Corolla", 2020, 4)
print(mi_auto.obtener_info())
print(mi_auto.acelerar(60))
print(mi_auto.frenar(20))
mi_camion = Camion("Mercedes", "Actros", 2019, 15000)
print(mi_camion.obtener_info())
print(mi_camion.cargar())
print(mi_camion.obtener_info())
```

Este ejemplo muestra cómo la herencia nos permite reutilizar el código común de Vehiculo mientras especializamos el comportamiento en cada tipo específico.

Polimorfismo: muchas formas

La tercera característica fundamental de la POO es el polimorfismo. El término viene del griego y significa "muchas formas".



El polimorfismo permite que objetos de diferentes clases respondan al mismo método de maneras diferentes. Podemos tratar objetos de distintos tipos de forma uniforme si comparten la misma interfaz.

Imaginemos un control remoto universal. El mismo botón "play" funciona con el televisor, el reproductor de DVD y el equipo de música, pero cada dispositivo responde de manera diferente. El televisor reproduce el canal actual, el DVD reproduce la película, el equipo reproduce música. El botón es el mismo, pero el comportamiento cambia según el dispositivo.

El polimorfismo funciona igual. Podemos llamar al mismo método en diferentes objetos y cada objeto responde de forma apropiada a su tipo:

```
def hacer_sonar_animales(animales):
    for animal in animales:
        print(animal.hacer_sonido())

mi_perro = Perro("Firulais", 3, "Labrador")
mi_gato = Gato("Michi", 2, "naranja")
un_animal = Animal("Desconocido", 1)
animales = [mi_perro, mi_gato, un_animal]
hacer_sonar_animales(animales)
```

La función hacer_sonar_animales() no necesita saber qué tipo específico de animal está procesando. Solo necesita saber que cada animal tiene un método hacer_sonido().

Creemos un sistema para calcular áreas de diferentes formas geométricas:

```
class Forma:
    def __init__(self, nombre):
        self.nombre = nombre

    def calcular_area(self):
        return 0

    def describir(self):
        return f"{self.nombre} con área {self.calcular_area():.2f}"

class Rectangulo(Forma):
    def __init__(self, base, altura):
        super().__init__("Rectángulo")
        self.base = base
        self.altura = altura
```

```
    def calcular_area(self):
        return self.base * self.altura

class Circulo(Forma):
    def __init__(self, radio):
        super().__init__("Círculo")
        self.radio = radio

    def calcular_area(self):
        return 3.14159 * (self.radio ** 2)

class Triangulo(Forma):
    def __init__(self, base, altura):
        super().__init__("Triángulo")
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return (self.base * self.altura) / 2

    def calcular_area_total(formas):
        total = 0
        for forma in formas:
            area = forma.calcular_area()
            print(forma.describir())
            total += area
        return total

formas = [ Rectangulo(10, 5), Circulo(7), Triangulo(8, 6), Rectangulo(15, 3) ]

print(f"\nÁrea total: {calcular_area_total(formas):.2f}")
```

La función calcular_area_total() puede procesar cualquier forma sin necesitar saber el tipo específico. Esto hace el código flexible y fácil de extender. Si queremos agregar un nuevo tipo de forma, simplemente creamos una nueva clase que herede de Forma e implemente calcular_area(), y funcionará automáticamente con el código existente.

Reutilización de código: organizando nuestras clases en módulos

Una de las ventajas más poderosas de la programación orientada a objetos es la capacidad de crear código reutilizable. Una vez que hemos definido una clase bien diseñada, podemos usarla en múltiples proyectos sin necesidad de reescribirla. Python nos permite organizar nuestro código en archivos separados llamados módulos y luego importar las clases que necesitemos.

Imaginemos que hemos creado nuestras clases de formas geométricas que vimos en la sección de polimorfismo y las hemos probado exhaustivamente. Ahora queremos usarlas en otro proyecto sin tener que copiar y pegar todo el código. La solución es guardar las clases en un archivo separado e importarlas cuando las necesitemos.

Supongamos que guardamos nuestras clases de formas en un archivo llamado geometria.py. Este archivo contendría todas nuestras definiciones de clases y en el archivo *geometria.py* ponemos:

```
class Forma:
        def init(self, nombre):
            self.nombre = nombre

        def calcular_area(self):
            return 0

        def describir(self):
            return f"{self.nombre} con área {self.calcular_area():.2f}"

class Rectangulo(Forma):
        def init(self, base, altura):
            super().init("Rectángulo")
            self.base = base
            self.altura = altura
        def calcular_area(self):
            return self.base * self.altura

class Circulo(Forma):
        def init(self, radio):
            super().init("Círculo")
            self.radio = radio
        def calcular_area(self):
            return 3.14159 * (self.radio ** 2)

class Triangulo(Forma):
        def init(self, base, altura):
            super().init("Triángulo")
            self.base = base
            self.altura = altura

def calcular_area(self):
            return (self.base * self.altura) / 2
```

Ahora, en otro archivo podemos importar y usar estas clases. Python nos ofrece varias formas de importar. Si solo necesitamos algunas clases específicas, podemos importarlas individualmente, separándolas por comas. En otro archivo *calcular_areas.py* ponemos:

```
from geometria import Rectangulo, Circulo

rectangulo = Rectangulo(10, 5)
circulo = Circulo(7)

print(rectangulo.describir())
print(circulo.describir())
```

Esta es la forma más recomendada porque hace explícito qué estamos usando del módulo. El código es más claro y evitamos conflictos de nombres. También podemos importar el módulo completo y acceder a sus clases usando el nombre del módulo como prefijo:

```
import geometria

rectangulo = geometria.Rectangulo(10, 5)
circulo = geometria.Circulo(7)
```

Esta forma es útil cuando queremos dejar claro de dónde viene cada clase, especialmente si importamos varios módulos. Si el nombre del módulo es largo, podemos darle un alias más corto:

```
import geometria as geo

rectangulo = geo.Rectangulo(12, 6)
area = rectangulo.calcular_area()
```

Ubicación de los archivos

Para que Python pueda encontrar nuestro módulo, el archivo debe estar en la misma carpeta que el archivo que lo está importando. Esta es la forma más simple y la que usaremos generalmente cuando estamos aprendiendo:

```
mi_proyecto/geometria.py

mi_proyecto/calcular_areas.py
```

Si el archivo está en otra carpeta, necesitamos especificar la ruta. Por ejemplo, si tenemos una carpeta llamada utilidades:

```
mi_proyecto/utilidades/geometria.py
```

Importaríamos así:

```
from utilidades.geometria import Rectangulo, Circulo
```

Organizando el código con *if __name__ == "__main__"*:

Por último, cuando creamos un módulo que será importado por otros programas, necesitamos distinguir entre el código que queremos que se ejecute cuando importamos el módulo y el código que solo debería ejecutarse cuando corremos el archivo directamente. Para esto usamos la construcción if __name__ == "__main__":.

Veamos un ejemplo completo en nuestro archivo geometria.py:

Archivo: geometria.py (versión mejorada)

```
class Forma:
    def __init__(self, nombre):
        self.nombre = nombre

    def calcular_area(self):
        return 0

    def describir(self):
        return f"{self.nombre} con área {self.calcular_area():.2f}"

class Rectangulo(Forma):
    def __init__(self, base, altura):
        super().__init__("Rectángulo")
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura

class Circulo(Forma):
    def __init__(self, radio):
        super().__init__("Círculo")
        self.radio = radio

    def calcular_area(self):
        return 3.14159 * (self.radio ** 2)

class Triangulo(Forma):
    def __init__(self, base, altura):
        super().__init__("Triángulo")
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return (self.base * self.altura) / 2

    def calcular_area_total(formas):
        total = 0
        for forma in formas:
            area = forma.calcular_area()
            print(forma.describir())
            total += area
        return total

if __name__ == "__main__":
    print("Probando el módulo geometria")
```

```
rect = Rectangulo(10, 5)
circ = Circulo(7)
trian = Triangulo(8, 6)

print(rect.describir())
print(circ.describir())
print(trian.describir())
print("Pruebas completadas exitosamente")

formas = [ Rectangulo(10, 5), Circulo(7), Triangulo(8, 6), Rectangulo(15, 3) ]

print(f"\nÁrea total: {calcular_area_total(formas):.2f}")
```

La línea if __name__ == "__main__" es una condición especial. Cuando ejecutamos directamente el archivo geometria.py, Python establece la variable __name__ con el valor "__main__", por lo que el código dentro del if se ejecuta. Pero cuando importamos este módulo desde otro archivo, __name__ tendrá otro valor y ese código no se ejecutará.

Esto es extremadamente útil porque nos permite:

- Incluir pruebas del módulo que se ejecutan solo cuando lo corremos directamente, pero no cuando lo importamos.
- Tener ejemplos de uso dentro del mismo archivo que documenten cómo usar las clases.
- Mantener el código limpio y profesional, separando la definición de las clases de su uso.

Ahora cuando creamos calcular_areas.py e importamos el módulo:

```
from geometria import Rectangulo, Circulo

rect = Rectangulo(15, 8) circ = Circulo(10)
print(f"Área del rectángulo: {rect.calcular_area()}")
print(f"Área del círculo: {circ.calcular_area()}")
```

Las clases están disponibles, pero el código de prueba dentro del if __name__ == "__main__": no se ejecuta. Sin embargo, si ejecutamos python geometria.py directamente, veremos las pruebas ejecutándose.

Esta organización del código es una práctica profesional estándar. A medida que nuestros proyectos crezcan, tendremos múltiples módulos con diferentes responsabilidades: uno para geometría, otro para estudiantes, otro para procesamiento de datos. La POO combinada con una buena organización en módulos nos permite construir sistemas complejos de manera ordenada y mantenible.

En las siguientes secciones, cuando implementemos árboles y grafos, podríamos guardar cada implementación en su propio módulo y luego importarlas según las necesitemos.

Esta es exactamente la forma en que trabajan las grandes bibliotecas de Python que usaremos más adelante en la carrera.



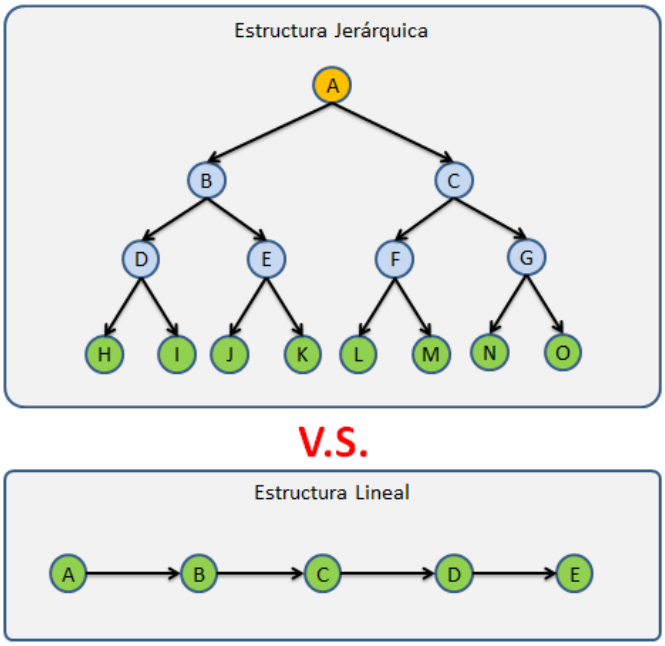
En este momento, está en condiciones de realizar la **actividad 2**.

Árboles binarios con programación orientada a objetos

Ahora que dominamos los conceptos de POO, apliquémoslos para implementar estructuras de datos más complejas. Comenzaremos con árboles binarios, estructuras jerárquicas fundamentales en inteligencia artificial.

Hasta ahora trabajamos con estructuras lineales: listas, pilas y colas organizan elementos en secuencia. Pero muchos problemas tienen una naturaleza jerárquica. Pensemos en un sistema de carpetas en una computadora, cada carpeta puede contener archivos y otras carpetas. O en un organigrama empresarial, cada puesto puede tener varios puestos subordinados. Los árboles nos permiten modelar estas relaciones de forma natural.

En inteligencia artificial, los árboles son fundamentales para la toma de decisiones. Cuando un sistema debe clasificar algo, puede usar un árbol donde cada decisión lleva a otras decisiones hasta llegar a una conclusión. También son la base de algoritmos de búsqueda eficientes y de estructuras que permiten organizar información de forma que sea rápido encontrarla.



Fuente: https://www.oscarblancarteblog.com/2014/08/22/estructura-de-datos-arboles/

Énfasis

Un árbol binario es una estructura jerárquica donde cada nodo tiene como máximo dos hijos. El nodo superior se llama raíz y los nodos sin hijos se llaman hojas.

Los árboles utilizan una terminología específica que es necesario conocer. La raíz es el nodo superior del árbol y representa el punto de entrada. Un padre es un nodo que tiene uno o más hijos, mientras que un hijo es un nodo que desciende directamente de otro. Una hoja es un nodo que no posee hijos y se encuentra en los extremos del árbol. La altura de un árbol corresponde a la mayor distancia entre la raíz y una hoja. El nivel, en cambio, indica la distancia de un nodo respecto de la raíz, considerando que la raíz se ubica en el nivel cero.

En la imagen, A es el nodo raíz y, al mismo tiempo, el nodo padre de B y C. El nodo B es padre de D y E, por lo que estos son nodos hijos. Los nodos H, I, J, K, ... corresponden a las hojas del árbol. La altura de este árbol es 4, ya que la mayor distancia desde la raíz hasta una hoja es de cuatro niveles.

Para implementar árboles binarios utilizaremos programación orientada a objetos. Definiremos dos clases: una que represente cada nodo del árbol y otra que modele la estructura completa del árbol, encargada de gestionar y organizar dichos nodos.

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izquierdo = None
        self.derecho = None
    def es_hoja(self):
        return self.izquierdo is None and self.derecho is None
    def __str__(self):
        return str(self.valor)
```

La clase Nodo encapsula un valor y referencias a los dos posibles hijos. Si un hijo no existe, su referencia es None. Los métodos adicionales nos ayudan a consultar el estado del nodo.

Ahora creemos la clase ArbolBinario que manejará el árbol completo:

```
class ArbolBinario:

    def __init__(self):
        self.raiz = None

    def esta_vacio(self):
        return self.raiz is None

    def insertar_raiz(self, valor):
        if self.raiz is None:
            self.raiz = Nodo(valor)
            return True
        return False

    def insertar_izquierdo(self, nodo, valor):
        if nodo.izquierdo is None:
            nodo.izquierdo = Nodo(valor)
            return nodo.izquierdo
        return None
```

```
def insertar_derecho(self, nodo, valor):
    if nodo.derecho is None:
        nodo.derecho = Nodo(valor)
        return nodo.derecho
    return None

def altura(self, nodo=None):
    if nodo is None:
        nodo = self.raiz
    if nodo is None:
        return 0
    if nodo.es_hoja():
        return 1
    altura_izq = self.altura(nodo.izquierdo)
    altura_der = self.altura(nodo.derecho)
    return 1 + max(altura_izq, altura_der)

def recorrido_inorden(self, nodo=None, resultado=None):
    if resultado is None:
        resultado = []
    if nodo is None:
        nodo = self.raiz
    if nodo is not None:
        self.recorrido_inorden(nodo.izquierdo, resultado)
        resultado.append(nodo.valor)
        self.recorrido_inorden(nodo.derecho, resultado)
    return resultado

def recorrido_preorden(self, nodo=None, resultado=None):
    if resultado is None:
        resultado = []
    if nodo is None:
        nodo = self.raiz
    if nodo is not None:
        resultado.append(nodo.valor)
        self.recorrido_preorden(nodo.izquierdo, resultado)
        self.recorrido_preorden(nodo.derecho, resultado)
    return resultado
```

Finalmente, creemos y usemos un árbol:

```
if __name__ == "__main__":
        arbol = ArbolBinario()
        arbol.insertar_raiz(10)
        izq = arbol.insertar_izquierdo(arbol.raiz, 5)
        der = arbol.insertar_derecho(arbol.raiz, 15)
        arbol.insertar_izquierdo(izq, 3)
        arbol.insertar_derecho(izq, 7)
        print(f"Altura del árbol: {arbol.altura()}")
        print(f"Recorrido inorden: {arbol.recorrido_inorden()}")
        print(f"Recorrido preorden: {arbol.recorrido_preorden()}")
```

Existen tres recorridos clásicos para un árbol binario. En el recorrido inorden, se visita primero el subárbol izquierdo, luego el nodo raíz y, por último, el subárbol derecho. El recorrido preorden comienza por la raíz, continúa con el subárbol izquierdo y finaliza con el subárbol derecho. En el recorrido postorden, se recorren primero ambos subárboles y se visita la raíz al final. Cada tipo de recorrido resulta adecuado para distintos objetivos y formas de procesamiento de la información.

Árbol de búsqueda binaria (BST)

Un árbol de búsqueda binaria es un árbol binario especial con una propiedad muy importante: para cada nodo, todos los valores en el subárbol izquierdo son menores que el valor del nodo, y todos los valores del subárbol derecho son mayores. Esta propiedad hace que los BST sean extremadamente eficientes para buscar valores.

```
class ArbolBusquedaBinaria:
    def __init__(self):
        self.raiz = None
    def insertar(self, valor):
        if self.raiz is None:
        self.raiz = Nodo(valor)
        else:
        self._insertar_recursivo(self.raiz, valor)

    def _insertar_recursivo(self, nodo, valor):
        if valor < nodo.valor:
        if nodo.izquierdo is None:
            nodo.izquierdo = Nodo(valor)
        else:
            self._insertar_recursivo(nodo.izquierdo, valor)
        else:
        if nodo.derecho is None:
            nodo.derecho = Nodo(valor)
        else:
            self._insertar_recursivo(nodo.derecho, valor)
```

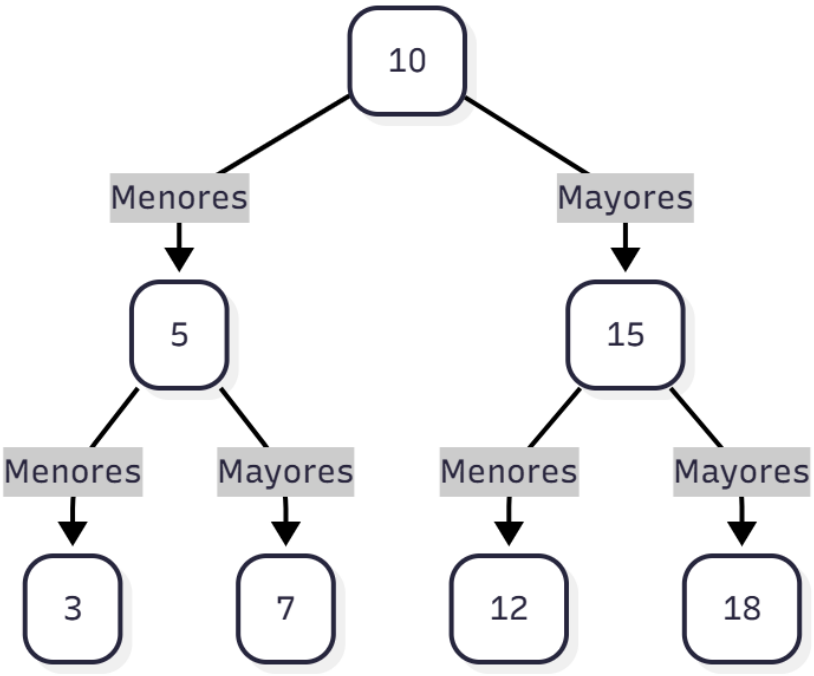
```
    def buscar(self, valor):
        return self._buscar_recursivo(self.raiz, valor)

    def _buscar_recursivo(self, nodo, valor):
        if nodo is None:
        return False
        if valor == nodo.valor:
        return True
        elif valor < nodo.valor:
        return self._buscar_recursivo(nodo.izquierdo, valor)
        else:
        return self._buscar_recursivo(nodo.derecho, valor)

    def recorrido_inorden(self, nodo=None, resultado=None):
        if resultado is None:
        resultado = []
        if nodo is None:
        nodo = self.raiz
        if nodo is not None:
        self.recorrido_inorden(nodo.izquierdo, resultado)
        resultado.append(nodo.valor)
        self.recorrido_inorden(nodo.derecho, resultado)
        return resultado

if __name__ == "__main__":
    bst = ArbolBusquedaBinaria()
    valores = [50, 30, 70, 20, 40, 60, 80]
    for valor in valores:
        bst.insertar(valor)
        print(f"¿Está el 40? {bst.buscar(40)}")
        print(f"¿Está el 100? {bst.buscar(100)}")
        print(f"Recorrido inorden (ordenado): {bst.recorrido_inorden()}")
```

Observemos cómo el recorrido inorden produce los valores ordenados. Esta es una propiedad fundamental de los BST que los hace muy útiles cuando necesitamos mantener datos ordenados.



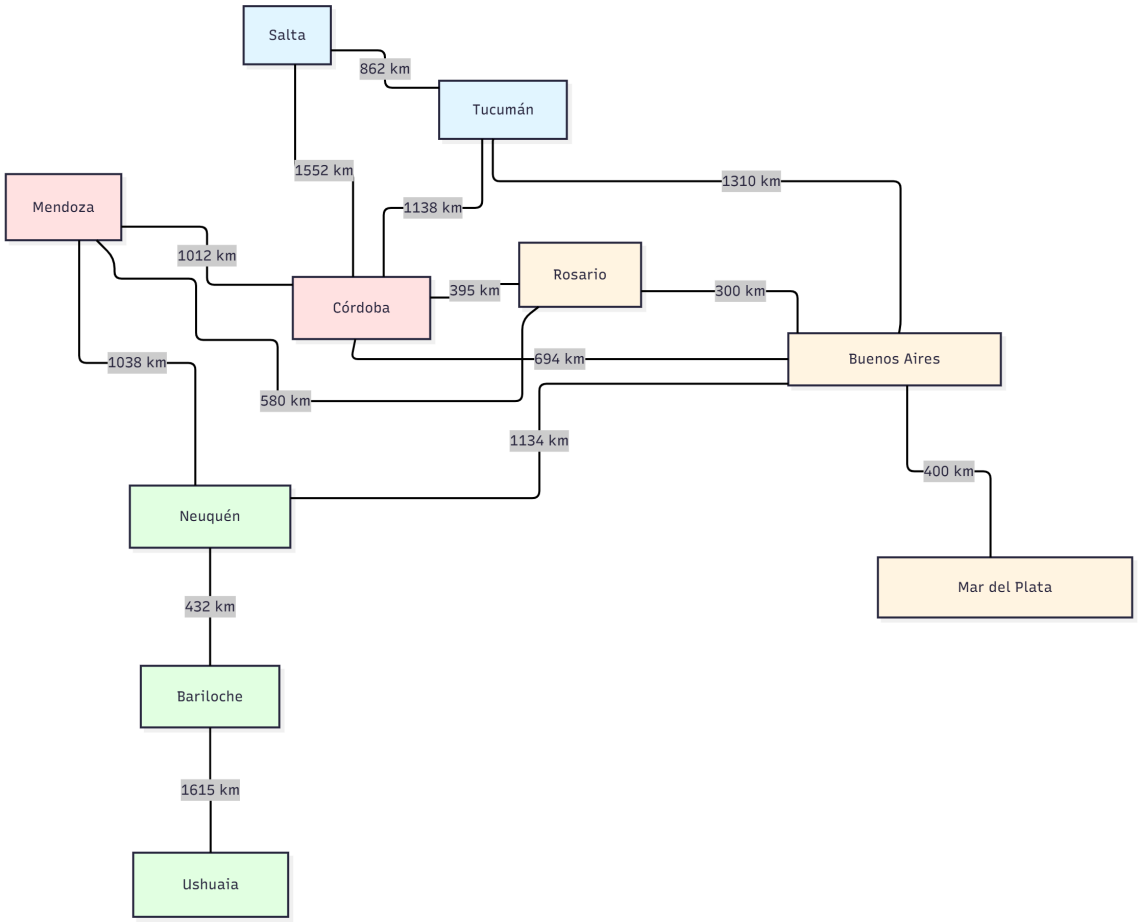
Fuente: Elaboración propia.

Grafos

Los grafos son estructuras que representan relaciones entre elementos. Mientras que en los árboles cada nodo tiene un solo padre y un camino único desde la raíz, en los grafos los nodos pueden conectarse de formas mucho más complejas y flexibles.

Hasta ahora trabajamos con estructuras donde las relaciones son claras y limitadas. Pero el mundo real está lleno de relaciones más complejas. En una red social, una persona puede tener muchos amigos, y esos amigos también están conectados entre sí. En un mapa de ciudades, cada ciudad puede conectarse con múltiples ciudades. Los grafos nos permiten modelar estas relaciones de muchos a muchos.

Los grafos son una de las estructuras más poderosas en inteligencia artificial. Los sistemas de recomendación como los de Netflix o Spotify usan grafos para entender qué usuarios tienen gustos similares. Los algoritmos de navegación como Google Maps usan grafos para encontrar la ruta más corta entre dos puntos. Las redes sociales usan grafos para sugerir amigos y detectar comunidades.



Fuente: Elaboración propia.

Énfasis

Un grafo es una estructura compuesta por vértices (o nodos) y aristas que los conectan. Según el tipo de relación que representen, los grafos pueden ser dirigidos, cuando las aristas tienen una dirección definida, o no dirigidos, cuando las conexiones son bidireccionales. Además, pueden ser ponderados, en cuyo caso cada arista posee un peso o costo asociado, que suele representar distancia, tiempo u otra magnitud relevante.

Ahora implementaremos grafos usando programación orientada a objetos. Crearemos clases para los vértices y para el grafo completo:

```
class Vertice:

    def __init__(self, id):
        self.id = id
        self.vecinos = {}

    def agregar_vecino(self, vecino, peso=1):
        self.vecinos[vecino] = peso

    def obtener_vecinos(self):
        return list(self.vecinos.keys())

    def obtener_peso(self, vecino):
        return self.vecinos.get(vecino, None)

class Grafo:
    def __init__(self, dirigido=False):
        self.vertices = {}
        self.dirigido = dirigido

    def agregar_vertice(self, id):
        if id not in self.vertices:
            self.vertices[id] = Vertice(id)
            return True
        return False

    def agregar_arista(self, origen, destino, peso=1):
        if origen not in self.vertices:
            self.agregar_vertice(origen)
        if destino not in self.vertices:
            self.agregar_vertice(destino)

        self.vertices[origen].agregar_vecino(destino, peso)

        if not self.dirigido:
            self.vertices[destino].agregar_vecino(origen, peso)

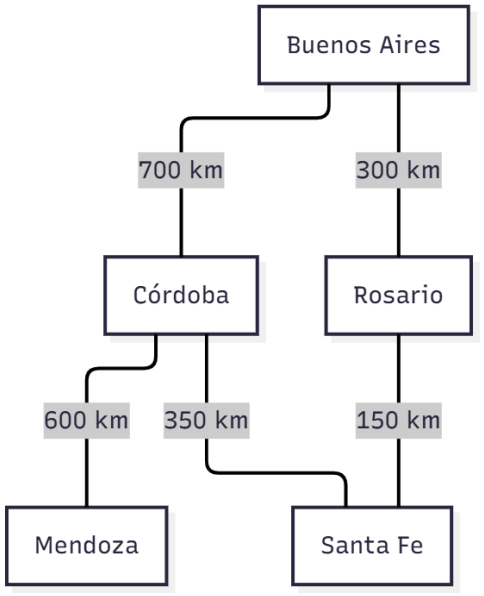
    def obtener_vecinos(self, id):
        vertice = self.vertices.get(id)
        if vertice:
            return vertice.obtener_vecinos()
        return []
```

```
def mostrar(self):
    for id_vertice in self.vertices:
        vertice = self.vertices[id_vertice]
        conexiones = [f"{v}(peso:{vertice.obtener_peso(v)})"
            for v in vertice.obtener_vecinos()]
        print(f"{id_vertice} → {', '.join(conexiones)}")
```

Esta implementación nos permite manejar tanto grafos dirigidos como no dirigidos, y soporta aristas ponderadas. Creemos y usemos un grafo de ciudades argentinas:

```
if __name__ == "__main__":
        grafo_ciudades = Grafo(dirigido=False)

        grafo_ciudades.agregar_arista("Buenos Aires", "Córdoba", 700)
        grafo_ciudades.agregar_arista("Buenos Aires", "Rosario", 300)
        grafo_ciudades.agregar_arista("Córdoba", "Mendoza", 600)
        grafo_ciudades.agregar_arista("Rosario", "Santa Fe", 150)
        grafo_ciudades.agregar_arista("Córdoba", "Santa Fe", 350)
        print("Estructura del grafo:")
        grafo_ciudades.mostrar()
        print(f"\nVecinos de Córdoba: {grafo_ciudades.obtener_vecinos('Córdoba')}")
```



Fuente: Elaboración propia.

Recorridos y búsqueda de caminos

Incorporaremos ahora métodos que permitan recorrer el grafo y calcular el camino más corto entre dos vértices, ampliando así sus posibilidades de análisis y resolución de problemas:

```
class Grafo: # ... métodos anteriores ...

    def bfs(self, inicio):
        if inicio not in self.vertices:
            return []

        visitados = set()
        cola = [inicio]
        visitados.add(inicio)
        resultado = []

        while len(cola) > 0:
            vertice_actual = cola.pop(0)
            resultado.append(vertice_actual)

            for vecino in self.obtener_vecinos(vertice_actual):
                if vecino not in visitados:
                    visitados.add(vecino)
                    cola.append(vecino)

        return resultado

    def camino_mas_corto(self, inicio, destino):
        if inicio not in self.vertices or destino not in self.vertices:
            return None

        if inicio == destino:
            return [inicio]

        visitados = set()
        cola = [inicio]
        visitados.add(inicio)
        padres = {inicio: None}

        while len(cola) > 0:
            vertice_actual = cola.pop(0)

            if vertice_actual == destino:
                camino = []
                nodo = destino
                while nodo is not None:
                    camino.append(nodo)
                    nodo = padres[nodo]
                return camino[::-1]

            for vecino in self.obtener_vecinos(vertice_actual):
                if vecino not in visitados:
                        visitados.add(vecino)
                        padres[vecino] = vertice_actual
                        cola.append(vecino)
```

```
        return None

if __name__ == "__main__":
        print("\nRecorrido BFS desde Buenos Aires:")
        print(grafo_ciudades.bfs("Buenos Aires"))

        print("\nCamino más corto de Buenos Aires a Mendoza:")
        camino = grafo_ciudades.camino_mas_corto("Buenos Aires", "Mendoza")
        if camino:
            print(" → ".join(camino))
```

El recorrido BFS (Breadth-First Search) explora el grafo por niveles. Comienza visitando el nodo inicial, continúa con todos sus vecinos directos y luego avanza hacia los vecinos de esos vecinos. Este tipo de recorrido resulta especialmente útil cuando se busca determinar el camino más corto entre dos nodos.

Aplicación en una red social

Para finalizar el módulo, apliquemos todo lo aprendido: POO, imports y organización de código. Para ello implementemos un sistema de red social, asumiendo que ya tenemos nuestras clases Grafo y Vertice guardadas en un módulo llamado grafos.py.

Veamos cómo se vería el código organizado de forma profesional:

Archivo: red_social.py

```
from grafos import Grafo

class RedSocial:
    def __init__(self):
        self.grafo = Grafo(dirigido=False)
        self.usuarios = {}

    def agregar_usuario(self, id, nombre):
        self.grafo.agregar_vertice(id)
        self.usuarios[id] = nombre

    def agregar_amistad(self, usuario1, usuario2):
        self.grafo.agregar_arista(usuario1, usuario2)

    def obtener_amigos(self, usuario):
        return self.grafo.obtener_vecinos(usuario)

    def sugerir_amigos(self, usuario):
        amigos = set(self.obtener_amigos(usuario))
        sugerencias = {}
        for amigo in amigos:
            amigos_del_amigo = self.obtener_amigos(amigo)
```

```
        for candidato in amigos_del_amigo:
            if candidato != usuario and candidato not in amigos:
                if candidato not in sugerencias:
                    sugerencias[candidato] = 0
                sugerencias[candidato] += 1

        sugerencias_ordenadas = sorted(sugerencias.items(), key=lambda x: x[1], reverse=True)
        return [(self.usuarios[id], count) for id, count in sugerencias_ordenadas]

    def grado_separacion(self, usuario1, usuario2):
        camino = self.grafo.camino_mas_corto(usuario1, usuario2)
        if camino:
                return len(camino)-1
        return None

if __name__ == "__main__":
    print("=== Probando Red Social ===\n")

red = RedSocial()
red.agregar_usuario("u1", "Ana")
red.agregar_usuario("u2", "Carlos")
red.agregar_usuario("u3", "Beatriz")
red.agregar_usuario("u4", "Diego")
red.agregar_usuario("u5", "Elena")
red.agregar_usuario("u6", "Franco")

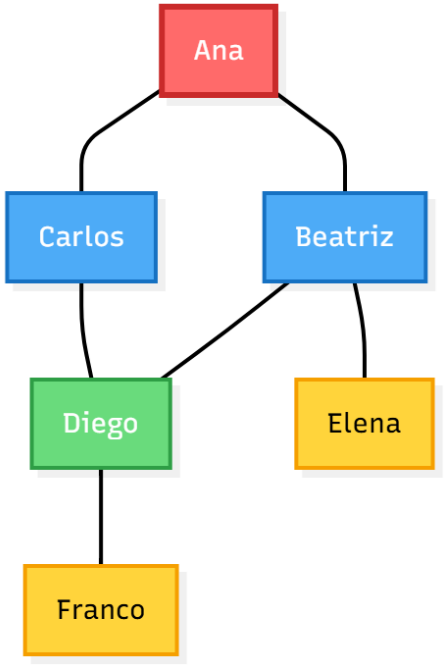
red.agregar_amistad("u1", "u2")
red.agregar_amistad("u1", "u3")
red.agregar_amistad("u2", "u4")
red.agregar_amistad("u3", "u4")
red.agregar_amistad("u3", "u5")
red.agregar_amistad("u4", "u6")

print(f"Amigos de Ana: {[red.usuarios[id] for id in red.obtener_amigos('u1')]}")

print("\nSugerencias de amistad para Ana:")
sugerencias = red.sugerir_amigos("u1")
for nombre, conexiones_comunes in sugerencias:
print(f" {nombre} ({conexiones_comunes} amigos en común)")
print(f"\nGrados de separación entre Ana y Franco: {red.grado_separacion('u1', 'u6')}")

print("\n¡Pruebas completadas!")
```

Observemos cómo organizamos el código de manera profesional. En la primera línea importamos la clase Grafo desde el módulo grafos.py. La clase RedSocial usa esta clase importada para manejar las conexiones. Todo el código de prueba está dentro del *if __name__ == "__main__":*, lo que significa que solo se ejecutará cuando corramos este archivo directamente.



Fuente: Elaboración propia.

Este sistema sugiere amigos basándose en el principio de amigos de amigos, exactamente cómo funcionan las redes sociales reales. Es una aplicación práctica y tangible de cómo los grafos modelan relaciones complejas del mundo real.

Con esto completamos nuestro recorrido por la programación orientada a objetos y su aplicación a estructuras de datos complejas. Hemos aprendido a pensar en términos de objetos que encapsulan datos y comportamiento. Vimos cómo la herencia nos permite reutilizar código y crear jerarquías de clases. Exploramos el polimorfismo que nos da flexibilidad para trabajar con objetos de diferentes tipos de forma uniforme.

Estos conceptos se aplicaron para implementar árboles binarios y grafos, estructuras fundamentales en el campo de la inteligencia artificial. Todo lo trabajado en este módulo constituye la base sobre la cual se construyen las bibliotecas y frameworks que se utilizarán más adelante. La programación orientada a objetos no debe entenderse solo como una técnica de programación, sino como una forma de pensar el diseño del software que permite organizar, escalar y manejar la complejidad propia de los sistemas grandes y reales.

Actividad

En este momento, está en condiciones de realizar la **actividad 3**.

Módulo 3 | Glosario

Árbol binario: estructura de datos jerárquica donde cada nodo puede tener como máximo dos hijos (izquierdo y derecho). Se usa para organizar información de forma eficiente y facilitar búsquedas rápidas.

Árbol de decisión: estructura arbórea usada en machine learning donde cada nodo interno representa una decisión basada en una característica, y cada hoja representa una clasificación o predicción final.

BFS (Breadth-First Search): algoritmo de recorrido de grafos que explora todos los vecinos de un nivel antes de pasar al siguiente nivel. Se implementa usando colas y encuentra caminos más cortos en grafos no ponderados.

Clase: plantilla o molde que define la estructura y comportamiento de un tipo de objeto. Especifica qué atributos tendrán los objetos y qué acciones podrán realizar mediante métodos.

DFS (Depth-First Search): algoritmo de recorrido de grafos que explora lo más profundo posible por cada rama antes de retroceder. Se implementa típicamente con pilas o recursión.

Encapsulamiento: principio de POO que consiste en ocultar los detalles internos de implementación y proteger datos sensibles, exponiendo solo lo necesario mediante una interfaz pública.

Grafo: estructura de datos formada por vértices (nodos) conectados por aristas (enlaces). Modela relaciones entre entidades y es fundamental para representar redes sociales, mapas de rutas, dependencias, entre otros.

Grafo dirigido: grafo donde las aristas tienen dirección, yendo de un vértice origen a un vértice destino. Se usa para modelar relaciones asimétricas como dependencias o flujos unidireccionales.

Grafo ponderado: grafo donde cada arista tiene un peso o valor asociado que representa costo, distancia, tiempo u otra métrica. Usado en problemas de optimización de rutas.

Herencia: mecanismo de POO que permite crear clases nuevas basadas en clases existentes, heredando sus atributos y métodos. La clase hija reutiliza código de la clase padre y puede agregar funcionalidad específica.

Módulo: archivo de Python que contiene definiciones de clases, funciones y variables que pueden importarse y reutilizarse en otros programas. Permite organizar el código en componentes separados y manejables.

Objeto: instancia concreta de una clase que tiene valores específicos en sus atributos. Si la clase es el molde, el objeto es cada pieza creada a partir de ese molde.

Polimorfismo: capacidad de objetos de diferentes clases de responder al mismo método de formas distintas. Permite tratar objetos de clases diferentes de manera uniforme cuando comparten una interfaz común.

Property (propiedad): decorador que permite controlar el acceso a atributos privados mediante métodos que se usan como si fueran atributos. Combina encapsulamiento con sintaxis simple.

Recorrido inorden: forma de visitar todos los nodos de un árbol binario siguiendo el orden: subárbol izquierdo, nodo actual, subárbol derecho. En árboles de búsqueda binaria produce valores ordenados.

Módulo 3 | Actividades

ACTIVIDAD 1

Primeros pasos con programación orientada a objetos

En la primera actividad del módulo aplicaremos los conceptos fundamentales de programación orientada a objetos: clases, objetos, atributos y métodos. El objetivo es que se familiaricen con la creación de clases, la instanciación de objetos y la definición de comportamientos que modelan situaciones del mundo real.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código Python completo y funcional, incluyendo capturas de pantalla de la ejecución de los programas mostrando diferentes casos de prueba para verificar que funcionan correctamente.

Recomendaciones

Preste especial atención a usar nombres descriptivos para las clases, atributos y métodos. Una clase bien diseñada es fácil de entender y usar. Recuerde que los nombres de clases por convención comienzan con mayúscula, mientras que los métodos y atributos usan minúsculas con guiones bajos. Piense en qué información necesita almacenar cada objeto y qué operaciones debe poder realizar.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos qué atributos y métodos necesita cada clase y ayudarse a identificar formas de mejorar el diseño del código.

Ejercicio 1: Sistema de biblioteca

Una biblioteca necesita un sistema para gestionar sus libros. Cada libro tiene información básica como título, autor y año de publicación. Los libros pueden estar disponibles o prestados. Escriba un programa que modele esta situación usando programación orientada a objetos. El sistema debe permitir prestar libros verificando que estén disponibles, devolverlos cuando un usuario los regresa, y consultar la información completa de cada libro incluyendo su estado actual. Cree varios libros de prueba y simule distintas operaciones de préstamo y devolución, mostrando cómo cambia el estado de disponibilidad.

Ejercicio 2: Gestión de cuentas bancarias

Un banco necesita un sistema para manejar las cuentas de sus clientes. Cada cuenta pertenece a un titular identificado por su nombre y número de cuenta, y tiene un saldo que inicialmente es cero. Los clientes pueden realizar depósitos, retiros y transferencias entre cuentas. Desarrolle un programa que implemente este sistema usando clases. El sistema debe validar que no se puedan retirar más fondos de los disponibles, que los montos sean siempre positivos, y que las transferencias solo se realicen si hay saldo suficiente. Cree varias cuentas de prueba y simule diferentes operaciones bancarias mostrando cómo evoluciona el saldo de cada cuenta.

Ejercicio 3: Control de inventario

Una tienda necesita controlar su inventario de productos. Cada producto tiene un nombre, un precio y una cantidad en stock. La tienda debe poder registrar nuevas entradas

de mercadería, realizar ventas que reduzcan el stock, aplicar descuentos porcentuales cuando sea necesario, y calcular el valor total de cada producto en inventario. Plantee un programa orientado a objetos que modele este sistema. El mismo debe poder manejar intentos de venta cuando no hay stock suficiente, validar que las cantidades sean positivas, y mantener actualizado el inventario. Cree varios productos diferentes y simule un día de operaciones con entradas de mercadería, ventas y aplicación de descuentos, mostrando reportes del estado del inventario.

Ejercicio 4: Figuras geométricas

Lleve a cabo un programa que modele rectángulos usando programación orientada a objetos. Cada rectángulo se define por su base y altura. El sistema debe poder calcular el área y perímetro de cualquier rectángulo, determinar si un rectángulo es en realidad un cuadrado, y modificar sus dimensiones aplicando un factor de escala. Cree varios rectángulos con diferentes dimensiones, realice cálculos sobre ellos, identifique cuáles son cuadrados, y pruebe el escalado de figuras mostrando cómo cambian sus propiedades.

Ejercicio 5: Registro académico

Una institución educativa necesita gestionar la información académica de sus estudiantes. Cada estudiante tiene datos personales como nombre, edad y carrera. Durante su cursada, los estudiantes se inscriben en materias y reciben calificaciones. El sistema debe permitir inscribir estudiantes en materias verificando que no se inscriban dos veces en la misma, registrar sus notas validando que estén en el rango de 0 a 10, calcular el promedio general, y determinar si el estudiante está en condición de aprobar o reprobar considerando que se aprueba con promedio mayor o igual a 6. Escriba un programa orientado a objetos que implemente este sistema. Cree varios estudiantes, inscríbalos en diferentes materias, registre distintas notas y genere reportes académicos completos.

Ejercicio 6: Gestor de tareas

Diseñe un sistema para la gestión de tareas pendientes. Cada tarea debe contar con una descripción, un nivel de prioridad (baja, media o alta), un estado que indique si se encuentra pendiente o completada, y la fecha de creación. El gestor de tareas debe poder almacenar múltiples tareas, marcarlas como completadas, cambiar su prioridad, filtrar las tareas pendientes, agruparlas por nivel de prioridad y generar estadísticas sobre cuántas están completadas. El sistema debe implementarse utilizando dos clases: una que represente las tareas individuales y otra que funcione como gestor. Cree varias tareas con distintas prioridades, simule la finalización de algunas de ellas y genere reportes que muestren las tareas pendientes organizadas por prioridad.

ACTIVIDAD 2
Aplicando encapsulamiento, herencia y polimorfismo

En esta segunda actividad aplicaremos las características avanzadas de la programación orientada a objetos: encapsulamiento, herencia y polimorfismo. El objetivo es crear jerarquías de clases que reutilicen código, protejan datos sensibles mediante encapsulamiento y aprovechen el poder de la POO para resolver problemas complejos de forma elegante.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código Python completo y funcional, incluyendo capturas de pantalla de la ejecución mostrando diferentes casos de prueba.



Recomendaciones

Al diseñar jerarquías de clases, identifique claramente qué características y comportamientos son comunes y deben estar en la clase padre y cuáles son específicos de cada clase hija. Use encapsulamiento para proteger datos sensibles que no deban modificarse directamente. Piense en qué información debe ser pública, protegida o privada según el nivel de acceso necesario.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos qué debe ir en la clase padre y qué en las clases hijas, qué datos deben protegerse mediante encapsulamiento y ayudarse a detectar oportunidades de mejora en el diseño.

Ejercicio 1: Flota de vehículos

Una empresa de transporte maneja diferentes tipos de vehículos: autos, motos y camiones. Todos los vehículos comparten características básicas como marca, modelo, año y velocidad actual que comienza en cero. Todos pueden acelerar aumentando su velocidad y frenar reduciéndola sin llegar a valores negativos. Sin embargo, cada tipo tiene particularidades: los autos tienen cantidad de puertas, las motos tienen cilindrada y los camiones tienen capacidad de carga y pueden estar cargados o vacíos. Escriba un programa usando herencia que modele esta situación. Diseñe una clase padre para las características comunes y clases hijas para cada tipo específico. Cree varios vehículos de cada tipo y muestre cómo el polimorfismo permite tratarlos de manera uniforme cuando es necesario.

Ejercicio 2: Sistema de nómina con encapsulamiento

Una empresa tiene diferentes tipos de empleados con distintas formas de calcular su salario. Todos los empleados tienen información básica como nombre, identificador y un salario base. El cálculo del salario final varía según el tipo: los empleados de tiempo completo reciben una bonificación del 20% sobre su salario base, los empleados por hora cobran según las horas trabajadas multiplicadas por su tarifa base y los empleados por comisión reciben su salario base más un porcentaje de sus ventas. Escriba un programa que implemente este sistema usando herencia y encapsulamiento. Identifique qué información sensible debe protegerse mediante atributos privados o protegidos y valide los datos antes de almacenarlos. El sistema debe poder procesar la nómina de una lista con empleados de diferentes tipos, calculando cuánto corresponde pagarle a cada uno y el total general. Cree varios empleados de cada tipo, demuestre las validaciones del encapsulamiento, y genere un reporte completo de nómina.

Ejercicio 3: Cálculo de volúmenes y áreas

Escriba un programa para calcular propiedades de figuras tridimensionales. Necesitará modelar cubos, esferas y cilindros. Todas estas figuras pueden calcular su volumen y el área de su superficie, aunque las fórmulas son diferentes para cada una. Un cubo se define por la longitud de su lado, una esfera por su radio y un cilindro por su radio y altura. Use las siguientes fórmulas: para el cubo el volumen es lado^3 y el área superficie es 6 ×

lado²; para la esfera el volumen es (4/3) × π × radio³ y el área superficie es 4 × π × radio²; para el cilindro el volumen es π × radio² × altura y el área superficie es 2 × π × radio × (radio + altura). Implemente este sistema usando herencia con una clase padre que defina la interfaz común y clases hijas que implementen los cálculos específicos. El programa debe poder procesar una colección de figuras mixtas y calcular totales, demostrando polimorfismo.

Ejercicio 4: Cuentas bancarias con control de acceso

Mejore el sistema de cuentas bancarias de la actividad anterior agregando tipos especializados y aplicando encapsulamiento riguroso. Identifique qué datos deben protegerse y no puedan modificarse directamente desde fuera de la clase. Cree tres tipos especializados: cuentas de ahorro que generan intereses periódicos sobre el saldo y tienen un límite de retiros mensuales, cuentas corrientes que permiten sobregiros hasta cierto límite dejando el saldo en negativo, y cuentas de inversión que tienen un rendimiento anual y un plazo mínimo durante el cual no se puede retirar dinero. Use herencia para evitar duplicar código y encapsulamiento para proteger información sensible. Cree cuentas de cada tipo, realice operaciones específicas demostrando tanto las validaciones como las diferencias entre tipos.

Ejercicio 5: Sistema de sensores IoT

Desarrolle un sistema para manejar diferentes tipos de sensores IoT que recopilan datos ambientales. Todos los sensores tienen un identificador, ubicación, estado de batería y un historial de lecturas. Los tipos específicos incluyen: sensores de temperatura que miden grados Celsius, sensores de humedad que miden porcentaje y sensores de calidad de aire que dan un índice numérico. Escriba un programa usando herencia donde cada sensor pueda tomar lecturas, validarlas según su rango permitido y almacenarlas. Use encapsulamiento para proteger el historial de lecturas y el nivel de batería, permitiendo solo las operaciones necesarias mediante métodos apropiados. Cada tipo de sensor debe validar que sus lecturas estén en rangos razonables. El sistema debe poder procesar lecturas de múltiples sensores simultáneamente, generar reportes agregados, detectar sensores con batería baja e identificar lecturas anormales. Cree una red de al menos diez sensores de diferentes tipos y simule la toma de múltiples lecturas.

Ejercicio 6: Sistema de publicaciones académicas

Una biblioteca digital maneja diferentes tipos de publicaciones académicas: artículos científicos, libros académicos y tesis de grado. Todas las publicaciones tienen información común como título, lista de autores, año de publicación y un contador de citaciones. Sin embargo, cada tipo tiene información específica: los artículos tienen nombre de revista y volumen, los libros tienen editorial y número ISBN y las tesis tienen universidad y tipo de grado. Además, cada tipo genera referencias bibliográficas en formato diferente. Escriba un programa usando herencia que modele este sistema. Incluya una clase para la biblioteca digital que pueda almacenar publicaciones de cualquier tipo, buscarlas por autor, filtrar las más relevantes y generar automáticamente una bibliografía completa demostrando polimorfismo. Cree varias publicaciones de diferentes tipos, registre citaciones y genere reportes y bibliografías.

ACTIVIDAD 3
Árboles, grafos y aplicaciones en IA

En esta tercera y última actividad del módulo aplicaremos todo lo aprendido sobre programación orientada a objetos para implementar y trabajar con estructuras de datos avanzadas: árboles y grafos. Como vimos, estas estructuras son fundamentales en inteligencia artificial para problemas de clasificación, búsqueda, análisis de relaciones y toma de decisiones.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código Python completo y funcional, incluyendo capturas de pantalla de la ejecución mostrando diferentes casos de prueba que demuestren que el código funciona correctamente.

Recomendaciones

Estas estructuras son más complejas que las vistas anteriormente. Trabaje con paciencia, pruebe cada funcionalidad individualmente antes de integrar todo el sistema y no dude en crear funciones o métodos auxiliares que faciliten el trabajo. El código debe estar bien organizado, idealmente usando módulos separados que puedan importarse. Recuerde usar la estructura if **name** == "**main**": para separar el código de prueba del código reutilizable.

Estos ejercicios son ideales para trabajar en pareja. Pueden dividirse las tareas: uno/a implementa las estructuras básicas y otro/a los algoritmos de recorrido o búsqueda. Luego integren y prueben juntos el sistema completo.

Ejercicio 1: Árbol genealógico

Una familia necesita un sistema para registrar y consultar su árbol genealógico. Cada persona en el árbol tiene información como nombre y año de nacimiento y puede tener referencias a su padre y madre. El sistema debe permitir registrar estas relaciones familiares y luego realizar consultas interesantes: encontrar todos los abuelos de una persona, identificar hermanos que compartan al menos un padre, obtener ancestros hasta cierto nivel de generaciones, determinar si dos personas son familiares porque tienen ancestros en común y visualizar el árbol desde cualquier persona. Escriba un programa orientado a objetos que implemente este sistema. Cree un árbol genealógico con al menos tres generaciones incluyendo abuelos, padres, hijos y nietos, establezca todas las relaciones y realice distintas consultas demostrando que el sistema encuentra correctamente las conexiones familiares.

Ejercicio 2: Evaluador de expresiones matemáticas

Los árboles son ideales para representar y evaluar expresiones matemáticas. Escriba un programa que use un árbol para modelar expresiones matemáticas donde los nodos internos son operadores y las hojas son números. Por ejemplo, la expresión "(3 + 5) × 2" se representa con "×" en la raíz, "+" como hijo izquierdo (que tiene 3 y 5 como hijos), y 2 como hijo derecho. El sistema debe poder construir estos árboles, evaluar la expresión recorriendo el árbol recursivamente y mostrar la expresión en diferentes notaciones: infija (la forma normal que usamos), prefija (operador antes de los operandos) y postfija

(operador después de los operandos). Cree varios árboles de expresiones con diferentes complejidades y verifique que las evaluaciones sean correctas.

Ejercicio 3: Árbol de decisión para clasificación

Los árboles de decisión son fundamentales en inteligencia artificial para problemas de clasificación. Escriba un programa que implemente un árbol de decisión simple para clasificar datos. Puede usar como ejemplo el problema de predecir si alguien debiera llevar paraguas basándose en condiciones como si está lloviendo, la probabilidad de lluvia y la temperatura. El sistema debe poder construir el árbol definiendo decisiones en cada nodo interno y clasificaciones en las hojas, hacer predicciones siguiendo el camino apropiado según los datos de entrada, evaluar la precisión del árbol comparando predicciones con resultados esperados y exportar las reglas de decisión en formato legible para humanos. Use programación orientada a objetos para modelar los nodos de decisión y el árbol completo. Cree un árbol de decisión, entrénelo o defínalo manualmente con reglas lógicas, pruébelo con varios casos y muestre las reglas que aprendió o definió.

Ejercicio 4: Red social con análisis de conexiones

Las redes sociales usan grafos para representar relaciones entre usuarios. Escriba un programa que modele una red social donde cada usuario tiene información como identificador, nombre, edad e intereses. Los usuarios pueden ser amigos entre sí, formando un grafo no dirigido donde las aristas representan amistades. El sistema debe poder agregar usuarios y amistades, obtener la lista de amigos de cualquier usuario, sugerir nuevas amistades basándose en amigos en común, calcular los grados de separación entre dos usuarios mostrando cuántas conexiones los separan, encontrar al usuario más popular contando sus amigos, recomendar amigos basándose en intereses comunes y generar estadísticas de la red. Implemente este sistema usando programación orientada a objetos con clases para usuarios y para el grafo de la red social. Cree una red con al menos diez usuarios, establezca múltiples conexiones de amistad, agregue intereses variados y pruebe todas las funcionalidades de análisis social.

Ejercicio 5: Sistema de navegación y rutas óptimas

Los sistemas de navegación como Google Maps usan grafos ponderados donde los vértices son ubicaciones y las aristas tienen pesos que representan distancias o tiempos. Escriba un programa que modele un sistema de navegación para ciudades argentinas. El sistema debe permitir agregar ciudades con sus coordenadas aproximadas, conectarlas con rutas bidireccionales indicando las distancias, encontrar el camino más corto entre dos ciudades usando algún algoritmo de búsqueda, calcular el tiempo estimado de viaje dada una velocidad promedio, determinar qué rutas pasan por una ciudad específica e identificar la ciudad más conectada. Use programación orientada a objetos para modelar el grafo de rutas y un sistema de navegación que lo utilice. Cree un mapa con al menos diez ciudades argentinas conectadas por rutas con distancias realistas y pruebe el sistema planificando viajes entre diferentes destinos mostrando las rutas óptimas encontradas.

Ejercicio 6: Sistema de planificación de materias con prerrequisitos

Las universidades organizan sus planes de estudio usando prerrequisitos: algunas materias deben cursarse antes que otras. Escriba un programa que modele este sistema usando un grafo dirigido donde cada materia es un vértice y una arista de A a B significa que A es prerrequisito de B (debe cursarse antes). Cada materia tiene información como nombre, código, cuatrimestre sugerido y cantidad de horas. El sistema debe poder agregar materias y establecer relaciones de prerrequisitos entre ellas, determinar en qué orden un estudiante puede cursar las materias respetando los prerrequisitos, identificar materias que no tienen prerrequisitos y pueden cursarse desde el inicio, detectar si hay errores en el plan de estudios como ciclos imposibles (A es prerrequisito de B, B de C y C de A), encontrar todos los prerrequisitos directos e indirectos necesarios para cursar una materia específica y sugerir qué materias puede cursar un estudiante dado su historial académico. Use programación orientada a objetos para modelar las materias y el grafo del plan de estudios. Cree un plan con al menos doce materias de una carrera universitaria con sus prerrequisitos, calcule el orden óptimo de cursada y genere reportes de planificación académica.

Módulo 4

Microobjetivos

- Interpretar y organizar datos numéricos y tabulares mediante NumPy y Pandas para preparar datasets de manera eficiente y profesional, estableciendo la base conceptual para proyectos de ciencia de datos e inteligencia artificial.

- Implementar modelos de machine learning tradicional con Scikit-learn aplicando algoritmos de clasificación y regresión y técnicas de preprocesamiento adecuadas, con el fin de analizar y resolver problemas reales de predicción de datos.

- Conceptualizar y estructurar redes neuronales básicas con TensorFlow y Keras comprendiendo los conceptos fundamentales de deep learning como capas, funciones de activación y optimizadores, para dar los primeros pasos en el desarrollo de sistemas de inteligencia artificial más complejos.

Contenidos

MÓDULO 4: MANEJO DE DATOS Y BIBLIOTECAS DE IA



Fuente: Freepik

Llegamos al cuarto y último módulo de Programación para Inteligencia Artificial, un momento emocionante porque aquí es donde todo lo aprendido cobra vida aplicado al campo de la inteligencia artificial. Hemos recorrido un largo camino: comprendimos Python desde sus fundamentos, exploramos estructuras de datos esenciales como listas, diccionarios, pilas y colas y aprendimos programación orientada a objetos, modelando el mundo en clases y objetos con herencia y polimorfismo. Incluso implementamos árboles binarios y grafos, estructuras complejas que modelan relaciones jerárquicas y conexiones entre datos.

Todo ese conocimiento fue preparándonos para este momento. Ahora vamos a trabajar con las bibliotecas que convierten a Python en la herramienta dominante para inteligencia artificial y ciencia de datos. Estas bibliotecas no solo facilitan el trabajo, sino que están optimizadas para manejar millones de datos con velocidad y eficiencia. Sin ellas, construir sistemas de IA sería extremadamente complicado y lento.

En este módulo aprenderemos NumPy, la biblioteca fundamental para cálculo numérico que nos permite trabajar con arrays multidimensionales de forma eficiente. Luego exploraremos Pandas, la herramienta por excelencia para manipular datos tabulares, leer archivos, limpiar información y transformar datasets. Estos dos pilares son la base de todo proyecto de ciencia de datos e inteligencia artificial. Después nos adentraremos en el aprendizaje automático con Scikit-learn, una biblioteca que encapsula algoritmos de clasificación y regresión en interfaces simples y elegantes. Veremos cómo preprocesar datos, entrenar modelos y hacer predicciones. Finalmente, daremos nuestros primeros pasos en deep learning con TensorFlow y Keras, construyendo redes neuronales básicas, y exploraremos PyTorch, otra biblioteca popular para desarrollo de modelos de aprendizaje profundo.

Es fundamental que practiquemos mucho en este módulo. El código se aprende escribiéndolo, ejecutándolo, viendo los resultados y modificándolo para entender qué sucede. Las bibliotecas de IA tienen documentación extensa, pero el mejor aprendizaje viene de la experimentación. Estamos en el umbral de construir sistemas inteligentes, y este recorrido nos da las herramientas concretas para hacerlo.

¡Comencemos este último módulo con toda la energía!

Preparando nuestro entorno

Antes de trabajar con las bibliotecas de inteligencia artificial, necesitamos entender cómo instalarlas y gestionarlas. Python, por sí solo, viene con muchas funcionalidades, pero las bibliotecas especializadas como NumPy, Pandas, TensorFlow o PyTorch no están incluidas en la instalación base. Para usarlas, debemos instalarlas primero.

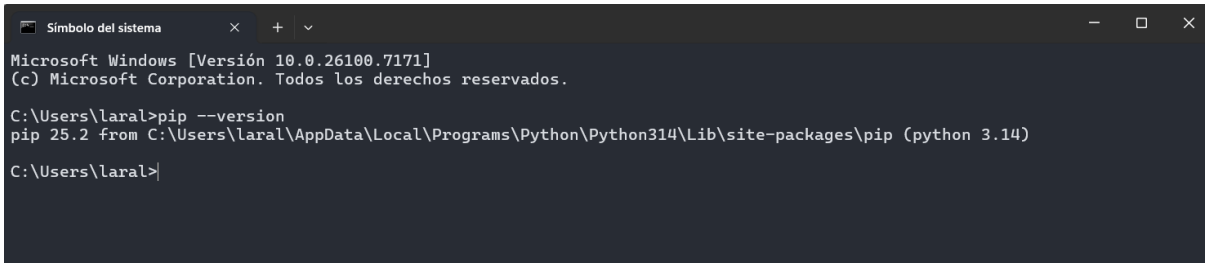
Énfasis

Pip es el gestor de paquetes de Python. Nos permite instalar, actualizar y eliminar bibliotecas de forma sencilla desde la línea de comandos. Es la herramienta estándar para gestionar dependencias en proyectos Python.

Cuando instalamos Python siguiendo los pasos del módulo uno, pip se instaló automáticamente junto con él. Para verificar que está disponible, abrimos una terminal o consola de comandos y escribimos:

pip —version

Si todo está correcto, aparecerá un mensaje indicando la versión de pip instalada, algo como "pip 23.2.1 from..." o similar. Esto confirma que pip está listo para usar.



Fuente: Elaboración propia.

En algunos sistemas, especialmente en Linux o macOS, es posible que necesitemos usar pip3 en lugar de pip para referirnos específicamente a Python 3. Si pip no funciona, probemos con pip3 —version.

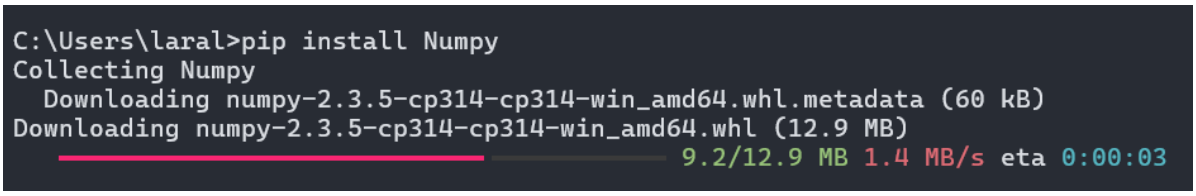
La sintaxis básica para instalar una biblioteca es muy simple. Abrimos una terminal y escribimos:

```
pip install nombre_biblioteca
```

Por ejemplo, para instalar NumPy, escribimos:

```
pip install numpy
```

pip se conecta automáticamente al índice de paquetes de Python, conocido como PyPI (Python Package Index), descarga la biblioteca y todas sus dependencias, y las instala en nuestro sistema. El proceso toma unos segundos o minutos dependiendo del tamaño de la biblioteca y de la velocidad de nuestra conexión.



Fuente: Elaboración propia.

Durante la instalación veremos mensajes indicando el progreso: qué archivos se están descargando y qué dependencias se están resolviendo. Al finalizar, aparecerá un mensaje como "Successfully installed numpy-1.24.3" confirmando que la instalación fue exitosa.

Podemos instalar múltiples bibliotecas en un solo comando separándolas con espacios:

```
pip install numpy pandas matplotlib
```

Esto instala NumPy, Pandas y Matplotlib en una sola operación, lo cual es muy conveniente cuando configuramos un proyecto nuevo.

Si queremos instalar una versión específica de una biblioteca, podemos especificarla:

```
pip install numpy==1.24.3
```

Esto asegura que instalamos exactamente esa versión, lo cual es útil cuando trabajamos en proyectos que requieren compatibilidad estricta. Con el tiempo, las bibliotecas se actualizan con nuevas funcionalidades y correcciones. Para actualizar una biblioteca a su última versión:

```
pip install —upgrade numpy
```

Si necesitamos desinstalar una biblioteca:

```
pip uninstall numpy
```

pip pedirá confirmación antes de eliminarla, asegurándonos de que realmente queremos hacerlo.

Para ver todas las bibliotecas instaladas en nuestro sistema:

```
pip list
```

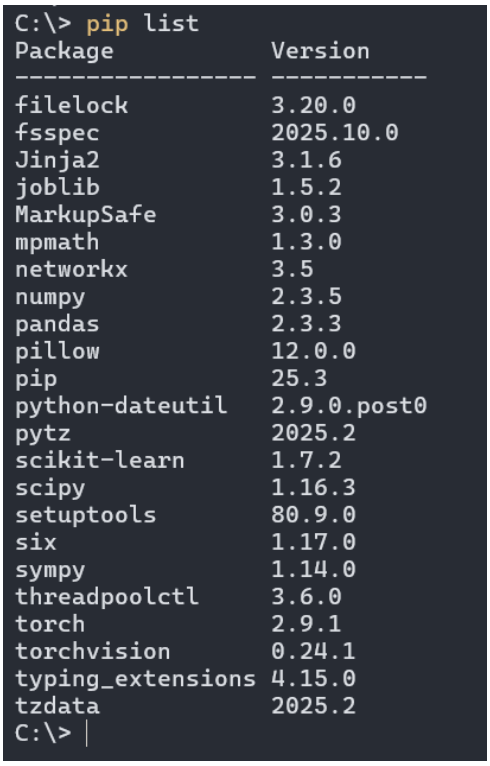
Esto muestra un listado completo con los nombres y versiones de cada paquete instalado.

Instalando las bibliotecas que usaremos en este módulo

Para seguir este módulo, necesitamos instalar varias bibliotecas. Abramos una terminal y ejecutemos los siguientes comandos uno por uno:

```
pip install numpy
pip install pandas
pip install scikit-learn
pip install tensorflow
pip install torch torchvision
```

Cada instalación puede tardar un poco, especialmente TensorFlow y PyTorch que son bibliotecas grandes. Es normal que descarguen varios cientos de megabytes. Una vez completada cada instalación, estaremos listos para usarlas en nuestros programas.



Fuente: Elaboración propia.

Con nuestro entorno preparado, estamos listos para comenzar a explorar estas poderosas herramientas.

NumPy, la base del cálculo numérico en Python

NumPy es la biblioteca fundamental para cálculo numérico en Python. Su nombre proviene de "Numerical Python" y es la base sobre la cual se construyen casi todas las demás bibliotecas de ciencia de datos e inteligencia artificial. Pandas, Scikit-learn, TensorFlow y PyTorch, todas dependen de NumPy o usan conceptos similares.

Énfasis

NumPy proporciona arrays multidimensionales eficientes y operaciones matemáticas optimizadas para trabajar con grandes cantidades de datos numéricos. Estas operaciones son mucho más rápidas que las equivalentes con listas de Python.

Recordemos que en el módulo dos trabajamos con listas para almacenar colecciones de datos. Las listas son versátiles y fáciles de usar, pero tienen limitaciones cuando trabajamos con grandes volúmenes de datos numéricos. Las operaciones matemáticas elemento por elemento son lentas porque Python debe procesar cada elemento individualmente.

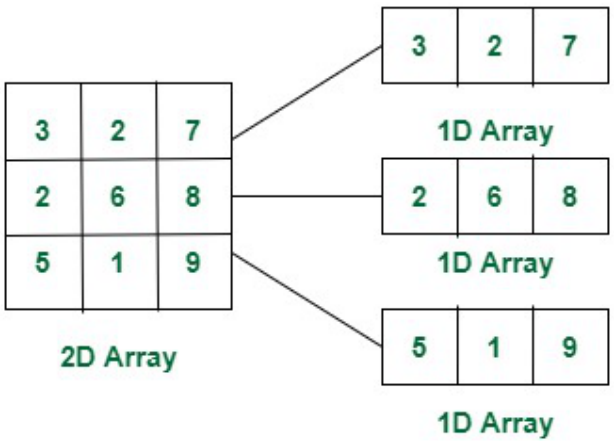
NumPy resuelve este problema con arrays: estructuras de datos especializadas para almacenar números de forma eficiente en memoria y procesarlos usando operaciones vectorizadas implementadas en lenguajes de bajo nivel como C y Fortran. El resultado es que las operaciones son decenas o cientos de veces más rápidas que con listas.

Para usar NumPy, primero debemos importarlo. Por convención, se importa con el alias np:

```
import numpy as np
```

Podemos crear arrays de varias formas:

```
lista = [1, 2, 3, 4, 5]
array = np.array(lista)
print(array)
matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matriz)
```



Fuente: https://www.geeksforgeeks.org/dsa/how-to-iterate-a-multidimensional-array/

NumPy ofrece funciones para crear arrays especiales:

```
ceros = np.zeros((3, 4))
unos = np.ones((2, 3))
rango = np.arange(0, 10, 2)
lineal = np.linspace(0, 1, 5)
print(ceros)
print(unos)
print(rango)
print(lineal)
```

Propiedades fundamentales de los arrays

Los arrays tienen propiedades importantes que nos ayudan a entender su estructura:

```
print(f"Forma del vector: {array.shape}")
print(f"Forma de la matriz: {matriz.shape}")
print(f"Dimensiones del vector: {array.ndim}")
print(f"Total de elementos: {array.size}")
print(f"Tipo de datos: {array.dtype}")
```

La propiedad shape indica las dimensiones del array. Por ejemplo, en un vector de cinco elementos, su valor es (5,), mientras que en una matriz de dos filas y tres columnas es (2, 3). La propiedad ndim señala la cantidad de dimensiones: un vector tiene una dimensión y una matriz tiene dos. La propiedad size informa el número total de elementos del array, y dtype especifica el tipo de datos que almacena, como int32 o float64.

Indexación y slicing

El acceso a los elementos de un array se realiza de forma similar al acceso a los elementos de una lista, utilizando índices:

```
array = np.array([10, 20, 30, 40, 50])
matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(array[0])
print(matriz[1, 2])
print(matriz[0:2, 1:3])
print(matriz[1, :])
print(matriz[:, 1])
```

Operaciones matemáticas básicas

Las operaciones se aplican elemento por elemento automáticamente:

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
print(a + b)
print(a * b)
print(b / a)
print(a ** 2)
print(a + 10)
```

Además NumPy incluye funciones matemáticas avanzadas:

```
array = np.array([1, 4, 9, 16, 25])
print(f"Raices cuadradas: {np.sqrt(array)}")
```

Operaciones estadísticas

En cuanto a este tipo de operaciones, NumPy ofrece funciones para calcular estadísticas sobre arrays. La media es el promedio de todos los valores, la mediana es el valor central cuando ordenamos los datos y la desviación estándar mide cuánto se dispersan los valores respecto al promedio:

```
datos = np.array([12, 15, 18, 22, 25, 30, 35])
media = np.mean(datos)
mediana = np.median(datos)
desviacion = np.std(datos)
maximo = np.max(datos)
minimo = np.min(datos)
suma = np.sum(datos)
print(f"Media: {media}")
print(f"Mediana: {mediana}")
print(f"Desviación estándar: {desviacion}")
print(f"Máximo: {maximo}")
print(f"Mínimo: {minimo}")
print(f"Suma: {suma}")
```

Reshape y concatenación

Además de realizar cálculos estadísticos, NumPy permite reorganizar la forma de los arrays. Con reshape() es posible cambiar sus dimensiones sin modificar los datos que contienen:

```
array = np.arange(12)
matriz = array.reshape(3, 4)
print(matriz)
```

También se pueden unir arrays entre sí. La función vstack() los concatena de forma vertical, mientras que hstack() lo hace de manera horizontal:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.vstack([a, b]))
print(np.hstack([a, b]))
```

Broadcasting: operaciones entre arrays de diferentes formas

Supongamos que contamos con una matriz que contiene las notas de tres estudiantes en tres exámenes y queremos sumar un puntaje extra a cada examen. Sin utilizar broadcasting, sería necesario recorrer la matriz con un bucle y agregar el bonus fila por fila. En cambio, gracias al broadcasting, NumPy permite realizar esta suma de forma directa y automática, aplicando el ajuste a todos los elementos correspondientes sin escribir código adicional de iteración.



El broadcasting es la capacidad de NumPy de realizar operaciones entre arrays de diferentes tamaños sin necesidad de bucles explícitos. NumPy automáticamente "expande" el array más pequeño para que coincida con el más grande.

Veamos cómo funciona:

```
notas = np.array([[85, 90, 78], [92, 88, 95], [78, 85, 82]])
print("Notas originales:")
print(notas)
bonus = np.array([5, 3, 2])
notas_con_bonus = notas + bonus
print("\nNotas con bonus:")
print(notas_con_bonus)
```

Aquí ocurre algo interesante. La matriz notas tiene forma tres por tres, pero el vector bonus solo tiene tres elementos. NumPy automáticamente entiende que queremos sumar el bonus a cada fila de la matriz. Es como si NumPy copiara el vector bonus tres veces para crear una matriz del mismo tamaño y luego sumara. Pero esto ocurre sin copiar datos realmente, lo que hace el proceso muy eficiente.

Otro caso común es cuando queremos aplicar la misma operación a todas las columnas. Supongamos que queremos convertir temperaturas de Celsius a Fahrenheit para varias ciudades a lo largo de varios días:

```
temperaturas_celsius = np.array([[20, 22, 19], [25, 28, 24], [18, 21, 20]])
temperaturas_fahrenheit = temperaturas_celsius * 9/5 + 32
print(temperaturas_fahrenheit)
```

Aquí multiplicamos y sumamos valores escalares a toda la matriz. El broadcasting permite que estos números se apliquen a cada elemento automáticamente. Sin broadcasting, tendríamos que escribir bucles anidados, haciendo el código más lento y difícil de leer.

De esta manera, el broadcasting resulta muy útil en machine learning cuando procesamos múltiples muestras simultáneamente. Por ejemplo, si tenemos mil imágenes y queremos restar la media de cada píxel, el broadcasting nos permite hacerlo en una sola operación en lugar de procesar cada imagen por separado.

Cómo normalizar datos, un ejemplo

En machine learning es crucial que todas las características tengan escalas similares. Imaginemos que estamos prediciendo el precio de una casa a partir de dos características: la cantidad de habitaciones, que va de uno a cinco, y la superficie en metros cuadrados, que va de cincuenta a quinientos. Si alimentamos estos datos directamente a un algoritmo, la superficie dominará el aprendizaje simplemente porque sus números son mucho más grandes, no porque sea más importante.



La normalización transforma los datos para que tengan media cero y desviación estándar uno. Esto pone todas las características en la misma escala, permitiendo que el algoritmo de machine learning las trate con igual importancia.

Trabajemos en un ejemplo concreto. Supongamos que tenemos los ingresos mensuales de diez personas en miles de pesos:

```
ingresos = np.array([50, 65, 45, 70, 55, 80, 60, 75, 52, 68])
media = np.mean(ingresos)
desviacion = np.std(ingresos)
ingresos_normalizados = (ingresos–media) / desviacion
print(f"Media normalizada: {np.mean(ingresos_normalizados):.10f}")
print(f"Desviación normalizada: {np.std(ingresos_normalizados):.2f}")
```

Después de la normalización, la media es prácticamente cero y la desviación estándar es uno. Esta transformación no cambia la distribución ni las relaciones entre los datos, solo los reescala. La normalización facilita que los algoritmos de machine learning funcionen mejor, especialmente aquellos sensibles a la escala como regresión logística o redes neuronales que veremos más adelante.

Con NumPy internalizado, tenemos la base para trabajar con cualquier biblioteca de ciencia de datos e inteligencia artificial. Sus conceptos de arrays, operaciones vectorizadas y manipulación eficiente de datos aparecen constantemente en todo el ecosistema de Python para IA.

Pandas: manipulación de datos tabulares

Pandas es la biblioteca más popular para trabajar con datos estructurados en Python. Su nombre proviene de "Panel Data", un término estadístico para conjuntos de datos multidimensionales. Pandas está construida sobre NumPy y hereda su eficiencia, pero agrega estructuras de datos de alto nivel diseñadas específicamente para análisis de datos.



Pandas nos permite trabajar con datos tabulares, como hojas de cálculo o tablas de bases de datos, de forma intuitiva y eficiente. Es la herramienta fundamental para limpieza, transformación y análisis exploratorio de datos.

Cuando trabajamos en proyectos de inteligencia artificial, pasamos mucho tiempo preparando datos. Los datos reales son desordenados: tienen valores faltantes, tipos inconsistentes, formatos diferentes, duplicados. Pandas nos da las herramientas para limpiar y transformar esos datos hasta tenerlos listos para entrenar modelos.

Importamos Pandas con el alias pd por convención:

```
import pandas as pd
import numpy as np
```

Series: la estructura unidimensional

Pandas usa *Series* para datos unidimensionales con índice:

```
serie = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
print(serie)
print(serie['a'])
print(serie.mean())
```

Obtendremos una salida como esta:

```
Serie:
a 10
b 20
c 30
d 40
e 50
dtype: int64
Valor en 'a': 10
Media: 30.0
```

Las Series admiten operaciones vectorizadas, de manera similar a NumPy, lo que permite realizar cálculos eficientes sobre conjuntos completos de datos.

DataFrames: la estructura bidimensional

El DataFrame es la estructura principal de Pandas. Es una tabla bidimensional con filas y columnas etiquetadas, similar a una hoja de cálculo o una tabla de base de datos.

Énfasis

Un DataFrame es como una tabla donde cada columna es una Serie. Todas las columnas comparten el mismo índice de filas, pero cada columna puede tener un tipo de dato diferente.

Podemos crear un DataFrame desde un diccionario donde las claves son nombres de columnas:

```
datos = { 'nombre': ['Ana', 'Carlos', 'Beatriz', 'Diego'], 'edad': [22, 25, 23, 24], 'ciudad': ['Buenos Aires', 'Córdoba', 'Rosario', 'Mendoza'] }
df = pd.DataFrame(datos)
print(df)
```

Esto crea una tabla con tres columnas y cuatro filas. Pandas automáticamente genera un índice numérico para las filas.



Fuente: Elaboración propia.

Podemos ver las primeras filas con head():

```
print(df.head())
```

Por defecto muestra las primeras 5 filas. Podemos especificar cuántas:

```
print(df.head(2))
```

Para ver las últimas filas:

```
print(df.tail(3))
```

Para obtener información general del DataFrame. Esto muestra los tipos de datos de cada columna, cuántos valores no nulos hay, y el uso de memoria:

```
print(df.info())
```

Mientras que para ver estadísticas descriptivas de las columnas numéricas:

```
print(df.describe())
```

Esto calcula automáticamente la media, la desviación estándar, el valor mínimo, el máximo y los cuartiles. Los cuartiles dividen los datos en cuatro partes iguales: el veinticinco por ciento corresponde al valor por debajo del cual se encuentra el veinticinco por ciento de los datos; el cincuenta por ciento es la mediana; y el setenta y cinco por ciento corresponde al valor por debajo del cual se encuentra el setenta y cinco por ciento de los datos.

Acceso y manipulación de datos

Para acceder a columnas y filas, podemos utilizar las siguientes expresiones:

```
print(df['nombre'])
print(df[['nombre', 'edad']])
print(df.iloc[0])
print(df.loc[0, 'edad'])
```

Una operación muy común es filtrar filas según determinadas condiciones. Pandas permite hacer esto de forma muy elegante:

```
mayores_23 = df[df['edad'] > 23]
print(mayores_23)
```

La expresión df['edad'] > 23 genera una Serie de valores booleanos. Cuando usamos esa Serie para indexar el DataFrame, obtenemos solo las filas en las que la condición se cumple, es decir, donde el valor es True.

También es posible combinar múltiples condiciones:

```
filtrado = df[(df['edad'] > 22) & (df['ciudad'] == 'Buenos Aires')]
print(filtrado)
```

En estos casos, usamos & para "y" lógico y | para "o" lógico. Los paréntesis son importantes para que las condiciones se evalúen correctamente. Para agregar, modificar y eliminar datos:

```
df['pais'] = 'Argentina'
df['edad_doble'] = df['edad'] * 2
df.loc[0, 'edad'] = 23
df_ordenado = df.sort_values('edad')
df_sin_col = df.drop('pais', axis=1)
```

Manejo de valores faltantes

Los datos reales frecuentemente tienen valores faltantes. En Pandas, estos se representan como NaN (Not a Number), un valor especial que indica la ausencia de dato. Pandas ofrece herramientas para detectar y manejar estos valores:

```
datos_con_faltantes = { 'nombre': ['Ana', 'Carlos', 'Beatriz', 'Diego'], 'edad': [22, np.nan, 23, 24], 'salario': [50000, 60000, np.nan, 55000] }
df = pd.DataFrame(datos_con_faltantes)
print(df.isnull().sum())
df_limpio = df.dropna()
df_rellenado = df.fillna(df.mean())
```



Énfasis

El manejo correcto de valores faltantes es crucial en ciencia de datos. La estrategia depende del contexto del problema y del tipo de información que se esté analizando.

Lectura y escritura de archivos

Pandas permite leer y escribir datos en múltiples formatos de archivo:

```
df = pd.read_csv('datos.csv')
df = pd.read_excel('datos.xlsx')
df.to_csv('salida.csv', index=False)
df.to_excel('salida.xlsx', index=False)
```

Agrupación y agregación

El método *Groupby* permite agrupar datos y calcular estadísticas sobre esos grupos:

```
datos = { 'ciudad': ['Buenos Aires', 'Córdoba', 'Buenos Aires', 'Córdoba'], 'salario': [50000, 60000, 55000, 58000] }
df = pd.DataFrame(datos)
print(df.groupby('ciudad')['salario'].mean())
print(df.groupby('ciudad')['salario'].agg(['mean', 'max', 'min']))
```

Combinemos lo aprendido analizando un dataset de películas similar a los usados en sistemas de recomendación como Netflix:

```
datos_peliculas = { 'titulo': ['Matrix', 'Titanic', 'Avatar', 'Inception', 'Toy Story'], 'genero': ['Sci-Fi', 'Drama', 'Sci-Fi', 'Sci-Fi', 'Animacion'], 'año': [1999, 1997, 2009, 2010, 1995], 'rating_imdb': [8.7, 7.9, 7.8, 8.8, 8.3], 'recaudacion_millones': [467, 2187, 2847, 829, 373] }

df = pd.DataFrame(datos_peliculas)

print("Estadísticas descriptivas:")
print(df[['rating_imdb', 'recaudacion_millones']].describe())

print("\nPelículas por género:")

por_genero = df.groupby('genero').agg({ 'rating_imdb': 'mean', 'recaudacion_millones': 'sum' })

print(por_genero)
```

Este tipo de análisis exploratorio es fundamental antes de construir modelos predictivos, ya que nos ayuda a entender los datos y detectar patrones.

Transformación y limpieza de datos

Existen técnicas esenciales para preparar datos que luego podrán ser utilizado por modelos de machine learning:

```
df = pd.DataFrame({ 'nombre': [' Ana ', 'CARLOS'], 'edad': ['22', '25'] })
df['nombre'] = df['nombre'].str.strip().str.title()
df['edad'] = df['edad'].astype(int)
df_sin_duplicados = df.drop_duplicates()
```

Para el manejo de fechas:

```
df['fecha'] = pd.to_datetime(df['fecha_string'])
df['año'] = df['fecha'].dt.year
```

Para combinar DataFrames:

```
combinado = pd.merge(clientes, compras, on='cliente_id')
```

Preparando datos para machine learning

Los algoritmos de machine learning solo trabajan con números, no con categorías de texto. Para convertir variables categóricas en números, usamos get_dummies, que crea una columna binaria por cada categoría. Por ejemplo, si tenemos una columna género con valores 'pop' y 'rock', get_dummies crea dos columnas: genero_pop y genero_rock. Si una canción es de rock, tendrá 1 en genero_rock y 0 en genero_pop. Esto se llama codificación one-hot:

```
datos_ejemplo = { 'genero': ['pop', 'rock', 'pop', 'rock'] }

df = pd.DataFrame(datos_ejemplo)
genero_dummies = pd.get_dummies(df['genero'], prefix='genero')
print(genero_dummies)
df = pd.concat([df, genero_dummies], axis=1)
df = df.drop('genero', axis=1)
print("\nDataset listo para machine learning:")
print(df)
```

Este proceso de preparación de datos es fundamental antes de entrenar modelos. Incluye manejar valores faltantes, crear características derivadas y convertir variables categóricas a numéricas.



En este momento, está en condiciones de realizar la **actividad 1** del módulo.

Con NumPy y Pandas vistos, tenemos la base fundamental para trabajar con datos en Python. Estas dos bibliotecas son el cimiento sobre el cual se construye todo el ecosistema de ciencia de datos e inteligencia artificial. En las próximas secciones, veremos cómo usar estos datos preparados para entrenar modelos con Scikit-learn y construir redes neuronales con TensorFlow y PyTorch.

Scikit-learn: aprendizaje automático en Python

Scikit-learn es la biblioteca más popular para machine learning tradicional en Python. Proporciona implementaciones eficientes de los algoritmos más importantes de clasificación, regresión y clustering, junto con herramientas para preprocesamiento, selección de características y evaluación de modelos.


Énfasis

Scikit-learn destaca por su diseño consistente y elegante. Todos los modelos comparten la misma interfaz: fit() para entrenar, predict() para hacer predicciones y score() para evaluar. Esta uniformidad hace que sea fácil probar diferentes algoritmos cambiando solo una línea de código.

Antes de continuar, necesitamos entender qué es machine learning. El aprendizaje automático es un campo de la inteligencia artificial donde los sistemas aprenden patrones de los datos sin ser programados explícitamente para cada tarea. En lugar de escribir reglas manuales, alimentamos datos históricos a un algoritmo que encuentra patrones y puede hacer predicciones sobre datos nuevos.

Hay dos tipos principales de problemas en machine learning. La clasificación es cuando queremos predecir a qué categoría pertenece algo, como si un email es spam o legítimo, o si un cliente comprará o no. La regresión es cuando queremos predecir valores numéricos continuos, como el precio de una casa o la cantidad de ventas.

Importamos scikit-learn en partes según lo que necesitemos:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
```

El flujo típico de trabajo en machine learning incluye la preparación de los datos, su división en conjuntos de entrenamiento y prueba, el entrenamiento de un modelo, la generación de predicciones y la evaluación de los resultados. Scikit-learn facilita cada uno de estos pasos.

Antes de entrenar modelos, debemos preprocesar los datos. Ya vimos en NumPy cómo normalizar datos manualmente, pero Scikit-learn nos ofrece herramientas especializadas que además recuerdan la transformación para aplicarla consistentemente a datos nuevos.

El StandardScaler es fundamental cuando nuestras características tienen escalas muy diferentes. Imaginemos que estamos prediciendo si un cliente comprará un producto usando dos características: su edad en años y su ingreso anual en miles de pesos. La edad va de dieciocho a setenta, mientras que el ingreso va de cincuenta a quinientos. Sin nor-

malizar, el modelo podría pensar que el ingreso es más importante simplemente porque los números son más grandes. Veamos cómo funciona:

```
from sklearn.preprocessing import StandardScaler
import numpy as np
datos = np.array([[25, 80000], [35, 120000], [45, 95000], [55, 150000]])
print("Datos originales:")
print(datos)
print(f"\nMedia por columna: {datos.mean(axis=0)}")
print(f"Desv. estándar por columna: {datos.std(axis=0)}")
```

Aquí axis=0 indica que queremos calcular la media por columna, colapsando las filas. Es decir, calculamos la media de todas las edades y la media de todos los ingresos por separado.

```
scaler = StandardScaler()
datos_normalizados = scaler.fit_transform(datos)
print("\nDatos normalizados:"(datos_normalizados)
print(f"\nMedia por columna: {datos_normalizados.mean(axis=0)}")
print(f"Desv. estándar por columna: {datos_normalizados.std(axis=0)}")
```

El StandardScaler aplica la transformación que ya conocemos, restando la media y dividiendo por la desviación estándar, pero lo hace columna por columna. La ventaja de usar StandardScaler en lugar de hacerlo manualmente es que el objeto scaler guarda las medias y desviaciones calculadas durante fit_transform. Luego podemos aplicar exactamente la misma transformación a datos nuevos usando solo transform, garantizando consistencia entre entrenamiento y predicción.

Muchas veces necesitamos convertir variables categóricas en números. Por ejemplo, si tenemos una columna con valores de texto como bajo, medio o alto, necesitamos convertirlos a números. Hay dos formas principales de hacerlo dependiendo de si las categorías tienen un orden natural o no.

Para categorías con orden natural como bajo, medio o alto, usamos LabelEncoder:

```
from sklearn.preprocessing
import LabelEncoder

categorias = ['bajo', 'medio', 'alto', 'bajo', 'alto']
encoder = LabelEncoder()
categorias_codificadas = encoder.fit_transform(categorias)
print(categorias_codificadas)
```

Esto convierte las categorías en números: alto se vuelve cero, bajo se vuelve uno y medio se vuelve dos. El problema es que esto implica un orden: el algoritmo podría pensar que dos es el doble de uno, lo cual no siempre tiene sentido.

Por otra parte, para variables categóricas sin orden natural, como colores o géneros musicales, usamos one-hot encoding. Este método crea una columna separada para cada categoría posible, asignando uno en la columna correspondiente y cero en las demás:

```
from sklearn.preprocessing import OneHotEncoder

colores = [['rojo'], ['azul'], ['verde'], ['rojo'], ['azul']]
encoder = OneHotEncoder(sparse_output=False)
colores_codificados = encoder.fit_transform(colores)
print(colores_codificados)
print("\nColumnas: azul, rojo, verde")
```

Si el color es rojo, el resultado será [0, 1, 0], lo que indica cero para azul, uno para rojo y cero para verde. De este modo, se evita que el algoritmo asuma relaciones numéricas inexistentes entre las categorías.

División de datos en entrenamiento y prueba

Un error común en machine learning es entrenar y evaluar un modelo con los mismos datos. Si hacemos esto, el modelo puede simplemente memorizar los datos en lugar de aprender patrones generales, obteniendo resultados perfectos en datos conocidos, pero fallando con datos nuevos. Es como estudiar solo los ejemplos del examen anterior en lugar de entender realmente la materia.



Para evaluar modelos correctamente, separamos los datos en dos conjuntos: entrenamiento y prueba. Entrenamos el modelo solo con el conjunto de entrenamiento y luego lo evaluamos con el conjunto de prueba que el modelo nunca vio. Esto nos da una medida realista de qué tan bien funcionará con datos nuevos.

```
from sklearn.model_selection import train_test_split
X = np.array([[1, 2], [2, 3], [3, 4], [4, 5], [5, 6], [6, 7], [7, 8], [8, 9]]) y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
print(f"Entrenamiento: {len(X_train)} muestras")
print(f"Prueba: {len(X_test)} muestras")
```

El parámetro test_size indica qué porcentaje de los datos reservar para prueba. En este ejemplo se utiliza 0.25, lo que significa que el veinticinco por ciento de los datos se destina a prueba y el setenta y cinco por ciento restante a entrenamiento. El parámetro random_state fija la semilla aleatoria, asegurando que la división sea reproducible: cada vez que ejecutemos el código con el mismo random_state, obtendremos la misma división.

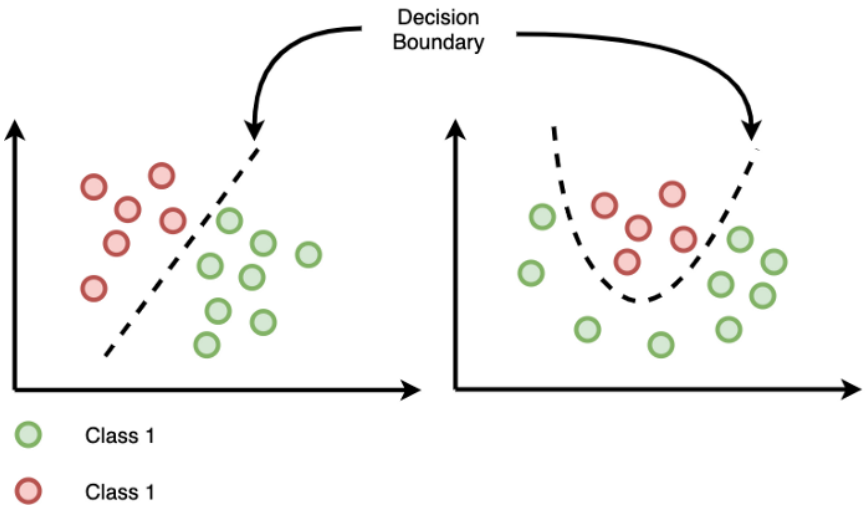
Clasificación: detector de spam con regresión logística

La clasificación es una tarea del machine learning cuyo objetivo es predecir a qué categoría pertenece un elemento. Algunos ejemplos habituales son determinar si un correo electrónico es spam o legítimo, si un cliente realizará una compra o no, o si una imagen contiene un perro o un gato. Cuando existen únicamente dos categorías posibles, se habla de clasificación binaria.

La regresión logística, a pesar de lo que su nombre podría sugerir, no se utiliza para predecir valores continuos, sino que es uno de los algoritmos más empleados para problemas de clasificación binaria.

Énfasis

Este algoritmo aprende a calcular la probabilidad de que una observación pertenezca a una clase u otra a partir de sus características, y luego toma una decisión en función de esa probabilidad (por ejemplo, clasificar un correo como spam o no spam).



Fuente: https://www.themachinelearners.com/regresion-logistica-regularizacion/

Construyamos un detector de spam paso a paso:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

#Creamos un dataset simulado con características típicas de emails
datos_emails = { 'cant_mayusculas': [45, 2, 38, 5, 3, 50, 8, 42, 6, 35], 'cant_signos_exclamacion': [8, 0, 6, 1, 0, 10, 1, 7, 0, 5], 'cant_urls': [5, 0, 4, 0, 1, 6, 0, 5, 0, 3], 'longitud': [1500, 300, 1200, 450, 350, 1600, 400, 1300, 380, 1100], 'es_spam': [1, 0, 1, 0, 0, 1, 0, 1, 0, 1] #1 = spam, 0 = legítimo }
df = pd.DataFrame(datos_emails)
#Separamos características (X) de la etiqueta objetivo (y)
X = df.drop('es_spam', axis=1)

#Todas las columnas excepto la etiqueta
y = df['es_spam'] #Solo la columna que queremos predecir

#Dividimos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

#Creamos el modelo de regresión logística
modelo = LogisticRegression()
```

```
#Entrenamos el modelo con los datos de entrenamiento
#El modelo aprende qué características indican spam
modelo.fit(X_train, y_train)

#Hacemos predicciones en el conjunto de prueba
predicciones = modelo.predict(X_test)

#Evaluamos qué tan bien funcionó
precision = accuracy_score(y_test, predicciones)
print(f"Precisión del detector: {precision:.2f}")

#El classification_report muestra métricas detalladas por clase
print("\nReporte completo:")
print(classification_report(y_test, predicciones, target_names=['Legítimo', 'Spam']))

#Probamos con un email nuevo nunca visto
nuevo_email = [[40, 7, 4, 1400]] #40 mayúsculas, 7 exclamaciones, 4 URLs, 1400 caracteres

#predict() da la clase predicha (0 o 1)
prediccion = modelo.predict(nuevo_email)

#predict_proba() da las probabilidades para cada clase
probabilidad = modelo.predict_proba(nuevo_email)
print(f"\nEmail nuevo clasificado como: {'SPAM' if prediccion[0] == 1 else 'Legítimo'}")
print(f"Confianza–Legítimo: {probabilidad[0][0]:.2%}, Spam: {probabilidad[0][1]:.2%}")
```

A partir de los datos de entrenamiento, el modelo aprendió que emails con muchas mayúsculas, abundancia de signos de exclamación o presencia de múltiples URLs presentan una mayor probabilidad de ser spam. Cuando recibe un email nuevo, utiliza estos patrones aprendidos para calcular la probabilidad de que pertenezca a la categoría "spam" o "no spam". Este tipo de enfoque es el que se encuentra en la base de los filtros antispam utilizados por servicios como Gmail, Outlook y otras plataformas de correo electrónico.

Clasificación con árboles de decisión

Los árboles de decisión funcionan diferente. En lugar de trazar un límite lineal, hacen una serie de preguntas tipo "sí/no" sobre las características, como un diagrama de flujo.



Un árbol de decisión construye reglas del tipo "si el cliente tiene menos de seis meses de antigüedad y llamó al soporte más de cinco veces, entonces abandonará". Para ello, divide los datos en subgrupos cada vez más homogéneos a partir de una secuencia de preguntas. Este proceso continúa hasta que cada hoja del árbol contiene mayoritariamente ejemplos de una misma clase.

La ventaja es que son muy interpretables y podemos ver exactamente qué reglas usa para decidir. Usémoslos para predecir abandono de clientes:

```
from sklearn.tree import DecisionTreeClassifier

#Dataset de clientes con historial de abandono
datos_clientes = { 'antiguedad_meses': [3, 24, 6, 48, 12, 36, 2, 60, 18, 8], 'llamadas_soporte': [8, 1, 5, 0, 2, 1, 10, 0, 3, 7], 'gasto_mensual': [25, 80, 40, 95, 60, 85, 20, 100, 70, 30], 'problemas_tecnicos': [5, 0, 3, 0, 1, 0, 6, 0, 1, 4], 'abandono': [1, 0, 1, 0, 0, 0, 1, 0, 0, 1] #1 = abandonó, 0 = permanece }

df = pd.DataFrame(datos_clientes)
X = df.drop('abandono', axis=1)
y = df['abandono']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

#Creamos un árbol con profundidad máxima de 4 niveles
#Esto evita que el árbol sea demasiado complejo y memorice los datos
modelo = DecisionTreeClassifier(max_depth=4, random_state=42)
modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test) precision = accuracy_score(y_test, predicciones)
print(f"Precisión en predicción de abandono: {precision:.2f}")
print("\nReporte de clasificación:")
print(classification_report(y_test, predicciones, target_names=['Permanece', 'Abandona']))

#Los árboles nos muestran qué características son más importantes para decidir
importancias = pd.DataFrame({'caracteristica': X.columns, 'importancia': modelo.feature_importances_ #Valores entre 0 y 1 }).sort_values('importancia', ascending=False)
print("\nCaracterísticas más importantes para predecir abandono:")
print(importancias)
```

Las importancias indican qué características utilizó con mayor frecuencia el árbol para tomar decisiones. Una importancia de 0.5 significa que esa característica es responsable del cincuenta por ciento de las decisiones correctas. Este tipo de modelo ayuda a empresas como Netflix o Spotify a identificar usuarios en riesgo y ofrecerles incentivos para que permanezcan.

Regresión: predicción de valores numéricos

La regresión, por su parte, es una tarea diferente a la clasificación. En lugar de predecir categorías, predecimos valores numéricos continuos como precios, temperaturas, cantidad de likes o ingresos. La regresión lineal es uno de los algoritmos más simples y ampliamente usados.

La regresión lineal busca la mejor línea recta (o plano en múltiples dimensiones) que se ajuste a los datos. Es como encontrar la ecuación matemática que mejor describe la relación entre las características y el valor que queremos predecir. Por ejemplo, si queremos predecir el precio de una casa basándonos en su superficie y cantidad de habitaciones,

Énfasis

buscamos una fórmula: precio = a × superficie + b × habitaciones + c, donde a, b y c son números que el algoritmo aprende de los datos.

Usémosla para predecir engagement en publicaciones de redes sociales:

```
from sklearn.linear_model import LinearRegression
import pandas as pd
from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error, r2_score

#Dataset de posts con sus características y likes obtenidos
datos_posts = { 'hora_publicacion': [9, 14, 20, 8, 18, 12, 21, 10, 15, 19], #Hora del día (0-23)

        'cant_hashtags': [5, 3, 8, 2, 6, 4, 10, 3, 5, 7], 'tiene_imagen': [1, 1, 1, 0, 1, 1, 1, 0, 1, 1], #1 =
        sí, 0 = no
        'longitud_texto': [150, 80, 200, 50, 180, 120, 250, 60, 140, 190], #Caracteres
        'likes': [850, 420, 1200, 180, 950, 650, 1500, 250, 780, 1100] #Variable objetivo
}

df = pd.DataFrame(datos_posts)
X = df.drop('likes', axis=1) #Características predictoras
y = df['likes'] #Variable que queremos predecir
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

#Creamos y entrenamos el modelo de regresión lineal
modelo = LinearRegression()
modelo.fit(X_train, y_train)

#El modelo aprende los coeficientes de la ecuación

#Hacemos predicciones
predicciones = modelo.predict(X_test)

#Evaluamos el modelo con métricas para regresión
mse = mean_squared_error(y_test, predicciones)
r2 = r2_score(y_test, predicciones)

print(f"Error cuadrático medio (MSE): {mse:.2f}")
print(f"R² score: {r2:.2f}")
```

```
#El MSE penaliza errores grandes: si predecimos 500 likes pero fueron 1000, el error es (1000-500)² = 250,000
#El R² indica qué tan bien el modelo explica la variabilidad: 1.0 es perfecto, 0.0 es malo

print("\nPredicciones vs Realidad:")

for real, pred in zip(y_test, predicciones):
print(f"Real: {real:,.0f} likes–Predicción: {pred:,.0f} likes–Error: {abs(real-pred):,.0f}")

#Probamos con un post nuevo

nuevo_post = [[18, 6, 1, 170]]
#18hs, 6 hashtags, con imagen, 170 caracteres

likes_estimados = modelo.predict(nuevo_post)
print(f"\nPost nuevo: se estiman {likes_estimados[0]:,.0f} likes")

#Podemos ver cuánto impacta cada característica en los likes
coeficientes = pd.DataFrame({'caracteristica': X.columns,
    'impacto': modelo.coef_ #Cuánto cambian los likes por cada unidad de la característica
    }).sort_values('impacto', ascending=False)

print("\nImpacto de cada característica en los likes:")
print(coeficientes)
print(f"\nIntercepto (likes base): {modelo.intercept_:.2f}")
```

El impacto refleja el valor del coeficiente en la ecuación del modelo. Si la variable tiene_imagen tiene un impacto de 200, significa que incluir una imagen incrementa, en promedio, la cantidad de likes en 200 unidades, manteniendo constantes las demás variables. Si cant_hashtags tiene un impacto de 50, cada hashtag adicional suma, en promedio, 50 likes. Este tipo de modelos resulta útil para influencers y marcas, ya que permite identificar qué decisiones de contenido tienen mayor efecto y optimizar sus publicaciones de manera informada.

Criterios para elegir un algoritmo

Una vez preparados los datos y definido un esquema de entrenamiento y evaluación, resulta habitual preguntarse qué algoritmo ofrece mejores resultados para un mismo problema. En este punto, Scikit-learn presenta una ventaja clave: su interfaz consistente. Todos los modelos utilizan los mismos métodos —fit() para entrenar y predict() para predecir—, lo que permite probar y comparar diferentes algoritmos fácilmente:

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

#Generamos datos sintéticos para clasificación #n_samples: cantidad de ejemplos, n_features: cantidad de características

X, y = make_classification(n_samples=200, n_features=10, n_informative=8, n_redundant=2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Normalizamos los datos
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) #Aprende media y desv, luego transforma
X_test_scaled = scaler.transform(X_test) #Solo transforma, usando los mismos parámetros

#Diccionario con diferentes algoritmos #Random Forest combina múltiples árboles de decisión para mejorar precisión
modelos = { 'Regresión Logística': LogisticRegression(random_state=42),
        'Árbol de Decisión': DecisionTreeClassifier(random_state=42),
        'Random Forest': RandomForestClassifier(random_state=42) }

print("Comparación de algoritmos:\n")

#Probamos cada algoritmo con el mismo procedimiento
for nombre, modelo in modelos.items():
    modelo.fit(X_train_scaled, y_train) #Entrenamos el modelo
    y_pred = modelo.predict(X_test_scaled) #Hacemos predicciones
    precision = accuracy_score(y_test, y_pred) #Calculamos precisión
    print(f"{nombre}: Precisión = {precision:.4f}")
```

Con solo cambiar el nombre del modelo, podemos probar algoritmos completamente diferentes. Esto facilita experimentar y encontrar el mejor para nuestro problema.



En este momento, está en condiciones de realizar la **actividad 2** del módulo.

TensorFlow y Keras: redes neuronales

TensorFlow es la biblioteca de deep learning más popular desarrollada por Google. Keras es su interfaz de alto nivel que simplifica la creación de redes neuronales complejas con pocas líneas de código.



Las redes neuronales son modelos inspirados en el cerebro humano, compuestos por capas de neuronas artificiales. Cada neurona recibe entradas, las procesa mediante una función de activación, y produce una salida. El aprendizaje ocurre ajustando los pesos de las conexiones entre neuronas.

Para comenzar a trabajar con estas herramientas, importamos TensorFlow y los módulos principales de Keras:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
print(f"TensorFlow versión: {tf.version}")
```

Conceptos básicos de redes neuronales

Antes de construir nuestra primera red, entendamos los componentes fundamentales. Una red neuronal está formada por capas de neuronas. Cada capa recibe datos, los procesa y los pasa a la siguiente capa. Las capas Dense (también llamadas fully connected o densas) conectan cada neurona con todas las neuronas de la capa anterior y constituyen el tipo de capa más utilizado.

Las funciones de activación determinan cómo las neuronas transforman sus entradas. La más común es ReLU (Rectified Linear Unit), que simplemente convierte valores negativos en cero y deja los positivos sin cambiar. En la capa final de un problema de clasificación binaria se utiliza sigmoid, ya que transforma cualquier valor numérico en un número entre cero y uno, interpretable como una probabilidad.

Durante el entrenamiento, la red requiere un optimizador y una función de pérdida. El optimizador, como adam, ajusta los pesos de la red de manera eficiente, mientras que la función de pérdida, como binary_crossentropy en clasificación binaria, mide la distancia entre las predicciones del modelo y los valores reales.

Construyendo nuestra primera red neuronal: clasificador de reseñas

Ahora sí, creemos una red neuronal para clasificar reseñas como positivas o negativas. Simulamos características extraídas de texto:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

np.random.seed(42)
n_muestras = 1000

palabras_positivas = np.random.randint(0, 50, n_muestras)
palabras_negativas = np.random.randint(0, 50, n_muestras)
longitud = np.random.randint(50, 500, n_muestras)
puntuacion_sentiment = np.random.uniform(-1, 1, n_muestras)
X = np.column_stack([palabras_positivas, palabras_negativas, longitud, puntuacion_sentiment])

y = ((palabras_positivas > palabras_negativas) & (puntuacion_sentiment > 0)).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

modelo = Sequential([ Dense(64, activation='relu', input_shape=(4,)),
        Dense(32, activation='relu'),
        Dense(16, activation='relu'),
        Dense(1, activation='sigmoid') ])

modelo.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

print("Arquitectura del clasificador de sentimientos:")

print(modelo.summary())
```

El summary nos muestra la estructura completa de la red: cuántas capas tiene, cuántas neuronas hay en cada capa y cuántos parámetros entrenables tiene el modelo. A partir de esta estructura, se procede al entrenamiento. Un epoch es una pasada completa por todos los datos de entrenamiento. Cincuenta epochs significa que la red verá todos los datos cincuenta veces, ajustando sus pesos en cada pasada. El batch_size indica cuántos ejemplos procesar antes de actualizar los pesos: treinta y dos es un valor común que balancea velocidad y estabilidad. El validation_split reserva parte del entrenamiento para validación, permitiendo ver cómo generaliza el modelo durante el entrenamiento:

```
historia = modelo.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=0)

loss, accuracy = modelo.evaluate(X_test, y_test, verbose=0)

print(f"\nPrecisión en clasificación de reseñas: {accuracy:.4f}")

print(f"Pérdida: {loss:.4f}")

nueva_reseña = scaler.transform([[35, 5, 280, 0.7]])
prediccion = modelo.predict(nueva_reseña, verbose=0)

print(f"\nNueva reseña clasificada como: {'Positiva' if prediccion[0][0] > 0.5 else 'Negativa'}")
print(f"Confianza: {prediccion[0][0]:.2%}")
```

Este tipo de modelo es la base del análisis de sentimientos utilizado en redes sociales, reseñas de productos y sistemas de feedback de usuarios.

Clasificación multiclase con redes neuronales

Para clasificar en múltiples categorías, necesitamos convertir las etiquetas a formato one-hot usando to_categorical. Si tenemos tres clases (0, 1, 2), esto convierte 0 en [1, 0, 0], 1 en [0, 1, 0], y 2 en [0, 0, 1]. En este caso, la capa de salida incluye una neurona por cada clase y se emplea la función de activación softmax, que transforma las salidas del modelo en probabilidades cuya suma es igual a uno:

```
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
y_train_cat = to_categorical(y_train, num_classes=3)
y_test_cat = to_categorical(y_test, num_classes=3)
modelo = Sequential([ Dense(64, activation='relu', input_shape=(4,)),
        Dense(32, activation='relu'),
        Dense(3, activation='softmax') ])

modelo.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
historia = modelo.fit(X_train, y_train_cat, epochs=50, batch_size=32, validation_split=0.2, verbose=0)
loss, accuracy = modelo.evaluate(X_test, y_test_cat, verbose=0)
print(f"Precisión: {accuracy:.4f}")
```

Regresión con redes neuronales

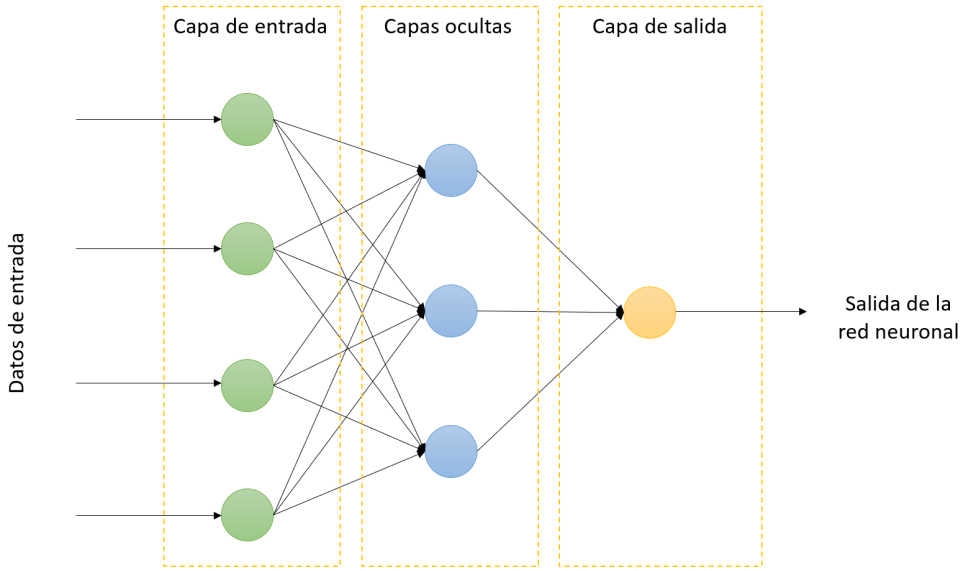
Cuando el objetivo es predecir valores continuos, como precios, cantidades o magnitudes numéricas, se utiliza una red neuronal de regresión. En este tipo de problemas, la capa de salida tiene una sola neurona y no se aplica ninguna función de activación, ya que el modelo debe producir un valor numérico libre, sin estar acotado a un rango específico.

Para entrenar el modelo se emplea la función de pérdida MSE (mean squared error o error cuadrático medio), que mide cuánto se alejan, en promedio, las predicciones de los valores reales elevando las diferencias al cuadrado. Además, se suele monitorear MAE (mean absolute error o error absoluto medio), que indica el error promedio en las mismas unidades que los datos originales, lo que facilita su interpretación.

En el siguiente ejemplo se construye un modelo con capas densas:

```
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

modelo = Sequential([ Dense(64, activation='relu', input_shape=(5,)),
        Dense(32, activation='relu'),
        Dense(1) ])
modelo.compile(optimizer='adam', loss='mse', metrics=['mae'])
historia = modelo.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.2, verbose=0)
loss, mae = modelo.evaluate(X_test, y_test, verbose=0)
print(f"Error absoluto medio: {mae:.0f} visualizaciones")
```



Fuente: https://interactivechaos.com/es/manual/tutorial-de-machine-learning/estructura-de-una-red-neuronal

PyTorch: construcción flexible de modelos

PyTorch es otra biblioteca popular de deep learning que ofrece mayor control y flexibilidad en la construcción de modelos.



PyTorch usa tensores similares a NumPy pero optimizados para deep learning. Es muy utilizado en el ámbito de la investigación justamente por esa flexibilidad y control que brinda al programador.

Importamos los módulos principales de PyTorch:

```
import torch
import torch.nn as nn
import torch.optim as optim

tensor = torch.tensor([1, 2, 3, 4, 5])
matriz = torch.tensor([[1, 2, 3], [4, 5, 6]])

print(tensor + 10)

print(torch.mm(matriz.T, matriz))
```

En PyTorch definimos redes como clases de Python que heredan de nn.Module. Esto nos da más control que Keras. El método **init** define las capas que tendrá la red, y el método forward define cómo los datos fluyen a través de ellas:

```
class RedSimple(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(RedSimple, self).__init__()
        self.capa1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.capa_salida = nn.Linear(hidden_size, output_size)
        self.sigmoid = nn.Sigmoid()

def forward(self, x):
        x = self.capa1(x)
        x = self.relu(x)
        x = self.capa_salida(x)
        x = self.sigmoid(x)
        return x
```

El entrenamiento en PyTorch es más explícito que en Keras. Definimos la función de pérdida y el optimizador, y luego ejecutamos un ciclo manual donde calculamos pérdida, gradientes y actualizamos pesos. Este enfoque requiere más código, pero ofrece un mayor control sobre cada paso del aprendizaje.



En este momento, está en condiciones de realizar la **actividad 3** del módulo.

Comparando TensorFlow/Keras y PyTorch

Ambas bibliotecas son herramientas sólidas y ampliamente utilizadas en deep learning.

Keras se caracteriza por su simplicidad y rapidez para prototipar, gracias a una API de alto nivel que abstrae muchos detalles internos. PyTorch, en cambio, ofrece mayor control y flexibilidad, con un estilo de programación más cercano a Python puro. Para comenzar, recomendamos empezar con Keras por su facilidad de uso y, más adelante, explorar PyTorch cuando se requiera un control más fino sobre el modelo y el proceso de entrenamiento.

Con este recorrido se completó el abordaje de las bibliotecas fundamentales de la inteligencia artificial. NumPy proporcionó la base para el cálculo numérico eficiente. Pandas permitió manipular y limpiar datos tabulares. Scikit-learn ofreció algoritmos de machine learning tradicional listos para usar. TensorFlow/Keras y PyTorch abrieron el camino hacia el deep learning y las redes neuronales.

Cada una de estas bibliotecas es una herramienta poderosa, pero su verdadero poder emerge cuando las combinamos. Habitualmente se utiliza Pandas para la limpieza de datos, NumPy para el preprocesamiento y Scikit-learn o TensorFlow para la construcción de modelos. Este flujo de trabajo constituye un estándar en la industria de la inteligencia artificial.

Todo lo aprendido en los módulos anteriores cobra sentido aquí. Las variables y funciones del módulo uno, las estructuras de datos del módulo dos y la programación orientada a objetos del módulo tres se integran constantemente en el uso de estas bibliotecas. Python se consolidó como el lenguaje de IA precisamente por este ecosistema rico y coherente.

El camino no termina aquí. Estas bibliotecas tienen muchísimas más funcionalidades que explorar. La documentación oficial de cada una es excelente y está llena de ejemplos. Los invitamos a experimentar, construir proyectos propios y seguir aprendiendo. La inteligencia artificial es un campo en constante evolución, y ahora tienen las herramientas fundamentales para ser parte de esa evolución.



Evaluación

Usted ya está en condiciones de realizar la **segunda parte** de la **evaluación integradora.**

Palabras finales

Con este módulo cerramos Programación para Inteligencia Artificial. Han recorrido un camino largo desde el primer "Hola Mundo" hasta construir redes neuronales. Cada concepto aprendido es un ladrillo en los cimientos de su carrera en IA. Ahora están preparados/as para las materias siguientes donde aplicarán estos conocimientos en proyectos cada vez más complejos y emocionantes.

¡Felicitaciones por llegar hasta aquí y mucho éxito en su camino como desarrolladores/as de inteligencia artificial!

A modo de cierre los invitamos a ver el siguiente video:



Módulo 4 | Glosario

Array: estructura de datos de NumPy que almacena elementos del mismo tipo de forma contigua en memoria. Permite operaciones matemáticas vectorizadas mucho más rápidas que las listas de Python.

Batch: grupo de ejemplos que se procesan juntos durante una iteración del entrenamiento de una red neuronal, balanceando eficiencia computacional y estabilidad del aprendizaje.

Clasificación: tarea de machine learning donde el objetivo es predecir a qué categoría pertenece una instancia. Cuando solo hay dos categorías posibles se llama clasificación binaria.

Conjunto de prueba: porción de los datos que se reserva para evaluar el modelo final después del entrenamiento. El modelo nunca ve estos datos durante el entrenamiento.

DataFrame: estructura de datos bidimensional de Pandas que organiza información en filas y columnas etiquetadas, similar a una hoja de cálculo. Cada columna es una Series.

Dropout: técnica de regularización para redes neuronales que consiste en desactivar aleatoriamente un porcentaje de neuronas durante el entrenamiento para prevenir sobreajuste.

Entrenamiento: proceso mediante el cual un modelo de machine learning aprende patrones de los datos ajustando sus parámetros internos para minimizar el error entre predicciones y valores reales.

Época: ciclo completo durante el entrenamiento de una red neuronal donde el modelo procesa todos los ejemplos del conjunto de entrenamiento una vez.

Estandarización: técnica de preprocesamiento que transforma datos numéricos para que tengan media cero y desviación estándar uno, facilitando que los algoritmos de machine learning funcionen mejor.

Feature engineering: proceso de crear nuevas características derivadas a partir de las características originales para mejorar el desempeño del modelo.

Función de pérdida: medida que cuantifica qué tan lejos están las predicciones del modelo de los valores reales. Diferentes problemas requieren diferentes funciones de pérdida.

LabelEncoder: herramienta de Scikit-learn que convierte etiquetas categóricas de texto a números enteros. Útil cuando las categorías tienen un orden natural.

MAE (Mean Absolute Error): métrica de evaluación para regresión que calcula el promedio de las diferencias absolutas entre valores predichos y valores reales.

Matriz de confusión: tabla que muestra la cantidad de predicciones correctas e incorrectas organizadas por clase real versus clase predicha.

MSE (Mean Squared Error): métrica de evaluación para modelos de regresión que calcula el promedio de los cuadrados de las diferencias entre valores predichos y valores reales.

One-hot encoding: técnica de codificación que convierte variables categóricas en múltiples columnas binarias, una por cada categoría posible. Permite que los algoritmos trabajen con categorías sin asumir relaciones numéricas inexistentes.

Optimizador: algoritmo que ajusta los pesos de una red neuronal durante el entrenamiento para minimizar la función de pérdida. Adam es uno de los optimizadores más populares.

Precisión: métrica de evaluación que mide el porcentaje de predicciones correctas sobre el total de predicciones realizadas.

Regresión: tarea de machine learning donde el objetivo es predecir un valor numérico continuo como precios, temperaturas o ingresos.

ReLU: función de activación que devuelve cero para valores negativos y el mismo valor para valores positivos. Es la función de activación más común en capas ocultas de redes neuronales.

R-cuadrado: métrica de evaluación para regresión que indica qué proporción de la variabilidad en los datos objetivo es explicada por el modelo. Varía entre cero y uno.

Series: estructura de datos unidimensional de Pandas que combina un array de valores con un índice etiquetado. Es la base sobre la cual se construyen los DataFrames.

Sigmoid: función de activación que transforma cualquier valor numérico en un rango entre cero y uno. Se usa típicamente en la capa de salida para clasificación binaria.

Sobreajuste: problema que ocurre cuando un modelo aprende demasiado los detalles específicos del conjunto de entrenamiento en lugar de los patrones generales, obteniendo buen desempeño en entrenamiento, pero fallando con datos nuevos.

Softmax: función de activación usada en la capa de salida de redes neuronales para clasificación multiclase. Transforma las salidas en probabilidades que suman uno.

StandardScaler: transformador de Scikit-learn que estandariza características numéricas restando la media y dividiendo por la desviación estándar, guardando los parámetros para aplicarlos consistentemente a datos nuevos.

Train-test split: proceso de dividir un dataset en dos partes: una para entrenar el modelo y otra para evaluarlo. La función train_test_split de Scikit-learn realiza esta división aleatoriamente.

Tensor: estructura de datos multidimensional similar a los arrays de NumPy pero optimizada para operaciones de deep learning. Es la estructura fundamental sobre la cual trabajan TensorFlow y PyTorch.

Módulo 4 | Actividades

ACTIVIDAD 1

Análisis y preparación de datos con NumPy y Pandas

En esta primera actividad del módulo aplicaremos NumPy y Pandas para analizar, limpiar y transformar datos. El objetivo es dominar la manipulación de datos numéricos con arrays multidimensionales y datos tabulares con DataFrames, preparando datasets para su posterior uso en machine learning.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código Python completo y funcional, incluyendo capturas de pantalla de la ejecución de los programas mostrando diferentes casos de prueba para verificar que funcionan correctamente.

Recomendaciones

La preparación de datos es la etapa más importante en cualquier proyecto de ciencia de datos. Los modelos de machine learning solo funcionan bien si los datos están limpios, bien formateados y libres de inconsistencias. Preste especial atención a identificar y manejar valores faltantes, transformar tipos de datos correctamente y validar que las transformaciones producen resultados coherentes. Use operaciones vectorizadas de NumPy siempre que sea posible para mayor eficiencia.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos la lógica de cada implementación y verificar que las estructuras se comporten correctamente en diferentes escenarios.

Ejercicio 1: Sistema de análisis de ventas mensuales

Una empresa registra sus ventas diarias durante tres meses completos. Escriba un programa que use NumPy para crear una matriz donde cada fila represente un mes y cada columna un día, con valores de ventas generados aleatoriamente entre mil y diez mil pesos. El programa debe calcular el total de ventas de cada mes, identificar el día del mes con mejores ventas promedio considerando los tres meses y determinar en qué mes se vendió más en total. Use operaciones por eje de NumPy aplicando operaciones vectorizadas sobre la matriz completa.

Ejercicio 2: Preprocesamiento de datos para análisis

Un laboratorio tiene mediciones de tres variables experimentales con escalas completamente diferentes: temperatura en grados Celsius, presión en pascales, y humedad en porcentaje. Escriba un programa usando NumPy que genere cien mediciones simuladas y normalice estos datos usando dos métodos: transformación min-max que lleva todos los valores al rango entre cero y uno, y estandarización que transforma los datos para tener media cero y desviación uno. Después de cada normalización verifique que se cumplan las propiedades esperadas y compare ambos métodos mostrando cómo afectan los datos.

Ejercicio 3: Análisis de dataset de estudiantes

Una institución educativa tiene información de sus estudiantes. Escriba un programa usando Pandas que cree un DataFrame simulando datos de al menos cincuenta estudiantes con información de nombre, edad, carrera, promedio de calificaciones, cantidad de materias aprobadas y año de ingreso. El programa debe agrupar estudiantes por carrera y calcular el promedio de calificaciones de cada una, identificar qué carrera tiene estudiantes con mejor rendimiento académico y filtrar estudiantes con promedio superior a siete que califiquen para becas.

Ejercicio 4: Limpieza y transformación de datos inconsistentes

Una base de datos de productos de comercio electrónico tiene información con múltiples problemas de calidad. Escriba un programa que cree un DataFrame con datos de al menos cuarenta productos incluyendo nombre del producto, categoría, precio, cantidad en stock, calificación promedio de usuarios y fecha de lanzamiento. Introduzca deliberadamente problemas realistas: algunos precios como texto, nombres con espacios extra, categorías con mayúsculas y minúsculas inconsistentes, valores faltantes, registros duplicados y fechas en formato de texto. El programa debe identificar cada tipo de problema reportando cuántos casos hay, implementar un proceso de limpieza que corrija todos los problemas y mostrar el estado del dataset antes y después de la limpieza.

Ejercicio 5: Integración y análisis de múltiples fuentes de datos

Una startup necesita integrar información de clientes y sus compras que están en archivos separados. Escriba un programa usando Pandas que maneje dos DataFrames: uno con información de clientes incluyendo identificador único, nombre, ciudad, edad y fecha de registro, y otro con historial de compras incluyendo identificador del cliente, fecha de compra, monto gastado y categoría del producto. El programa debe combinar ambos DataFrames mediante el identificador de cliente, calcular el gasto total por cliente, identificar los clientes más valiosos y analizar qué ciudad genera más ingresos agrupando por ubicación.

Ejercicio 6: Transformación de datos para preparar un dataset

Un equipo de ciencia de datos recibió datos crudos que deben transformarse completamente antes de poder analizarlos. Escriba un programa que cree un DataFrame con información de propiedades inmobiliarias incluyendo dirección completa, tipo de propiedad escrito como texto, superficie en metros cuadrados, cantidad de ambientes, precio de venta y año de construcción. El programa debe convertir el tipo de propiedad en variables dummy usando get_dummies, calcular la antigüedad de cada propiedad restando el año de construcción del año actual, crear una nueva característica calculando el precio por metro cuadrado y generar un dataset final completamente numérico y limpio listo para ser usado en modelos de machine learning.

ACTIVIDAD 2

Construcción de modelos con Scikit-learn

En esta segunda actividad aplicaremos Scikit-learn para construir, entrenar y evaluar modelos de machine learning tradicional. El objetivo aquí es integrar la preparación de datos con NumPy y Pandas junto con algoritmos de clasificación y regresión para resolver problemas reales de predicción y análisis.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código Python completo y funcional, incluyendo capturas de pantalla de la ejecución mostrando diferentes casos de prueba.

Recomendaciones

El desarrollo de modelos de machine learning requiere iterar entre preparación de datos, entrenamiento y evaluación. No espere que el primer modelo sea perfecto. Experimente con diferentes algoritmos, pruebe distintas formas de preprocesar los datos y compare resultados objetivamente. La división correcta entre entrenamiento y prueba es fundamental para evaluar honestamente el rendimiento. Preste atención a qué características son más importantes para cada predicción y justifique técnicamente la selección del modelo final.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos la lógica de cada implementación y verificar que las estructuras se comporten correctamente en diferentes escenarios.

Ejercicio 1: Predictor de aprobación de préstamos personales

Un banco necesita automatizar la evaluación de solicitudes de préstamos personales. Escriba un programa que genere un dataset sintético con información de al menos doscientos solicitantes incluyendo edad, ingreso mensual, antigüedad laboral, monto del préstamo solicitado, historial crediticio representado con un puntaje, cantidad de préstamos previos y si el préstamo fue aprobado o rechazado. El programa debe preprocesar normalizando las características numéricas con StandardScaler, dividir los datos en entrenamiento y prueba, entrenar al menos tres modelos diferentes como regresión logística, árbol de decisión y random forest, evaluar cada modelo calculando precisión y reporte de clasificación y hacer predicciones sobre casos nuevos mostrando tanto la predicción como la probabilidad de aprobación.

Ejercicio 2: Sistema de clasificación de calidad de vinos

Una bodega quiere clasificar automáticamente la calidad de sus vinos basándose en propiedades químicas. Escriba un programa que genere datos sintéticos representando análisis de al menos ciento cincuenta vinos con características como acidez, contenido de azúcar residual, nivel de alcohol, pH o sulfatos, y una clasificación de calidad en tres categorías: baja, media o alta. El programa debe preprocesar normalizando las características y codificando la variable objetivo de texto a números usando LabelEncoder, entrenar múltiples clasificadores probando al menos regresión logística, árboles de decisión y random forest, evaluar cada modelo calculando precisión y reporte de clasificación, e identificar qué características químicas son más importantes para predecir calidad analizando los coeficientes o importancias del mejor modelo.

Ejercicio 3: Predictor de precios de automóviles usados

Una concesionaria de vehículos usados necesita estimar precios de mercado automáticamente. Escriba un programa que cree un dataset con datos de al menos cien automóviles incluyendo marca representada como texto, año de fabricación, kilometraje, tipo de combustible, cantidad de puertas, cilindrada del motor y precio de venta. El programa debe preprocesar codificando variables categóricas usando one-hot encoding y normalizando las variables numéricas, crear nuevas características derivadas como la antigüedad del vehículo, entrenar modelos de regresión como regresión lineal, árboles de regresión y random forest para regresión, evaluar cada modelo usando error cuadrático medio y R-cuadrado, y construir un sistema que permita ingresar características de un vehículo nuevo para obtener su precio estimado.

Ejercicio 4: Sistema de recomendación de productos

Una tienda online quiere recomendar productos a sus clientes basándose en sus características y preferencias históricas. Escriba un programa que genere datos de al menos doscientos clientes con información de edad, género, ciudad, frecuencia de compras mensual, monto promedio de compra, categorías de productos preferidas y si compró un producto específico recomendado. El programa debe preprocesar normalizando valores numéricos con StandardScaler y codificando variables categóricas usando one-hot encoding, entrenar múltiples clasificadores como regresión logística, árboles de decisión y random forest, comparar su desempeño y seleccionar el mejor modelo, y generar recomendaciones para nuevos clientes mostrando la probabilidad de que compren el producto usando predict_proba.

Ejercicio 5: Sistema de detección de fraude en transacciones

Una plataforma de pagos online necesita detectar transacciones fraudulentas automáticamente. Escriba un programa que cree un dataset desbalanceado simulando transacciones donde solo el cinco por ciento son fraudulentas, incluyendo características como monto de la transacción, hora del día, día de la semana, tipo de comercio, ubicación geográfica y si la transacción fue fraude o legítima. El programa debe analizar el desbalance de clases, preprocesar los datos normalizando y codificando variables categóricas, entrenar clasificadores como random forest, y evaluar usando métricas adecuadas más allá de precisión simple como recall, precision y F1-score enfocándose especialmente en la clase de fraude. Construya un sistema que clasifique transacciones nuevas mostrando el nivel de confianza en la detección.

Ejercicio 6: Modelo integrador de predicción de abandono de clientes

Una empresa de telecomunicaciones quiere predecir qué clientes tienen alta probabilidad de cancelar el servicio. Escriba un programa que genere un dataset realista con información de al menos trescientos clientes incluyendo datos demográficos como edad y género, datos del servicio como antigüedad y tipo de plan contratado, datos de uso como minutos consumidos y cantidad de llamadas al soporte técnico, datos financieros como monto de factura mensual, y si el cliente abandonó el servicio. El programa debe preprocesar manejando variables categóricas usando one-hot encoding, normalizando varia-

bles numéricas con StandardScaler y creando nuevas características relevantes, entrenar y comparar al menos tres modelos diferentes de Scikit-learn como regresión logística, árboles de decisión y random forest evaluándolos con múltiples métricas, generar análisis de importancia de características identificando qué factores más influyen en el abandono, y construir un sistema de predicción que calcule la probabilidad de abandono para clientes actuales usando predict_proba.

ACTIVIDAD 2

Redes neuronales con TensorFlow y Keras

En esta tercera y última actividad del módulo aplicaremos redes neuronales con TensorFlow y Keras para resolver problemas más complejos que requieren deep learning. El objetivo es integrar todo lo aprendido en el módulo construyendo soluciones completas desde la preparación de datos hasta el entrenamiento y evaluación de redes neuronales.

Para cada uno de los siguientes ejercicios deberá entregar un documento Word o PDF con el código Python completo y funcional, incluyendo capturas de pantalla de la ejecución mostrando diferentes casos de prueba para verificar que funcionan correctamente.

Recomendaciones

Las redes neuronales son herramientas poderosas, pero requieren cuidado en su diseño y entrenamiento. Experimente con diferentes arquitecturas modificando cantidad de capas y neuronas, pruebe distintas funciones de activación, ajuste hiperparámetros como tasa de aprendizaje y tamaño de batch. Monitoree el entrenamiento observando la evolución de la pérdida en cada época para detectar sobreajuste. Compare siempre el desempeño de redes neuronales con modelos tradicionales de Scikit-learn para determinar si la complejidad adicional está justificada por mejores resultados.

Esta actividad es ideal para trabajar con un/a compañero/a. Pueden debatir juntos la lógica de cada implementación y verificar que las estructuras se comporten correctamente en diferentes escenarios.

Ejercicio 1: Clasificador neuronal de tipos de iris

Construya un clasificador de redes neuronales usando el clásico dataset de flores iris. Escriba un programa que cargue el dataset iris de Scikit-learn que contiene mediciones de ciento cincuenta flores con cuatro características y tres especies diferentes. El programa debe preprocesar normalizando las características con StandardScaler y codificando las etiquetas con to_categorical para formato one-hot, construir una red neuronal con Keras usando Sequential que tenga al menos dos capas ocultas con funciones de activación ReLU y una capa de salida con softmax para las tres clases, compilar con optimizador Adam y función de pérdida categorical_crossentropy, entrenar monitoreando pérdida y precisión usando validation_split, evaluar el modelo en el conjunto de prueba calculando precisión final, y comparar el desempeño con un modelo de Scikit-learn como random forest para determinar si la red neuronal ofrece ventajas en este problema.

Ejercicio 2: Red neuronal para predicción de demanda de productos

Desarrolle un modelo de regresión con redes neuronales para predecir la demanda mensual de productos en un almacén. Escriba un programa que genere un dataset con información de al menos doscientos registros mensuales incluyendo características como mes del año, temperatura promedio del mes, eventos especiales como feriados o promociones, precio del producto, cantidad de publicidad invertida y demanda real de unidades vendidas. El programa debe preprocesar codificando variables categóricas con one-hot encoding y normalizando todas las características numéricas con StandardScaler, construir una red neuronal de regresión con al menos tres capas ocultas experimentando con diferentes cantidades de neuronas, compilar usando función de pérdida MSE y métricas como error absoluto medio, entrenar monitoreando pérdida usando validation_split para detectar sobreajuste, evaluar en conjunto de prueba calculando MSE y R-cuadrado y comparar con un modelo de regresión lineal de Scikit-learn como baseline.

Ejercicio 3: Clasificador binario de diagnóstico médico

Construya un sistema de clasificación binaria con redes neuronales para apoyar diagnósticos médicos. Escriba un programa que genere datos sintéticos simulando resultados de análisis clínicos de al menos quinientos pacientes con múltiples variables como edad, presión arterial, niveles de glucosa, colesterol, índice de masa corporal, frecuencia cardíaca, y otros indicadores de salud, junto con un diagnóstico binario de presencia o ausencia de una condición. El programa debe preprocesar normalizando todas las características con StandardScaler, construir una red neuronal con Keras experimentando con diferentes arquitecturas probando distintas cantidades de capas ocultas y neuronas, incluir técnicas de regularización como Dropout para prevenir sobreajuste, compilar con función de pérdida binary_crossentropy, entrenar usando validation_split, evaluar calculando precisión, recall, F1-score y analizando la matriz de confusión, y comparar la red neuronal con modelos de Scikit-learn justificando cuál es más apropiado para esta aplicación médica.

Ejercicio 4: Red neuronal profunda para clasificación multiclase

Desarrolle un clasificador complejo que distingue entre múltiples categorías usando arquitecturas profundas. Escriba un programa que trabaje con un dataset de al menos quinientas instancias distribuidas en cinco categorías diferentes, donde cada instancia tiene al menos veinte características numéricas. El programa debe preprocesar normalizando características con StandardScaler y codificando etiquetas con to_categorical para formato one-hot, construir una red neuronal profunda con al menos cuatro capas ocultas experimentando con diferentes cantidades de neuronas y funciones de activación ReLU, incluir capas de Dropout para prevenir sobreajuste, compilar con optimizador Adam y función de pérdida categorical_crossentropy, entrenar monitoreando pérdida y precisión usando validation_split para detectar sobreajuste, evaluar el modelo final calculando precisión y analizando la matriz de confusión, y comparar con modelos de Scikit-learn como random forest para generar conclusiones sobre cuándo justificar el uso de redes neuronales profundas.

Ejercicio 5: Sistema de predicción comparando modelos tradicionales y redes neuronales

Construya un sistema completo de machine learning comparando modelos tradicionales y redes neuronales. Escriba un programa que trabaje con un problema de clasificación binaria usando datos que simulen predicción de éxito académico de estudiantes, con al menos cuatrocientas instancias y quince características representando información académica como horas de estudio semanales, asistencia a clases, calificaciones previas, participación en actividades extracurriculares, nivel socioeconómico y si el estudiante aprobó o no la materia. El programa debe preprocesar normalizando con StandardScaler y creando características derivadas, entrenar múltiples modelos de Scikit-learn como regresión logística, árboles de decisión y random forest, construir varias arquitecturas de redes neuronales probando redes con dos, tres y cuatro capas ocultas variando la cantidad de neuronas, evaluar todos los modelos calculando precisión, recall, F1-score y analizando la matriz de confusión, comparar modelos usando múltiples métricas, y generar un reporte técnico con conclusiones fundamentadas sobre cuándo usar cada tipo de modelo.

Ejercicio 6: Proyecto integrador final de deep learning

Desarrolle un proyecto completo de machine learning de principio a fin integrando todo lo aprendido en el módulo. Elija uno de estos escenarios:

- Sistema de recomendación de productos basado en características de usuarios y productos.
- Predictor de demanda de energía eléctrica usando variables temporales y climáticas.
- Clasificador de riesgo de enfermedades cardíacas usando múltiples indicadores de salud.

El programa debe generar un dataset realista de al menos quinientas instancias con múltiples características relevantes, implementar un proceso completo de preprocesamiento que incluya normalización con StandardScaler, codificación de variables categóricas usando one-hot encoding o LabelEncoder según corresponda, e ingeniería de características creando variables derivadas útiles, entrenar múltiples modelos baseline usando Scikit-learn como regresión logística, árboles de decisión y random forest para establecer línea base de desempeño, diseñar y entrenar al menos tres arquitecturas diferentes de redes neuronales con Keras variando profundidad y complejidad, incluir técnicas de regularización como Dropout para prevenir sobreajuste, evaluar exhaustivamente todos los modelos usando múltiples métricas relevantes al problema, seleccionar el mejor modelo justificando técnicamente la decisión, implementar el modelo final con capacidad de hacer predicciones sobre datos nuevos, y generar un reporte ejecutivo completo documentando objetivos, metodología, resultados obtenidos, comparaciones entre enfoques y recomendaciones para mejora futura.